Orchestron User Documentation

Compiled from markdown documentation

Generated on 8 September 2026 at 09:14

Table of Contents

Orchestron User Documentation

This manual documents the current Orchestron user-facing functionality as implemented in the repository (frontend, backend API behavior visible to users, and feature history from the git log) as of **2026-09-07**, including the DAW audio-routing and persistent Perform mixer changes in commit `b487abd`.

It is organized by chapter and subchapter so you can read it front-to-back or jump directly to a workflow.

When Orchestron is served locally from the backend, opening `http://localhost:8000/` redirects to the client application at `/client`.

Scope

- Visual instrument design (opcode graph editor, formulas, GEN tables, compile/runtime testing)
- Live performance workflow (instrument rack, audio mixer, Master, inserts, sends, sequencers, piano rolls, MIDI controllers, import/export)
- Configuration and runtime behavior (audio engine settings, MIDI setup, browser-clock audio, persistence)

Reading Order

- Start with **Instrument Design** if you are creating patches.
- Continue with **Performance** to build and perform a multi-instrument setup.
- Use **Configuration** for audio engine tuning, MIDI setup, UI language/help, and browser-clock audio.

Instrument Design

Performance

Configuration

- Persistence and Defaults

Feature Coverage Map

Feature	Where It Is Documented
Multi-language UI (EN/DE/FR/ES)	GUI Language and Integrated Help
Integrated help pages and opcode docs modal	GUI Language and Integrated Help, Opcode Catalog and Integrated Documentation
Visual graph editor, zoom, selection, deletion	Graph Editor
Stereo audio interfaces, built-in patch starters and draft audition	Patch Toolbar, Graph Editor, Audition
Persistent audio mixer, Master, inserts, sends and routing repair	Audio Mixer and Routing
Input Formula Assistant (multi-input formulas + functions)	Input Formula Assistant
GEN meta-opcode editor (GEN01/GENpadsynth/etc.)	GEN Table Editor (GEN Meta-Opcode)
Instrument import/export bundles + CSD export	Instrument Import / Export and CSD Export
Performance rack, multi-instrument assignments, engine transport	Instrument Rack and Engine Transport
Melodic sequencers, pattern pads, pad looper, transposition	Melodic Sequencers and Step Editing, Pattern Pads, Queued Switching, and Pad Looper
Controller sequencers and CC curve editor	Controller Sequencers
Backend arpeggiator virtual instruments and presets	Arpeggiators
Piano rolls, scale/mode following, mixed-mode highlighting	Piano Rolls
Manual MIDI controller panel (up to 6 knobs)	MIDI Controllers
Performance bundle import/export with conflict resolution	Performance Import / Export
Offline performance render export (`.csd + .mid` ZIP or inline-score `.csd` ZIP)	Performance Import / Export
Audio engine settings (`sr` , `control_rate` , `ksmps` , buffers)	Audio Engine Settings (Config Page)
Browser audio mode (`browser_clock` PCM)	Browser-Clock Latency
Browser-clock latency tuning for unified browser audio	Browser-Clock Latency
App-state persistence and defaults	Persistence and Defaults
Supported opcode catalog (144 opcodes)	Supported Opcodes

File Format Quick Reference

- Instrument definition export: `.orch.instrument.json` or `.orch.instrument.zip`
- Performance export: `.orch.json` or `.orch.zip`
- Offline performance Csound export: `.csd.zip` (`Export CSD (MIDI)` includes a `.mid` ; `Export CSD (SCORE)` embeds notes and controller sweeps in the score)
- Csound export from Instrument Design: `.csd`
- ZIP exports are used automatically when referenced GEN01 audio assets are included.

Instrument Design

This chapter covers the complete **Instrument Design** workflow on the `Instrument Design` page.

What You Can Do Here

- Build instruments visually from Csound opcodes using the graph editor.
- Maintain multiple instrument tabs (parallel drafts or different patches).
- Use localized integrated help and opcode-level documentation without leaving the app.
- Choose built-in instrument, effect and output starters and define grouped stereo audio interfaces.
- Compile the current graph, inspect generated ORC, and audition drafts in isolation or in a temporary performance snapshot.
- Define advanced input-combine formulas when multiple signals feed the same input.
- Configure function tables with the `GEN` meta-opcode (including `GEN01` audio-file tables and `GENpadsynth`).
- Export instruments as Orchestron bundle files and as `.csd`.

Chapter Contents

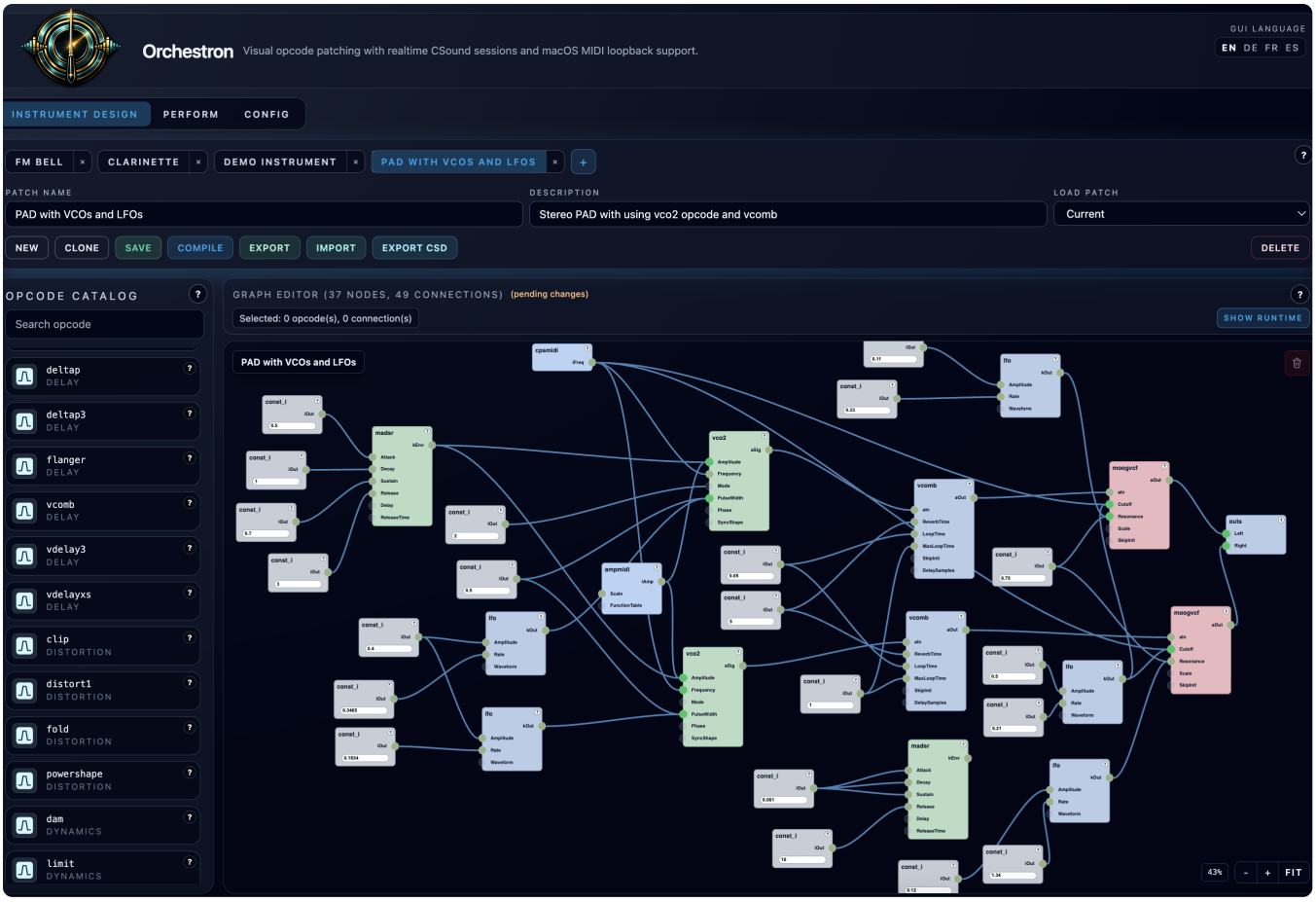
Recommended Workflow

1. Create or load a patch in the patch toolbar.
2. Add opcodes from the catalog (click or drag-and-drop).
3. Connect signals and set constant values.
4. If needed, define an Input Formula on a socket with multiple inputs.
5. Compile and inspect the runtime panel output.
6. Save the patch.
7. Export a `.csd` or an Orchestron instrument bundle.

Important Behavior

- The compile status badge is per active instrument tab/patch snapshot (`compiled`, `pending changes`, `errors`).
- The runtime panel can be collapsed to give more graph space and reopened with the `Show runtime` button.
- Saving a normal patch performs compile validation first; a failing compile prevents a bad save.
- Saving a patch marked `Template?` skips compile validation so incomplete starter graphs can be stored and reused. Template patches remain loadable in Instrument Design but are not selectable as Perform rack instruments.

Screenshots



Instrument Design page overview with catalog, graph editor, and runtime panel.

Patch Toolbar and Instrument Tabs

The patch toolbar is the main control surface for patch metadata, patch library operations, and instrument-level file actions.

Patch Toolbar Layout

The toolbar includes:

- Instrument tabs (multiple editable patch workspaces)
- Patch metadata fields (`Patch Name` , `Description`)
- Patch picker (`Load Patch`)
- Action buttons (`New` , `Clone` , `Delete` , `Save` , `Compile` , `Export` , `Import` , `Export CSD`)

Instrument Tabs

- Use the `+` button to add a new instrument tab.
- Each tab stores its own editable patch snapshot (graph + metadata).
- Tabs can be switched without losing the current edit state.
- Tabs can be closed with the `x` button.
- Tabs are persisted in app state (see Configuration chapter), so drafts can be restored after reload.

Patch Metadata

- `Patch Name` is the display name used in the patch library and performance rack dropdowns for normal patches.
- `Description` is a three-line patch note field and accepts up to 2048 characters.
- `Template?` marks the patch as a reusable starter graph. Template patches show a `TEMPLATE` token beside their name and are excluded from Perform rack instrument choices.
- `Activation: MIDI notes / Continuous` controls scheduling. Continuous patches run when added to the Perform rack and started. They may generate audio without an inlet. Receiving audio requires an actual inlet, independently of activation. Musical role is separate interface metadata.
- Metadata updates affect the current tab immediately, but they are not stored in the patch library until you save.

Patch Library Loading

- `Load Patch` loads an existing saved patch from the backend patch library into the active tab.
- The dropdown shows saved patch names, including template patches marked with `TEMPLATE` .
- Loading a patch replaces the active tab contents with that saved patch snapshot.

Actions

New

- Opens the built-in creation chooser: Playable instrument, Audio effect, Output / Master processor, or Empty patch.
- Creates an unsaved draft with the selected starter graph and default engine settings.
- This draft is not yet stored in the patch library until `Save` is used.

New from Template

- Opens the same chooser with the four built-in starters plus any saved templates.
- Creates a new unsaved normal patch with the chosen template's graph and description.
- Built-in choices remain available when the library has no saved templates.

Clone

- Creates a new saved patch by duplicating the current patch graph and metadata.

- Preserves the `Template?` and `Activation` flags from the source patch.
- If the name already exists, Orchestron generates a `(... copy)` style name.
- The cloned patch is loaded automatically after creation.

Delete

- Deletes the currently saved patch from the patch library.
- Requires confirmation.
- After delete, Orchestron loads another available patch or creates a new draft if none remain.

Compile

- Compiles the current instrument graph into generated Csound ORC/CSD artifacts.
- Updates compile status badge state.
- Does not save the patch or start audio automatically. Use Audition in the graph header to hear an unsaved draft.

Save

- Performs compile validation first for normal patches.
- If compile succeeds, persists the patch to the backend patch library.
- If compile fails, save is blocked and the patch remains unsaved.
- Template patches skip compile validation so incomplete starter graphs can be saved.
- Always-on patches still save as normal patch definitions; they run continuously only after they are added to a started Perform rack. A continuous source need not be an effect or have audio inlets.

Export / Import

- `Export` writes an Orchestron instrument definition bundle (`.orch.instrument.json` or `.orch.instrument.zip`).
- `Import` loads an instrument definition bundle and supports conflict handling (overwrite / rename / skip).
- Details are in Instrument Import / Export and CSD Export.

Export CSD

- Triggers compile and downloads the compiled `.csd` file.
- Useful for running the generated instrument outside Orchestron.

Practical Notes

- `Compile` is the fast iteration action while designing.
- `Save` is the persistence action for the patch library.
- `Export` is the sharing/transfer action for Orchestron-specific instrument bundles.
- `Export CSD` is the interoperability action for raw Csound usage.
- The integrated `?` help for this toolbar now focuses on tab-local draft state versus saved library state, plus the difference between `Compile`, `Save`, `Clone`, and export actions.

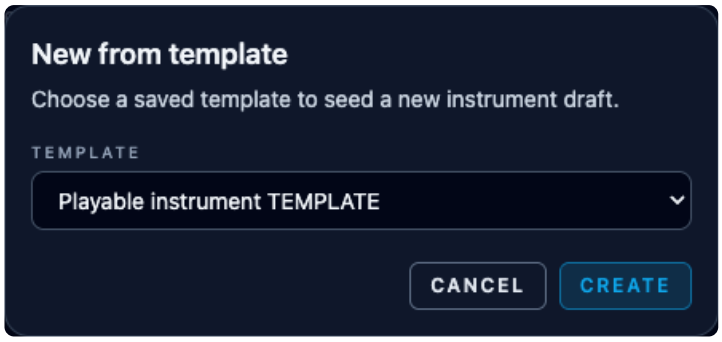
Built-in creation choices

New and New from template offer Playable instrument (MIDI pitch/velocity, sine oscillator and madsr), Audio effect (stereo pass-through), Output / Master processor (stereo input to direct output), and Empty patch. These choices work without saved templates. Guided instruments connect to an ordinary neutral Master when added to a performance.

Choose a starter

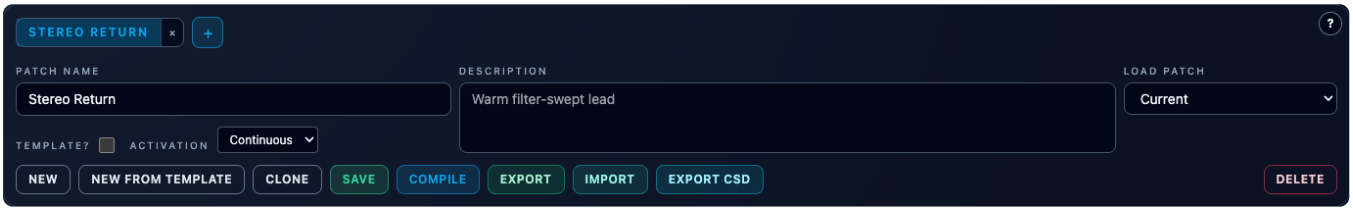
Choice	Activation and audio interface	Next step
Playable instrument	MIDI notes; named stereo output; pitch, velocity, sine oscillator and envelope	Save, add to Perform, and play its MIDI channel.
Audio effect	Continuous; named stereo input and output, initially pass-through	Add processing between the input and output before using it as an effect.
Output / Master processor	Continuous; stereo input to Direct Audio Output (`outs`)	Save for use as a Master/output processor.
Empty patch	Empty graph for custom design	Build and compile the required signal path.

Role describes the patch's audio interface; Activation determines when it runs. Naming a patch “Master” alone does not select it as the performance's Master. Use the Mixer Master selector or Create neutral Master.



New opens the starter chooser even when no saved templates exist.

Screenshots



Patch toolbar with a continuous stereo-effect draft, Activation selection and library actions.

Runtime Panel and Compilation Workflow

The Runtime panel supports live testing of the current patch and exposes compile/runtime diagnostics for the unified browser-clock runtime.

Runtime Panel Sections

- `MIDI Input` selector
- `Compile Output` (generated ORC)
- `Browser Audio` status for browser-clock mode
- `Session Events` log (recent backend/runtime events)

MIDI Input Binding

Use the `MIDI Input` dropdown to bind an external MIDI input to the active runtime session.

- This is how external hardware/DAW MIDI reaches the running instrument session.
- `internal:loopback` is always available as the built-in app-only loopback.
- Internal sequencers, piano rolls, and manual controller lanes always go straight to the session engine and do not depend on this binding.
- The selected input is session-specific and also reflected on the Performance page status footer.

See MIDI Setup and Inputs for OS-level MIDI setup guidance.

Compile Output (Generated ORC)

After compiling a patch, the panel shows the generated ORC text.

This is useful for:

- Verifying the compiled signal chain
- Understanding how formulas/GEN/meta-opcodes are rendered
- Debugging compile issues with concrete generated code

A compilable patch must contain at least one audio output sink: either `outs` for direct stereo output or `outleta` for named audio routing into another instrument. This allows source instruments to feed always-on effects without also writing directly to the final stereo output.

Session Events

The event list shows recent session events (most recent first, limited window in the UI).

Typical uses:

- Confirm that start/stop events happened
- Inspect runtime payloads and state transitions
- Diagnose issues during live testing

Browser Audio

The runtime now always uses `browser_clock` audio mode, so the Runtime panel can show:

- Browser PCM queue/runtime status (`connecting`, `live`, `error`)
- Error message when the controller socket or AudioWorklet path fails
- No `<audio>` player, because playback is owned directly by the browser `AudioContext`

See Browser-Clock Latency for the browser-clock mode guide and tuning workflow.

Runtime Panel Collapse / Show

To maximize graph editor space:

- Click `Hide` in the Runtime panel to collapse it
- Use `Show runtime` in the graph header to bring it back

This is especially useful on smaller displays or complex patches.

Compile Status Badge (Graph Header)

The instrument page header above the graph displays a compile-state badge for the current patch/tab snapshot:

- `compiled` - the current graph snapshot matches the last successful compile
- `pending changes` - the graph changed since the last successful compile
- `errors` - the last compile for this exact snapshot failed

This helps you avoid assuming a graph is live-valid after edits.

Recommended Test Loop

1. Edit the graph
2. Compile
3. Read compile output / diagnostics
4. Open Audition, choose a preview context, then Prepare audition
5. Use Stop and audition to hear the draft; close the preview when finished
6. Save only after compile is clean

The integrated `?` help for this panel now focuses on session-specific external MIDI binding, generated ORC inspection, runtime event feedback, and browser-clock PCM playback.

Audition a draft

Compile validates the graph without starting audio. Audition offers Isolated and In current performance. An isolated effect receives the built-in test instrument or a selected saved source. Current-performance audition replaces one selected rack instance in a temporary snapshot; choose the instance explicitly when a patch appears more than once.

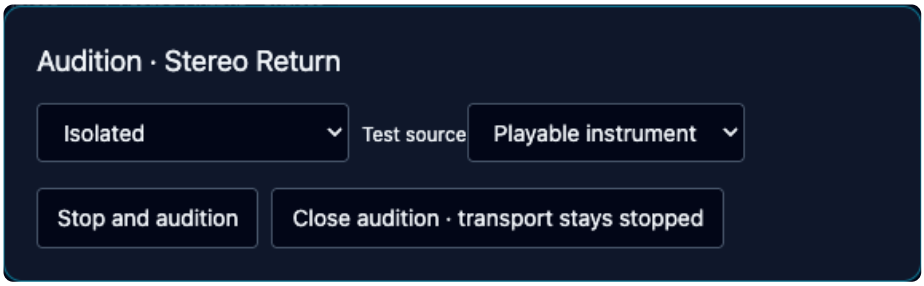
Prepare compiles the temporary preview before interrupting the performance. Stop and audition explicitly stops the original performance and starts the preview. Closing deletes the temporary session and restores the original performance state and playhead with transport stopped. Draft patches, test sources and preview routes are never saved into the library.

Preview step by step

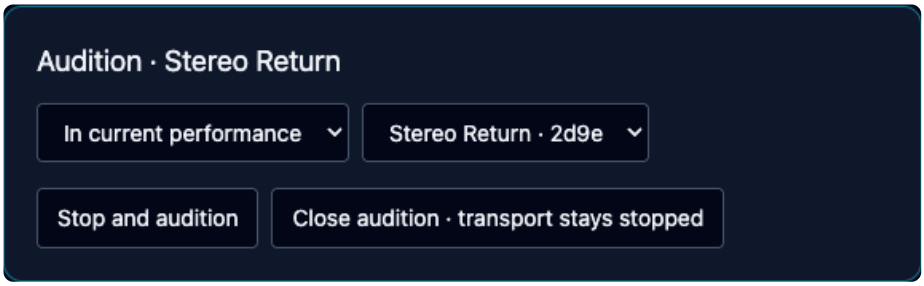
1. Click **Audition** in the graph header with the draft you want to hear.
2. Choose **Isolated** to test only this patch. For an effect, select **Test source**: the built-in Playable instrument or a saved source with suitable outputs. Isolated preview needs a stereo main output; source outputs must match the effect input count.
3. Alternatively choose **In current performance** and select the rack instance in **Source**. The short ID distinguishes instances of the same patch, including a return and a separate insert. The draft replaces only that selected instance in the temporary performance snapshot.
4. Click **Prepare audition**. Compilation errors appear in the dialog; fix the draft and prepare again. Preparation does not interrupt the original performance.
5. Click **Stop and audition** to stop the original engine and start the compiled preview. Note-triggered preview sources receive a test note (MIDI 60, velocity 96); current-performance mode also uses the performance sequencer configuration.

6. Click **Close audition · transport stays stopped**. The preview session is deleted and the original performance state/playhead is restored with transport stopped. Restart the performance explicitly when ready.

If you edit the draft after preparing, prepare again before listening. Preview is temporary: use Save separately to update the library patch.



Isolated effect preview after successful preparation, ready for Stop and audition.



Current-performance preview targets one explicitly selected rack instance.

Screenshots

Runtime panel showing MIDI input selection, generated ORC, active browser-owned PCM runtime, queue diagnostics and recent session events.

MIDI Setup and Inputs

This page covers practical MIDI setup and how external MIDI binding works in Orchestron's unified browser-clock runtime.

External MIDI Input Binding In Orchestron

External MIDI input is bound in the **Instrument Design Runtime panel** (not on the Config page).

Workflow:

1. Open Instrument Design.
2. Use the Runtime panel `MIDI Input` dropdown.
3. Select the desired input:

- `internal:loopback` for the built-in app-only loopback and default fallback binding
- a `host_bridge` input published by `host-midi-helper` for external hardware or DAW MIDI

1. Start the instrument session and play from your hardware/DAW.

The selected MIDI input is reflected in the Perform page footer status line.

Internal Performance Controls Do Not Require External MIDI

These features generate MIDI/control events internally and do not require an external MIDI input device or host bridge:

- melodic sequencers
- controller sequencers
- arpeggiators
- piano rolls (on-screen keyboard)
- manual MIDI controller knob lanes

External MIDI input is only needed when you want to play Orchestron from outside the app. Internal MIDI is always delivered directly into the session engine and ignores the external input binding.

That means:

- switching the session input does not disable sequencers, arpeggiators, piano rolls, or manual controller lanes
- binding a helper-provided external input adds another source into the same engine scheduler

External MIDI Bridge

The backend no longer relies on direct OS MIDI enumeration for session playback. External MIDI arrives through the optional Rust host bridge in `host-midi-helper/`.

General setup pattern:

1. Start the backend with `VISUALCSOUND_HOST_MIDI_TOKEN` set.
2. Start `host-midi-helper` on the same host OS with the same token.
3. Bind the helper-published input in the Runtime panel.

If no helper is running, `internal:loopback` remains available and internal performance tools still work.

The backend still performs backend-native MIDI enumeration where available, but live session playback now uses the helper-backed `host_bridge` path for external devices.

macOS Loopback Setup (IAC Driver)

For DAW/app -> Orchestron MIDI routing on macOS:

1. Open **Audio MIDI Setup**.
2. Open **MIDI Studio**.
3. Open **IAC Driver**.
4. Enable **Device is online**.
5. Add/select an IAC bus (for example `IAC Driver Bus 1`).
6. Route your DAW/app MIDI output to that IAC bus.
7. Select that input in Orchestron's Runtime panel.

Troubleshooting MIDI Inputs

No External MIDI Inputs Listed

- Ensure your OS/device/virtual bus is enabled (for example IAC Driver on macOS, ALSA/JACK route on Linux, loopMIDI on Windows).
- Confirm `host-midi-helper` is running and uses the same token as the backend.
- Restart the helper after changing system MIDI configuration.

Perform Page Footer Shows `midi input: internal:loopback`

- Internal app MIDI is working.
- External hardware/DAW MIDI will not reach the session until a helper-provided input is bound.

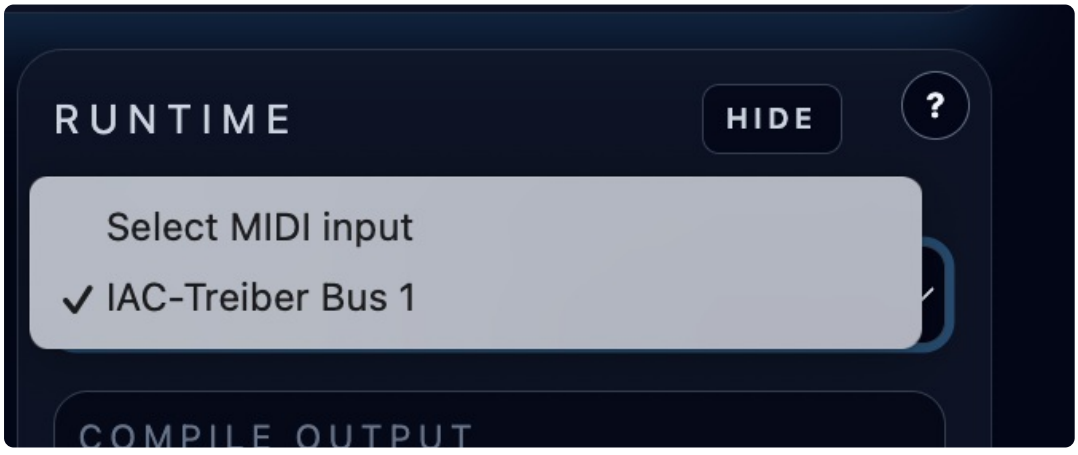
Sequencer/Piano Roll Works But Hardware MIDI Does Not

- This usually means internal controls are working, but no external helper-provided MIDI input is bound.

Related Pages

- Runtime Panel and Compilation Workflow
- Live Status and Safety Controls

Screenshots



Runtime panel MIDI input dropdown expanded with available MIDI inputs.

Live Status

The Perform page includes several status for live operation.

Browser Audio Health

The Runtime panel shows queued PCM and pending-render milliseconds, underrun and overrun counters, and the latest backend render-time/audio-time percentage while a browser-clock session is active.

Audio refills and PCM writes run in a Dedicated Worker. Sequencer step and pad updates are carried with the rendered PCM and applied only when their target frame becomes audible. If the Perform page is hidden, visual animation pauses and resumes from the current audible snapshot instead of replaying missed steps.

Performances are limited to 128 flattened backend note tracks. Each drummer row becomes one backend note track, so the total includes melodic sequencers plus all drummer rows.

Footer Status Bar

At the bottom of the Perform page, Orchestron shows live status values such as:

- `playhead` position (`current step / step count`)
- `cycle` count
- active `midi input` name (for example `internal:loopback` or a helper-provided device)

This footer is useful during performance because it gives quick feedback without opening the Instrument Design runtime panel.

Track State Labels (Queued Changes)

A track's state badge can reflect queued actions that apply at the next cycle boundary, for example:

- starting @ step 1
- stopping @ step 1

This helps you understand why a track button press did not take effect immediately while transport is running.

Error Banner (Transport / Runtime Errors)

The Perform page displays an error banner for problems such as:

- no active instrument session when trying to start sequencers
- trying to play piano roll notes before the instrument engine is ready
- pad queue request failures
- controller send failures
- sequencer config sync failures

MIDI Input Reference

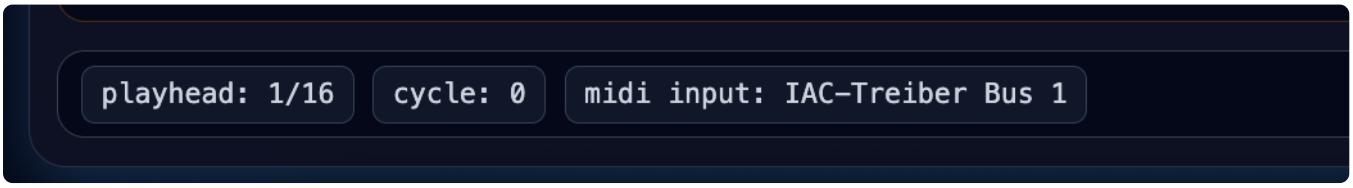
The footer's MIDI input display reflects the session's bound external MIDI input (selected in the Instrument Design Runtime panel). If it shows `internal:loopback`, internal app MIDI is still active, but external hardware/DAW MIDI will not reach the session until a helper-provided input is bound.

See Runtime Panel and Compilation Workflow and MIDI Setup and Inputs.

Live Performance Safety Checklist

1. Confirm instrument session is running.
2. Confirm expected MIDI input is bound.
3. Check track/controller/piano-roll enable states.
4. Save the performance after major routing changes.

Screenshots



Footer status bar with playhead/cycle/MIDI input.

Browser-Clock Latency

Orchestron now uses backend audio mode `browser_clock` on every supported platform. The browser becomes the playback clock and requests PCM render chunks from the backend, so the most important latency and stability controls live in the Config page's `Browser-Clock Latency` section.

When This Section Appears

The `Browser-Clock Latency` section is shown when the backend is running in `browser_clock` mode, for example:

- `docker compose up --build`
- CLI argument `--audio-output-mode browser_clock`
- environment variable `VISUALCSOUND_AUDIO_OUTPUT_MODE=browser_clock`

This is the standard runtime path for macOS, Linux, Windows, and Docker. In practice, that means this section describes the normal live-audio path of the app, not a Docker-only special case.

Why These Settings Matter

On the browser-clock path, audio is not pushed to a local DAC on the backend host. Instead:

- the backend renders PCM blocks on demand
- a dedicated browser audio worker keeps the target queue of decoded PCM and requests more audio
- the AudioWorklet consumes that queue independently of React rendering

That makes the browser-clock queue settings the main tradeoff between low live-play latency and glitch resistance.

The queue producer no longer runs on the React/main thread. Main-thread stalls can delay visual updates, but they do not stop the worker from requesting or buffering PCM. Playback starts only after the startup high-water target is reached. The Runtime panel reports current queued and pending-render milliseconds, underrun/overrun counts, and the latest backend render-time/audio-time ratio.

The backend suppresses Csound performance-time logging by default in headless browser-clock sessions. This prevents stdout or Docker logging I/O from delaying `performKsmps()`. To diagnose Csound's own runtime messages, start the backend with `VISUALCSOUND_CSOUND_PERFORMANCE_LOGGING=true`; this is intended as a temporary diagnostic setting rather than a latency control.

Lower values usually feel more immediate, but they also leave less headroom for:

- backend scheduling jitter
- network / websocket round-trip jitter between browser and backend
- browser tab throttling or temporary main-thread stalls
- heavier patches that take longer to render

Higher values add safety margin, but manual playing and note feedback feel less direct.

Main Control Groups

Queue Targets

- `Steady Low Water` / `Steady High Water` define the normal browser PCM queue window after startup has stabilized.
- `Startup Low Water` / `Startup High Water` define the larger queue window used while the stream is priming or recovering.

If you hear underruns after connect or after transport restarts, start by increasing the steady or startup queue targets slightly.

Recovery Headroom

- `Underrun Recovery Boost` adds extra queue headroom after an underrun is detected.
- `Max Underrun Boost` limits how large that temporary recovery buffer may grow.

These controls help the browser recover cleanly from brief dropouts without forcing a permanently larger steady-state latency.

Render Request Size And Parallelism

- `Max Blocks Per Request` limits how much audio the browser asks for in one render request.
- `Steady Parallel Requests`, `Startup Parallel Requests`, and `Recovery Parallel Requests` control how many render requests may stay in flight in each state.

Smaller requests can improve responsiveness, while larger or more parallel requests can make it easier to keep the queue full on slower systems.

Live Note Responsiveness

- `Immediate Note Render Blocks` sends a small urgent render burst after a live note-on event.
- `Immediate Note Render Cooldown` limits how often those urgent note-triggered bursts may happen.

These are the most direct controls for making manual keyboard or MIDI playing feel snappier in browser-clock mode.

Manual MIDI Safety Limits

Browser-clock manual MIDI is bounded on the backend even when events are routed through an arpeggiator. Deployments can tune the maximum accepted future timestamp with `VISUALCSOUND_BROWSER_CLOCK_MANUAL_MIDI_MAX_FUTURE_MS`, the accepted manual MIDI rate with `VISUALCSOUND_BROWSER_CLOCK_MANUAL_MIDI_RATE_PER_SECOND` and `VISUALCSOUND_BROWSER_CLOCK_MANUAL_MIDI_BURST`, and the per-session arpeggiator pending input cap with `VISUALCSOUND_ARPEGGIATOR_PENDING_INPUT_MAX_EVENTS`.

Session Resource Limits

Runtime sessions allocate backend Csound worker resources, so session creation and live session-event observers are also bounded. `VISUALCSOUND_SESSION_MAX_ACTIVE` limits total active sessions, `VISUALCSOUND_SESSION_MAX_ACTIVE_PER_CLIENT` limits sessions owned by one client address, `VISUALCSOUND_SESSION_CREATE_RATE_PER_MINUTE` and `VISUALCSOUND_SESSION_CREATE_RATE_BURST` limit repeated creation attempts, and `VISUALCSOUND_SESSION_IDLE_TIMEOUT_SECONDS` deletes stopped idle sessions so capacity is returned automatically. Session event WebSocket observers are capped with `VISUALCSOUND_SESSION_EVENT_WS_MAX_SUBSCRIPTIONS_TOTAL` and `VISUALCSOUND_SESSION_EVENT_WS_MAX_SUBSCRIPTIONS_PER_SESSION`, while connection attempts are rate-limited with `VISUALCSOUND_SESSION_EVENT_WS_CONNECT_RATE_PER_MINUTE` and `VISUALCSOUND_SESSION_EVENT_WS_CONNECT_RATE_BURST`.

Practical Tuning Order

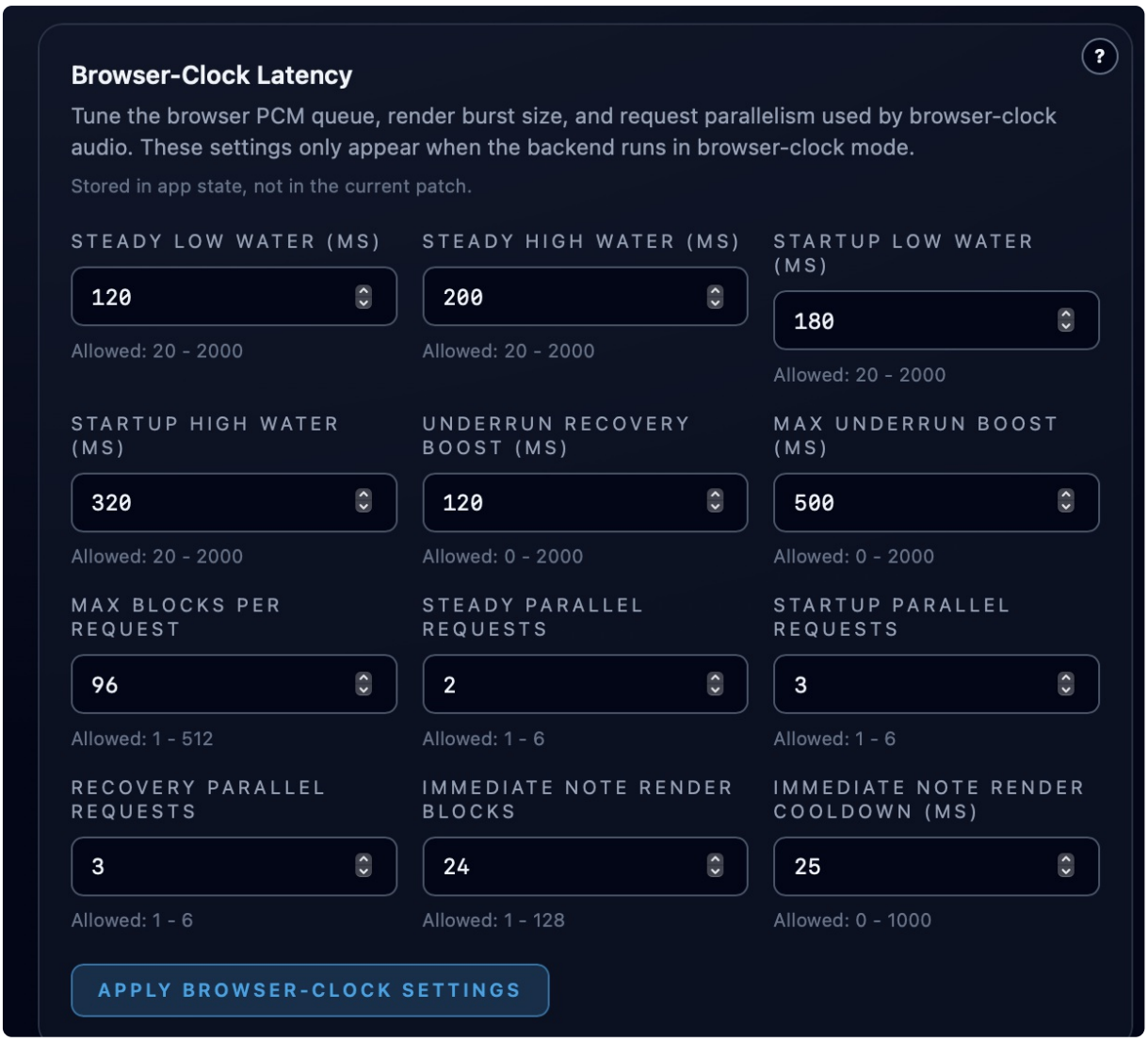
1. Start from the defaults shown by the Config page.
2. If the stream crackles or underruns, raise `Steady High Water` and `Startup High Water` first.
3. If latency still feels too high, lower the steady queue targets cautiously.
4. If live note response feels sluggish, reduce `Max Blocks Per Request` or adjust the immediate-note controls in small steps.
5. Apply the settings and test again with a realistic patch load.

Keep in mind that these settings are stored in app state for the current workspace. They are not saved into the patch itself.

They also work together with the engine settings from the patch:

- browser-clock settings tune queueing and request behavior
- patch engine settings such as `sr`, `ksmps`, `-b`, and `-B` tune the backend render block geometry

Screenshot



The Config page's Browser-Clock Latency section, used when Orchestron runs headless in `browser_clock` mode and the browser owns audio playback.

Instrument Import / Export and CSD Export

This page documents all user-facing import/export options available from the Instrument Design page.

Instrument Export Types

1. Orchestron Instrument Bundle (`Export` button)

The `Export` button creates an Orchestron instrument definition bundle containing the patch graph and metadata.

File extensions:

- `.orch.instrument.json` (JSON bundle)
- `.orch.instrument.zip` (ZIP bundle)

JSON vs ZIP Export Selection

Orchestron chooses the format automatically:

- JSON export when no referenced uploaded GEN01 or `sfload` assets are present
- ZIP export when the instrument references uploaded GEN01 audio assets or `sfload` SoundFont assets (so audio dependencies can be included)

2. Csound CSD Export (`Export CSD` button)

The `Export CSD` button compiles the patch and downloads a `.csd` file.

Use this when you want to run the generated instrument outside Orchestron in a Csound workflow.

What Is In An Instrument Bundle

An instrument bundle contains:

- patch name
- patch description
- template flag
- always-on flag
- schema version
- full patch graph (`nodes`, `connections`, `ui_layout`, `engine_config`)

If a ZIP bundle is required, it also contains referenced uploaded audio/SoundFont assets under an `audio/` folder in the archive.

Instrument Import (`Import` button)

The import button accepts:

- standalone instrument bundle JSON
- standalone instrument bundle ZIP
- performance bundle JSON/ZIP (to extract included patch definitions)

This means you can import patches directly from a performance export without manually splitting files.

Conflict Resolution (Patch Name Already Exists)

When importing one or more patch definitions and a patch name already exists, Orchestron opens a conflict dialog.

Per patch, you can choose:

- `Overwrite` (update existing patch)
- `Rename` (disable overwrite and provide a new target name)
- `Skip` (patch imports only)

The dialog validates name collisions before proceeding.

Audio Dependency Handling

For ZIP imports that include referenced GEN01 audio or `sfload` SoundFont assets:

- Orchestron validates the archive structure
- Imports the audio files into backend storage automatically
- Restores the patch graph with the asset references intact
- Rejects archives that exceed backend bundle limits before unpacking large members

Advanced Format Notes (for power users)

- Instrument ZIP exports contain exactly one root JSON file plus optional `audio/...` entries
- Missing referenced audio assets in a ZIP import cause import failure (prevents broken GEN01 references)
- Default import limits are 256 MiB compressed request body, 8 MiB root JSON, 512 ZIP entries, and 256 MiB total uncompressed ZIP content
- Deployments can tune these with `VISUALCSOUND_BUNDLE_IMPORT_MAX_BYTES`, `VISUALCSOUND_BUNDLE_IMPORT_JSON_MAX_BYTES`, `VISUALCSOUND_BUNDLE_IMPORT_ZIP_MAX_MEMBERS`, and `VISUALCSOUND_BUNDLE_IMPORT_ZIP_MAX_UNCOMPRESSED_BYTES`
- Individual imported audio/SoundFont assets are still limited by `VISUALCSOUND_GEN_AUDIO_ASSET_MAX_BYTES`

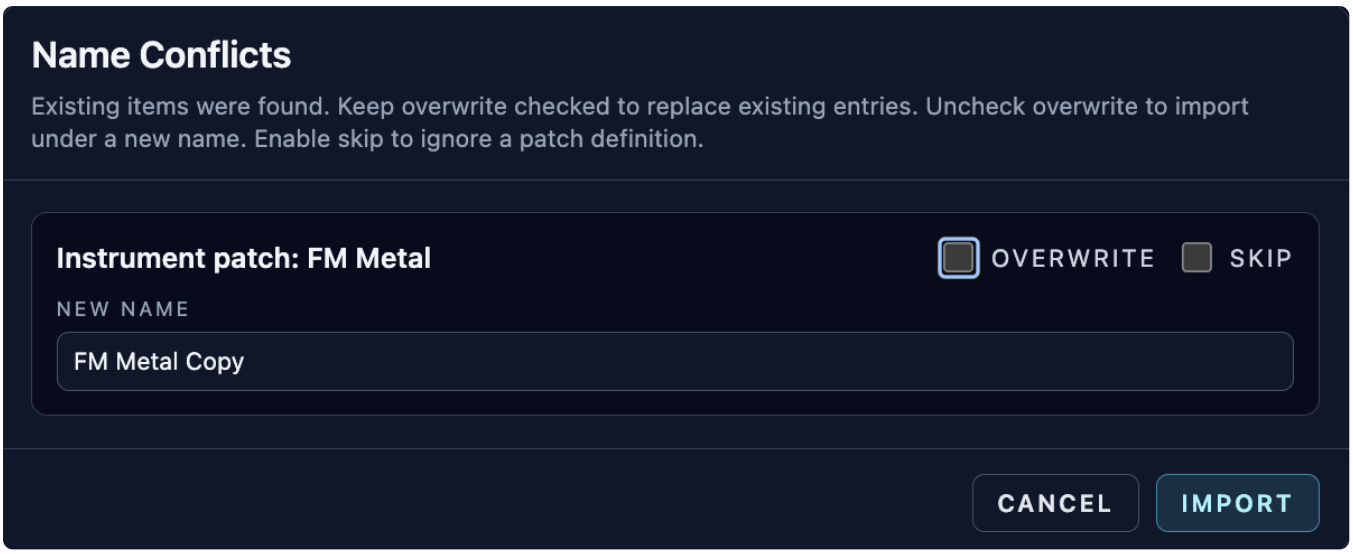
Instrument Bundle vs Performance Bundle

Use the right export for the right task:

- Instrument bundle: share one patch design
- Performance bundle: share a full performance setup (instrument rack + sequencers + controllers + piano rolls + optional patch definitions)

See Performance Import / Export for performance bundles.

Screenshots



Patch import conflict dialog with overwrite/rename/skip decisions.

Performance Import / Export

Performance import/export lets you move complete live setups between machines or save versioned backups.

Performance Export (`Export`)

The Performance page `Export` button creates an Orchestron performance bundle from the current performance workspace.

Use this export when you want to:

- Re-import the performance into Orchestron later
- Move a full performance setup to another machine
- Keep versioned backups of a performance together with its patch references

File extensions:

- `.orch.json`
- `.orch.zip`

What Gets Exported

A performance export includes:

- Performance metadata (`name`, `description`)
- Sequencer/drummer-sequencer/arpeggiator/piano-roll/controller/controller-sequencer configuration snapshot
- Instrument assignments with stable instance IDs
- Explicit audio routes, Master selection, insert ownership, strip gain/balance/mute/solo, and send level/pre-post settings
- Referenced patch definitions for the instruments currently assigned in the performance rack
- Patch names (included in the snapshot for easier remapping on import)

If any referenced patch contains uploaded GEN01 audio assets or `sfload` SoundFont assets, export is automatically produced as a ZIP and includes those assets.

Offline Render Export (`Export CSD (MIDI)` / `Export CSD (SCORE)`)

The Performance page provides two offline Csound render exports:

- `Export CSD (MIDI)` creates the traditional ZIP with a compiled CSD and a separate MIDI file.
- `Export CSD (SCORE)` creates a ZIP with the performance notes and controller sweeps embedded as Csound score events.
- Both CSD export modes seed enabled manual MIDI Controller lane values at time 0 on every assigned instrument channel.

This export is different from the normal `Export` bundle:

- `Export` is for Orchestron import/export workflows
- `Export CSD (MIDI)` and `Export CSD (SCORE)` are for rendering outside Orchestron with standard Csound tools

The MIDI ZIP contains:

- A compiled `.csd` with every non-template instrument currently used in the performance rack
- Offline render settings forced to `sr = 48000` and `ksmps = 1`
- WAV output written as 32-bit float (`-f`) so exported files preserve headroom instead of baking clipping into 16-bit samples
- Always-on effect instruments started with Csound `alwayson`
- A finite `f 0 ...` score duration sized for the exported arranger playback plus a release-tail buffer

- The arranger playback rendered as a `.mid` file from beginning to arrangement end
- Enabled manual MIDI Controller lane values written into the MIDI file at tick 0 on every assigned instrument channel
- Uploaded bundled sample audio / SoundFont files used by the exported instruments
- A `README.txt` with the exact Csound command line needed to render the package

The SCORE ZIP contains the same compiled instruments and bundled assets, but omits the `.mid` file. Instead, it embeds note events as score `i` statements, embeds controller sequencer sweeps and enabled manual MIDI Controller lane values as score-controlled CC setter events, and rewrites supported MIDI opcodes such as `cpsmidi`, `ampmidi`, `midi_note`, `notnum`, and `midictrl` for score playback. If export-time approximations are needed, such as best-effort `ampmidi` function-table mapping, the ZIP includes `WARNINGS.txt` and the README lists the warnings.

Only assets stored through Orchestron's upload/import flow are bundled. GEN01 and `sfload` nodes must reference uploaded assets; compile, session start, and offline performance CSD export reject raw filesystem `samplePath` values instead of passing them to Csound.

Offline performance CSD export is bounded before synthesis starts: looping playback is rejected, playback ranges are limited to 65,536 transport steps, a single step can carry at most 16 notes, and the estimated MIDI event budget is limited to 200,000 events. MIDI generation also stops if it exceeds the event budget or takes more than 5 seconds.

ZIP layout:

- The ZIP contains a single top-level directory
- That directory has the same basename as the exported performance `.csd`
- Inside the MIDI export directory you will find the `.csd`, `.mid`, `README.txt`, and `assets/` subdirectory
- Inside the SCORE export directory you will find the `.csd`, `README.txt`, optional `WARNINGS.txt`, and `assets/` subdirectory

Typical extracted structure:

```
Offline_Export/
  Offline_Export.csd
  Offline_Export.mid
  README.txt
  assets/
```

Typical SCORE structure:

```
Offline_Export/
  Offline_Export.csd
  README.txt
  WARNINGS.txt
  assets/
```

Typical use:

- Share a performance as a portable offline render package
- Render the arrangement outside Orchestron with stock Csound
- Archive a self-contained `.csd` + `.mid` + assets bundle, or a single inline-score `.csd` + assets bundle, for later mastering or batch rendering

Rendering workflow:

1. Click `Export CSD (MIDI)` or `Export CSD (SCORE)` on the Performance page.
2. Extract the downloaded ZIP.
3. Change into the bundled directory inside the extracted archive.
4. Run the command from `README.txt`, for example the MIDI export command:

```
csound -d -W -f -o Offline_Export.wav -F Offline_Export.mid Offline_Export.csd
```

For SCORE exports the command omits `-F` :

```
csound -d -W -f -o Offline_Export.wav Offline_Export.csd
```

The exported `.csd` already includes matching `CsOptions` , so `csound Offline_Export.csd` also works after you change into that directory.

Performance Import (`Import`)

The Performance page `Import` button supports two categories of files:

Bundle imports are bounded by backend request and ZIP limits before large archive members are decompressed. Defaults are 256 MiB compressed request body, 8 MiB import JSON, 512 ZIP entries, and 256 MiB total uncompressed ZIP content. Tune these with `VISUALCSOUND_BUNDLE_IMPORT_MAX_BYTES` , `VISUALCSOUND_BUNDLE_IMPORT_JSON_MAX_BYTES` , `VISUALCSOUND_BUNDLE_IMPORT_ZIP_MAX_MEMBERS` , and `VISUALCSOUND_BUNDLE_IMPORT_ZIP_MAX_UNCOMPRESSED_BYTES` ; individual bundled audio/SoundFont assets still use `VISUALCSOUND_GEN_AUDIO_ASSET_MAX_BYTES` . Persistent generated-asset storage is capped by `VISUALCSOUND_GEN_AUDIO_ASSETS_MAX_TOTAL_BYTES` and `VISUALCSOUND_GEN_AUDIO_ASSETS_MAX_COUNT` ; unreferenced generated assets older than `VISUALCSOUND_GEN_AUDIO_ASSET_GC_MIN_AGE_SECONDS` are eligible for garbage collection.

1. Full Orchestron Performance Export (recommended)

If the file matches the performance export format, Orchestron opens an import options dialog.

2. Raw/legacy sequencer snapshot JSON

If the file is not a full performance export bundle, Orchestron attempts to apply it directly as a sequencer configuration snapshot.

This is useful for advanced workflows and backward compatibility.

Import Options Dialog (Performance Bundle)

When importing a full performance bundle, you can choose whether to import:

- `performance`
- `patch definitions` (if present in the bundle)

You may import either or both.

Typical uses:

- Import both (full environment restore)
- Import only patch definitions (extract instruments from a performance file)
- Import only performance (if the destination machine already has matching patches)

Conflict Resolution Dialog

If names already exist, Orchestron opens a conflict dialog.

For Patches

Per patch definition you can choose:

- Overwrite existing patch

- Rename and create a new patch
- Skip importing that patch

For Performance

For the performance itself you can choose:

- Overwrite existing performance
- Rename and create a new performance

(Performance entries do not use `Skip` in the conflict dialog; skipping is handled via the import options step.)

Patch ID Remapping During Performance Import

Performance bundles store instrument patch references. On import, Orchestron remaps patch IDs using:

1. Imported patch definitions (source ID -> newly created/updated destination patch ID)
2. Existing patch name matches (when patch definitions are not imported but patch names match)

This makes performance imports more portable across machines and patch databases.

Import Validation (Resolvable Instruments Required)

A performance import fails if no instrument assignments can be resolved to available patches.

In that case:

- import patch definitions from the bundle, or
- create/match patches locally by name first

Related Workflows

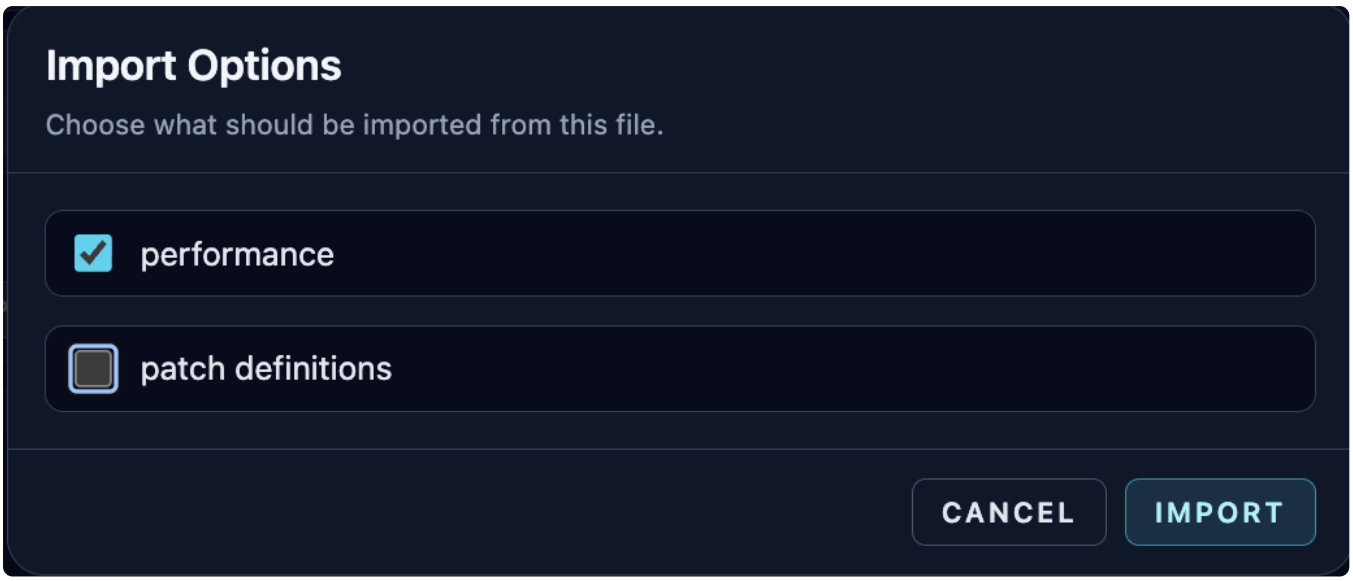
- Instrument bundle import/export is documented in Instrument Import / Export and CSD Export.
- You can also import patch definitions from a performance bundle on the Instrument Design page.

Mixer snapshot

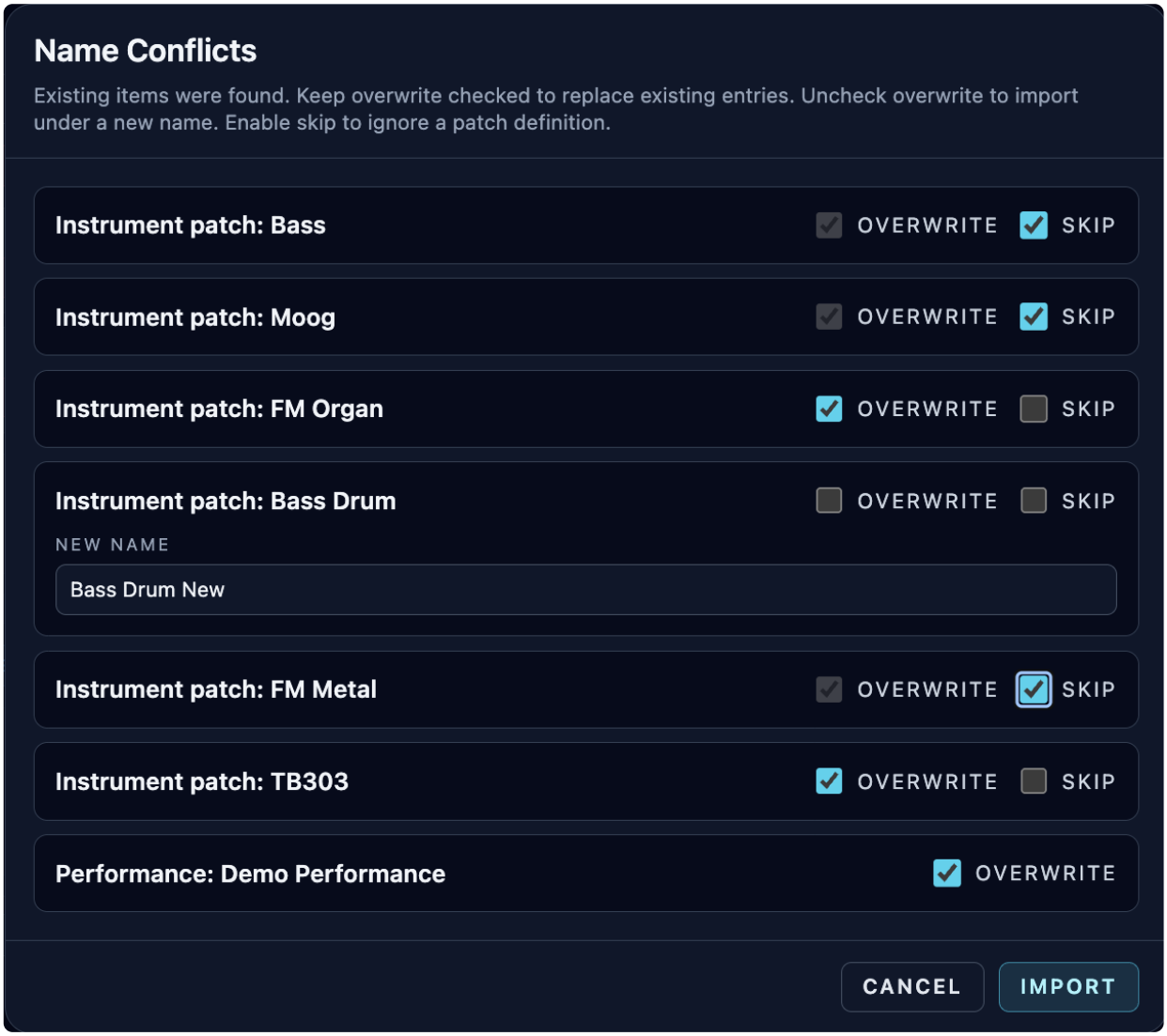
Native bundles and both performance CSD modes capture version 11 routing and mixer settings: gain, pan/balance, mute/solo, sends including pre/post, Master and inserts. Export waits for outstanding live mixer acknowledgments and reports synchronization failures. Mixer values are initialized inside each CSD and require no control client. MIDI and SCORE notes retain their authored velocities; SCORE instrument references come from the compiler manifest.

Offline rendering remains 48 kHz with ksmps=1, float WAV output, packaged assets and release tails. Equivalent routing and settings are guaranteed, not sample-identical live/offline waveforms. Finite exports without MIDI notes are supported when a routed continuous source can produce audio. Standalone patch exports remain independent of Perform settings. See Audio mixer and routing for migration and signal flow.

Screenshots



Performance import options dialog for importing performance and/or patch definitions.



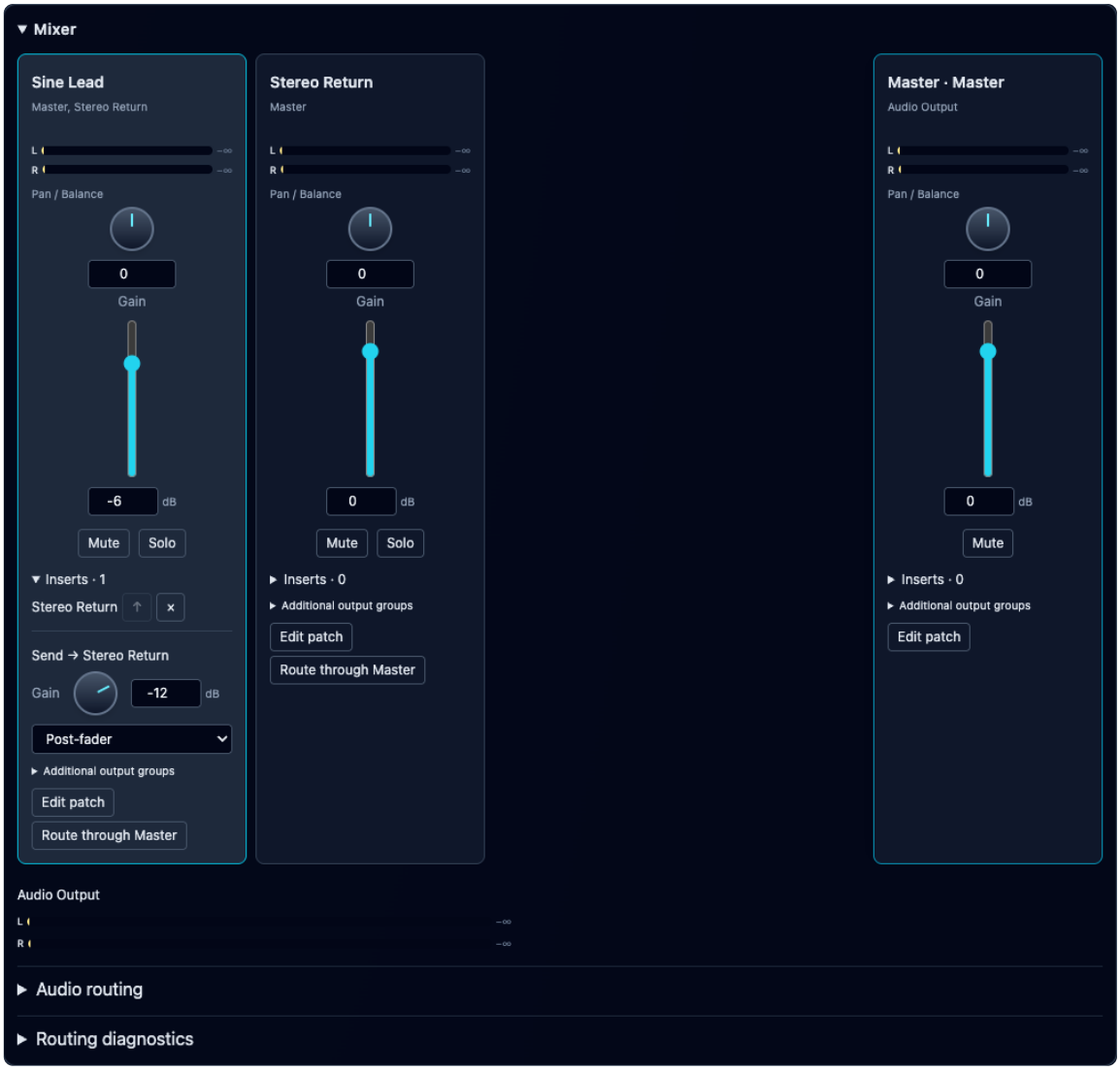
Conflict dialog for imported patch/performance name collisions.

Audio mixer and routing

The collapsible mixer sits below the instrument rack. Select a strip to edit its outputs and sends. The pinned Master controls audio routed into it. A **Direct Audio Output** badge identifies paths that bypass Master; these still respond to their own strip's fader, balance, mute and solo.

Build your first mix

1. In Instrument Design, choose **New → Playable instrument**, name the draft and **Save** it. Create and save an **Audio effect** when you need a processor. The built-in effect is a stereo pass-through starter; add your processing inside its graph.
2. In Perform, use **Add Instrument** and select the saved instrument. Set its MIDI channel. Guided instruments connect to an ordinary neutral Master automatically. For existing patches, inspect their destination labels and use **Create neutral Master** / **Route through Master** if needed.
3. Expand **Mixer** below the rack. Each strip header selects its source for the routing editor. The Master strip stays pinned at the right; scroll horizontally when the rack has many strips.
4. Start with 0 dB gain and centered Pan / Balance. Build routes and insert chains while stopped, then open **Routing diagnostics → Check routing**.
5. Click **Start Instruments** and play via a piano roll, sequencer or MIDI input. Change gain, balance, mute/solo and existing send settings while listening. Watch strip meters and the final Audio Output meter.
6. Use **Save Performance** to retain this mix. Stop the engine before adding/removing routes, replacing assignments or reordering inserts.



A stopped example mix: Sine Lead at -6 dB, a -12 dB post-fader send to Stereo Return, and pinned Master. The insert uses a separate instance of the same pass-through patch; no effect processing is implied by its name.

Signal flow

Path	Processing order
Instrument	Summed voices → Perform inserts → pan/balance → fader → destination
Pre-fader send	After inserts → send amount → destination
Post-fader send	After pan/balance and fader → send amount → destination
Return	Summed inputs → effect patch → inserts → balance → return fader
Master	Incoming routes → Master patch/inserts → balance → Master fader → Audio Output

Effects drawn inside an instrument graph run per voice. Perform inserts receive summed voices. Each inserted effect is a dedicated rack instance. Reordering an insert changes explicit connections; if custom connections no longer form a simple chain, **Custom routing** preserves those connections.

Additional output groups retain their own port names and mappings. They share the owning strip's gain and mute; inserts process the designated main output. Custom routes can express other processing arrangements.

Inserts versus shared returns

An **insert** processes one strip in series after its voices are summed. Select that strip, choose a saved continuous patch with audio inputs and outputs in the **Inserts…** selector below the routing controls, then click the adjacent **Add**. Open **Inserts** on the strip to inspect its chain. Click a processor name to edit its patch, ↑ to move it earlier, or × to remove that insert instance. Reusing one saved effect creates separate processing instances; an insert does not appear as a separate mixer strip.

A **return** is a continuous rack instance that can receive sends from several sources and has its own mixer strip. Add it with **Add Instrument**, connect its output to Master, then add sends from the source strips. For a parallel reverb/delay arrangement, design the return patch to output the desired wet signal; the starter pass-through would merely add another copy of the dry signal.

Add a stereo send

1. Stop the engine. Add the saved effect as a rack instance and use its **Route through Master** button to connect its main output.
2. In **Audio routing**, choose the instrument under **Source** and the return under **Destination**.
3. Leave **Stereo** checked and verify **Exact channel mapping**, for example `left / right → left / right`. Choose **Send** in the Route selector and click the routing row's **Add**.
4. On the source strip, find **Send → [return name]**. New sends start at silence; enter a value such as -12 dB to hear the return.
5. Choose **Post-fader** when the send should follow the source's fader/balance, or **Pre-fader** for an independent send amount. The source's Mute still silences either kind of send.

Adding a send does not replace the main output route. Avoid adding duplicate connections: parallel paths sum and can increase output level. Up to 1,024 individual channel connections and 64 rack instances (including processors) are supported; a stereo pair uses two connections.

Controls

- Faders span –60 to +12 dB, with an exact-silence position below –60. Clearing the numeric value selects silence. Double-click resets to 0 dB.
- Sends span silence to +6 dB. New sends start silent and post-fader. Pre/post can change during playback.
- Rotary controls support vertical dragging, arrow keys, Page Up/Down, Home/End, exact numeric editing and double-click reset. Shift-drag provides finer adjustment.
- Stereo balance preserves both channels at unity in the center and attenuates the opposite channel toward either side. Explicit mono-to-stereo routing uses equal-power pan. Custom mono mappings stay unchanged.
- Mute blocks all outgoing paths, including pre-fader sends. Processors keep running, allowing downstream tails to decay.
- Multiple solos are allowed. Solo preserves required upstream inputs and downstream processors. Soloing a return excludes its sources' unrelated dry branches. Mute takes precedence. Master has no Solo control.
- Gain, balance, send and mute/solo changes use finite 20 ms ramps. Playback starts at the saved values, without an initial fade.

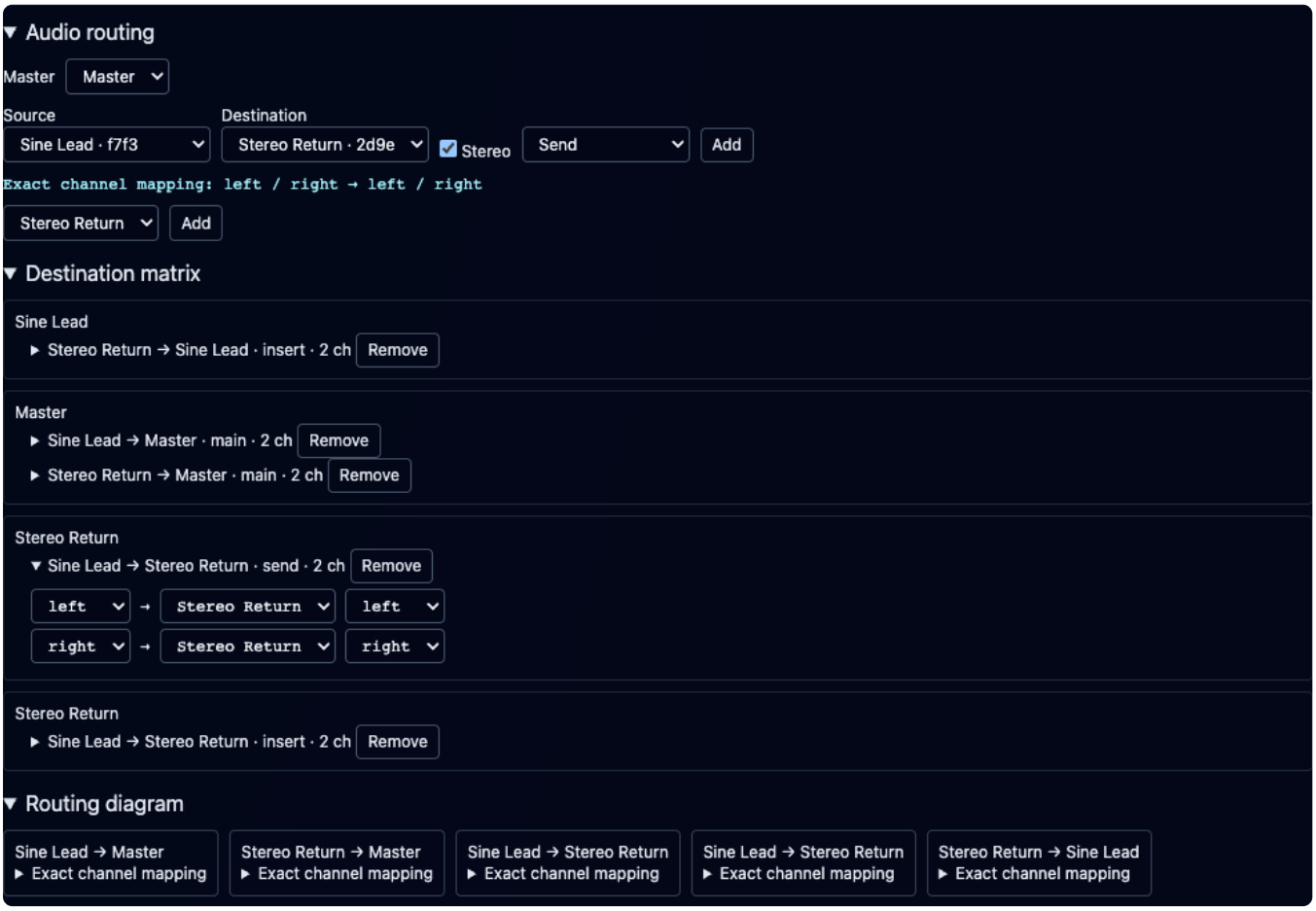
Meters show stereo peak and RMS. CLIP indicates a peak at or above 0 dBFS; no automatic limiting, normalization or clipping is applied. The final **Audio Output** meter includes Master and direct paths. Connectivity diagnostics describe structure; meters measure actual audio activity.

Routing and repair

Choose a source, destination and explicit channel mapping before adding a main route, send or custom connection. Stereo groups expand into their exact member ports. Uncheck **Stereo** to select one exact source port and destination inlet at a time, including custom or mono mappings. Group display names are not inlet names; legacy label suggestions never rename ports.

Open **Destination matrix** to inspect incoming routes grouped by destination. Expand a route row to change its source port, destination instance or destination port; **Remove** deletes that grouped connection. This is also where broken references can be repaired instead of discarding the performance.

Open **Routing diagram** to follow the same connections as source → destination cards. Expand **Exact channel mapping** on a card to inspect port names and processing stages. Click source/destination names to select routing endpoints; use the strip's **Edit patch** action to open its instrument graph.



The send's left and right mappings expanded in Destination matrix. Matrix and diagram show the same routes, including the dedicated insert's input and return-to-strip connections.

Route through Master replaces the selected strip's main routes with stereo connections to the designated Master in this performance. Sends and custom routes are retained. A neutral Master is an ordinary saved output patch; **Create neutral Master** creates/selects it when none exists. The **Master** dropdown can designate another eligible direct-output rack patch, but changing that designation alone does not rewire incoming routes.

New guided instruments connect to Master automatically. Loading an older performance preserves its authored direct/custom paths. Master gain or mute affects only paths routed into Master: a direct strip can still be audible. Check the **Direct Audio Output** badge and the final Audio Output meter when tracking down sound that remains after lowering Master.

Topology changes require stopping the engine. **Stop to edit routing** makes that action explicit. Existing mixer controls remain live. Invalid references and genuine feedback cycles block playback and CSD export. Broken routes remain available for repair. Unused ports, incomplete stereo pairs and parallel direct/processed paths are warnings.

Troubleshoot routing and silence

Expand **Routing diagnostics** and click **Check routing** after editing connections. Diagnostics describe connectivity, while meters report measured sound. Clicking an instance-related diagnostic can open its patch for inspection. A **Muted or silent path** entry selects the corresponding strip.

Symptom or diagnostic	What to check
Repair route / Missing reference	Restore the referenced patch or repair the destination and exact port names in Destination matrix.
Feedback cycle	Remove a connection that feeds a processing path back into itself. Playback and CSD export remain blocked until genuine cycles are resolved.
Connect an audio output / Select an audio source	Follow the source to an output sink; route audio into effects that require inputs.
Complete stereo mapping / Unused ports	Check both channel mappings and the declared audio groups. These structural warnings do not prove a patch is silent.
Review parallel output paths	Check whether both direct and processed copies intentionally reach the output.
Muted or silent path	Clear Mute or raise a fader/send from silence; also check active solos and whether the source is producing notes/audio.
CLIP	Reduce gain along the contributing paths. No limiter is applied automatically.

A valid route can still be silent: here the Sine Lead strip is muted.

Persistence and compatibility

Performance configuration version 11 stores stable instance IDs, explicit routes, mixer strips, send settings, Master and insert ownership. App state version 2 saves this same model. Save/Load, Clone and native JSON/ZIP bundles preserve the current settings. Mixer automation recording is not included.

Versions 1–10 convert old Level values using `20 * log10(level / 10)`, including continuous effects. Missing Level means 0 dB. Legacy inlet selection is resolved once and saved explicitly. Level no longer scales MIDI velocity; velocity-sensitive patches can therefore change timbre after migration. Authored note velocities remain unchanged.

Both performance CSD modes initialize the same routing, gain, balance, mute, solo and send settings without a browser. Offline rendering retains 48 kHz, `ksmps=1`, float WAV output and release tails. Live/offline settings match; different engine rates need not produce identical waveforms. A routed continuous source can render a finite range with no MIDI notes. Standalone instrument exports contain patch design and interface metadata, without Perform mixer settings.

Control API

Session creation and validation accept `audio_graph` and `mixer`. `GET /api/sessions/{id}/mixer` returns desired values and revision; `PUT` accepts batched partial scalar updates and an optional expected revision. Stale revisions return 409. The active browser-clock controller can send the equivalent `mixer_update` message and receive `mixer_ack` / `mixer_error` replies. Updates are queued and applied at render-block boundaries. Meter frames carry engine sample timestamps and are delivered at most 15 times per second.

`POST /api/sessions/preview` accepts a session plus inline draft patch definitions. It creates a transient runtime without saving drafts into the patch library. Normal stop/delete session operations clean it up.

The performance CLI preserves version 11 routing and mixer data. `--level` is deprecated and converts to audio gain. New CLI routes require exact destination inlet selection when names differ; `--inlet` makes that mapping explicit.

Opcode Catalog and Integrated Documentation

The opcode catalog is the source list of all supported building blocks you can place in the graph editor.

What The Catalog Supports

- Search by opcode name
- Search by category
- Search by description text
- Search by tags
- Click to add an opcode to the graph
- Drag-and-drop an opcode into the graph canvas at a chosen position

Search Behavior

The search field filters on these opcode fields:

- `name`
- `category`
- `description`
- `tags`

This makes it easy to find opcodes by function (for example `filter`, `midi`, `reverb`, `soundfont`) even if you do not remember the exact Csound opcode name.

Adding Opcodes

Click To Add

- Clicking an opcode row inserts the node into the graph using the app's add-node behavior.
- This is fastest when exact placement is not important yet.

Drag-and-Drop To Place

- Drag an opcode from the catalog and drop it into the graph canvas.
- The graph editor computes the drop location and places the node there.
- While dragging over the graph, the canvas highlights to confirm drop targeting.

Visual Information In The Catalog

Each catalog entry shows:

- Opcode icon (backend-served icon asset)
- Opcode name
- Category label

These same categories influence node coloring in the graph editor for quick visual grouping.

Integrated Opcode Documentation (`?` on node)

After placing an opcode in the graph, use the node's `?` button to open the opcode documentation modal.

The opcode documentation modal provides:

- Localized description (English/German/French/Spanish)
- Category and syntax/template summary

- Input/output port descriptions (including optional/default/accepted types)
- Tags
- Direct link to the Csound reference page (`Open Csound Reference`)

Context Help (`?` in page sections)

In addition to opcode-level docs, the app provides context help buttons for page sections (toolbar, catalog, graph, runtime, sequencer panels, config panels). These open integrated markdown help content in the currently selected GUI language.

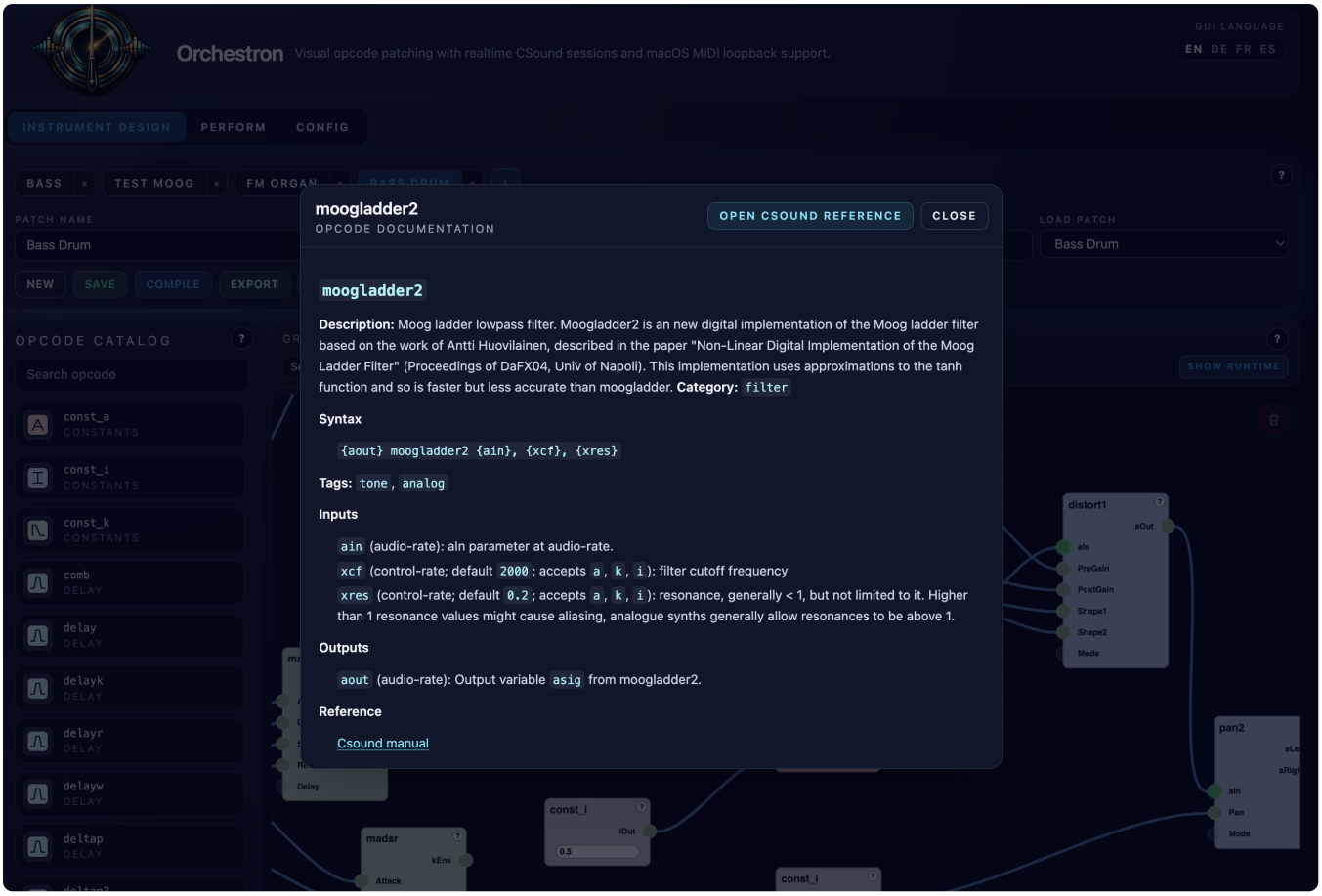
For the opcode catalog specifically, the integrated help now focuses on search/add workflow and on the distinction between catalog help and the node-level opcode documentation modal.

See also:

- GUI Language and Integrated Help
- Supported Opcodes for the full opcode index table

Screenshots

Opcode catalog filtered by search text.



Integrated opcode documentation modal opened from a graph node.

GUI Language and Integrated Help

Orchestron includes multilingual UI labels and integrated documentation so you can stay inside the app while learning and working.

GUI Language Selector

The language selector is in the top-right area of the main app header.

Supported languages:

- English
- German
- French
- Spanish

The selector is presented as quick language links (short labels) for fast switching.

What Changes When You Switch Language

Language switching affects user-facing text such as:

- Page labels (`Instrument Design`, `Perform`, `Config`)
- Toolbar/button labels
- Context help pages (integrated help modals)
- Opcode documentation modal content (localized generated markdown)
- Many labels inside advanced editors (including the GEN editor)

Integrated Help (`?` Buttons)

Orchestron provides context help buttons (`?`) across the UI.

Examples include:

- Instrument patch toolbar
- Opcode catalog
- Graph editor header/area
- Runtime panel
- Sequencer sections (rack, tracks, multitrack arranger, piano rolls, MIDI controllers)
- Config page sections

Clicking a context `?` opens a markdown help modal tailored to that UI area.

On the Instrument Design, Perform, and Config pages, these help modals focus on the current tool, device, transport section, or engine setting panel instead of repeating generic UI conventions.

Opcode Documentation (`?` On Opcode Node)

Placed opcode nodes provide a node-level `?` button that opens opcode documentation.

The modal includes:

- Localized description
- Inputs and outputs (with detailed port descriptions)
- Syntax/template summary
- Tags

- `Open Csound Reference` link to the official Csound documentation page for that opcode

Modal Behavior

Both help modals and opcode documentation modals support:

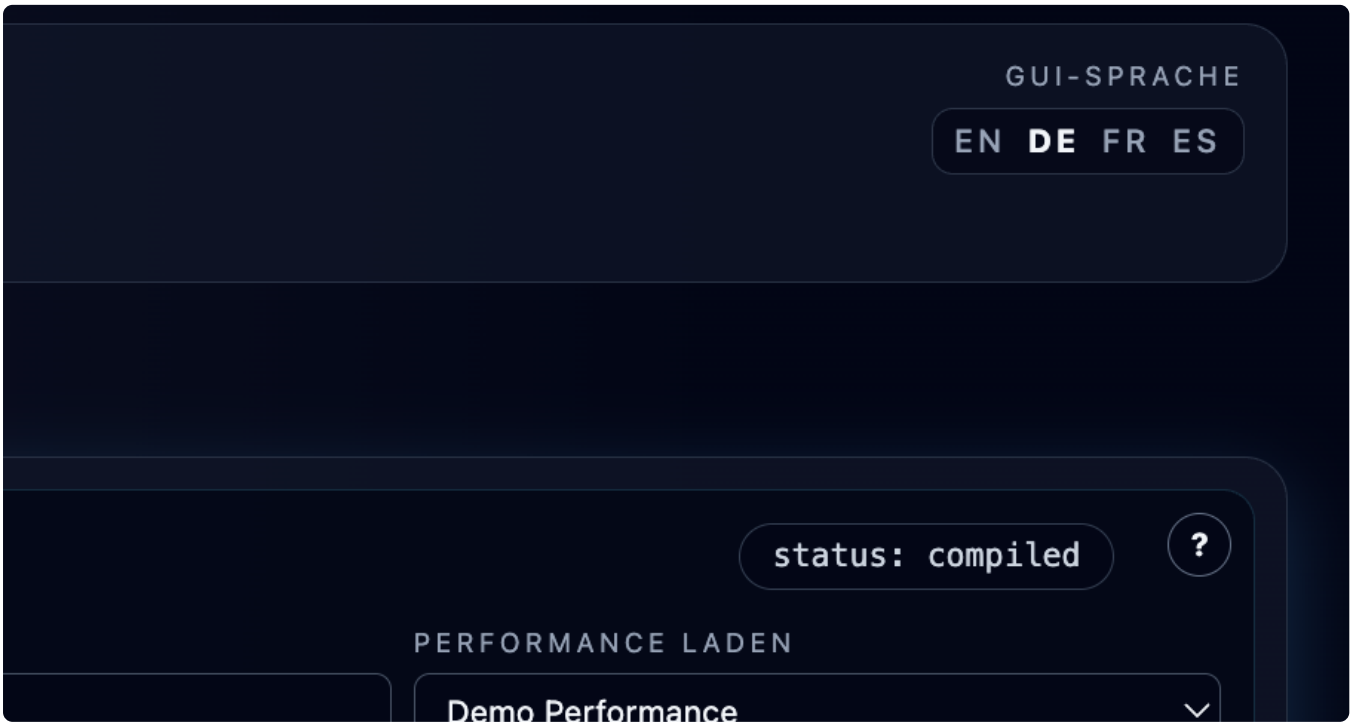
- Close button
- Click outside modal to close
- `Esc` key to close

Best Practice

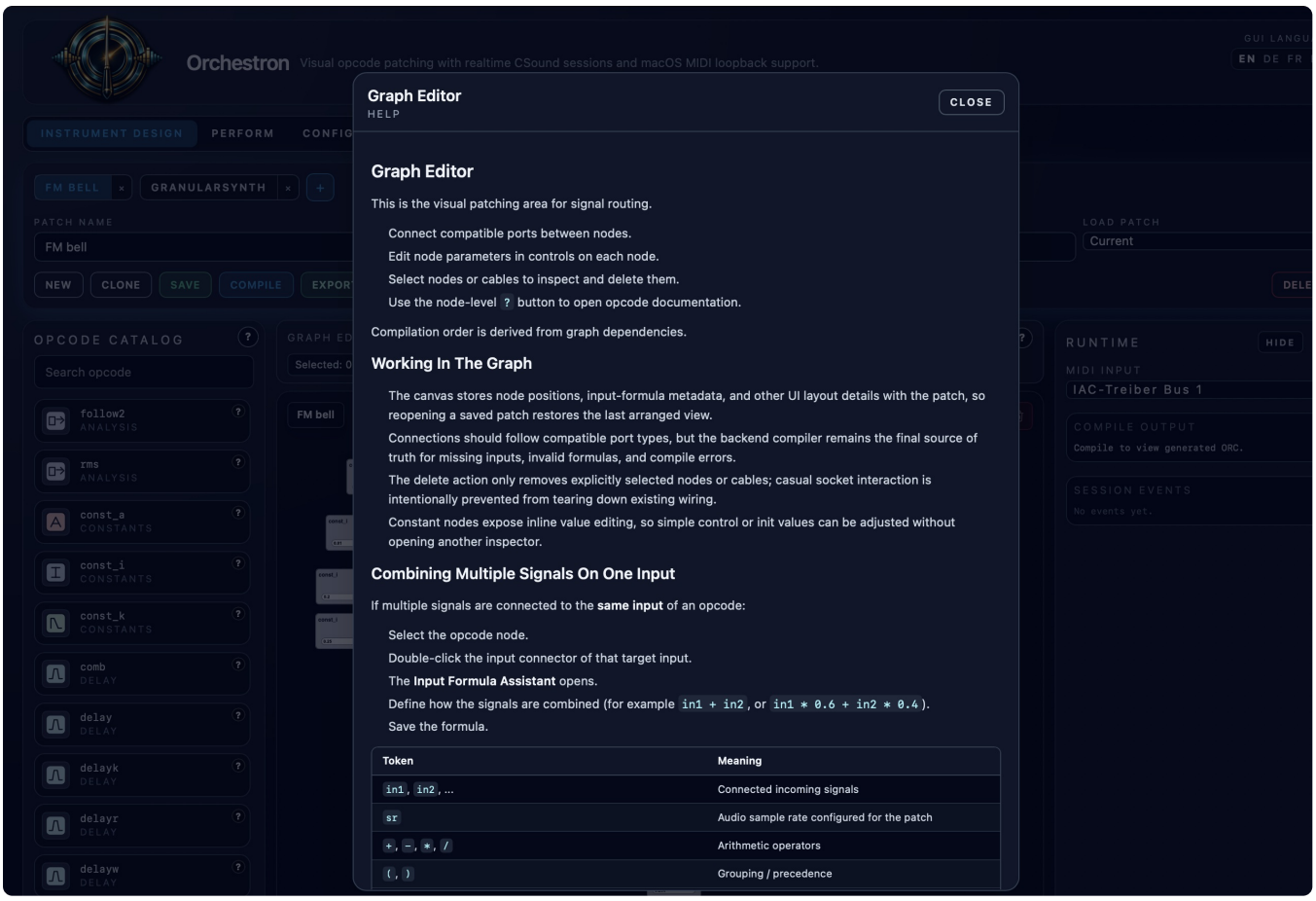
Use context help for workflow guidance and opcode docs for technical/operator details.

This combination is faster than switching repeatedly between the app and external documentation while designing patches.

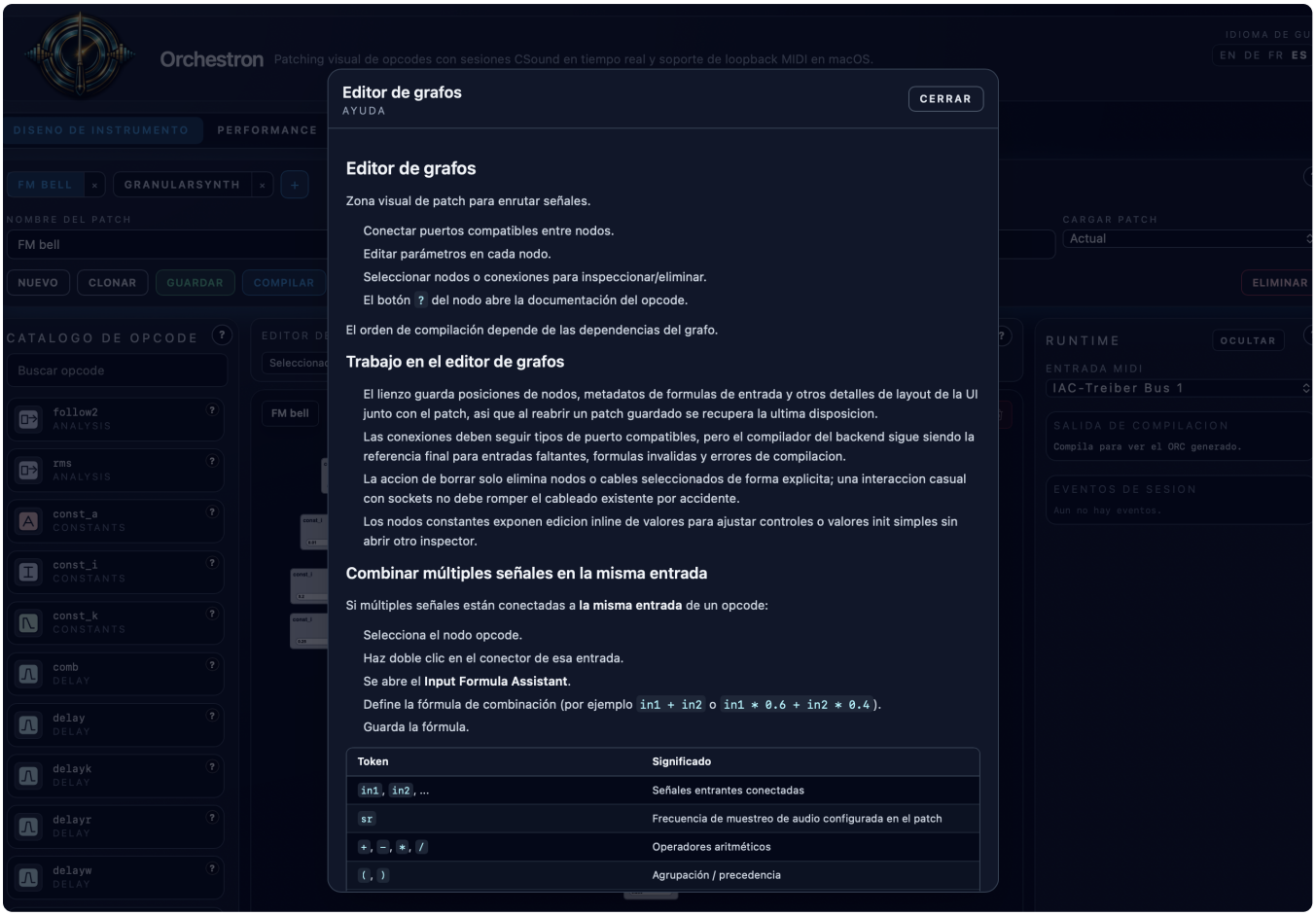
Screenshots



Top-right header language selector links.



Context help modal integrated into the workflow.



Integrated help and UI localization in multiple supported languages.

Supported Opcodes

This appendix is generated from `backend/app/data/opcodes.json` and currently lists **144** user-selectable opcodes in the Instrument Design opcode catalog.

How To Use This Appendix

- Use the table for quick discovery (category, signal I/O, one-line purpose).
- In the app, click the `?` on an opcode node to open the integrated, localized documentation with port-level details and a Csound reference link.
- The `Inputs / Outputs` columns show port count and signal-rate shorthand.
- Signal legend: `a` = audio-rate, `k` = control-rate, `i` = init-rate, `S` = string, `f` = phase-vocoder frame signal.

Category Counts

Category	Count
analysis	2
constants	4
delay	11
distortion	4
dynamics	2
envelope	12
filter	16
fm	10
math	3
midi	7
mixer	3
modulation	10
noise	3
oscillator	17
output	1
physical_modeling	11
reverb	4
routing	4
soundfont	3
spectral	8
tables	4
utility	5

Opcode Index

analysis

Opcode	Inputs	Outputs	Short Description
follow2	3 (a, k, k)	1 (a)	Envelope follower with independent attack and release controls.
rms	3 (a, i, i)	1 (k)	RMS envelope follower for an audio signal.

constants

Opcode	Inputs	Outputs	Short Description
const_a	-	1 (a)	Audio-rate constant value.
const_i	-	1 (i)	Init-rate constant value.
const_k	-	1 (k)	Control-rate constant value.
const_s	-	1 (S)	String constant value.

delay

Opcode	Inputs	Outputs	Short Description
comb	4 (a, k, i, i)	1 (a)	Comb filter / feedback delay.
delay	3 (a, i, i)	1 (a)	Simple non-interpolating audio delay line.
delayk	3 (k, i, i)	1 (k)	Control-rate delay line.
delayr	2 (i, i)	1 (a)	Read tap from a classic delay-line memory buffer.
delayw	1 (a)	-	Write into the delay-line memory buffer.
deltap	1 (k)	1 (a)	Read a delay tap using linear interpolation.
deltap3	1 (k)	1 (a)	Read a delay tap using cubic interpolation.
flanger	4 (a, k, k, i)	1 (a)	Flanger effect with delay modulation and feedback.
vcomb	6 (a, k, k, i, i, i)	1 (a)	Variable-time comb reverb with controllable loop time.
vdelay3	4 (a, k, i, i)	1 (a)	Variable delay line with cubic interpolation.
vdelayxs	6 (a, a, a, i, i, i)	2 (a, a)	Stereo variable delay with high-quality sinc interpolation.

distortion

Opcode	Inputs	Outputs	Short Description
clip	4 (a, i, i, i)	1 (a)	Signal clipper with selectable transfer curves.
distort1	6 (a, k, k, k, k, i)	1 (a)	Waveshaping distortion with configurable transfer curve.
fold	2 (a, k)	1 (a)	Artificial foldover effect for audio signals.
powershape	3 (a, k, i)	1 (a)	Power-law waveshaper for controllable nonlinear distortion.

dynamics

Opcode	Inputs	Outputs	Short Description
dam	6 (a, k, i, i, i, i)	1 (a)	Dynamic amplitude processor (downward compressor/noise suppressor).
limit	3 (a, k, k)	1 (a)	Hard clamp limiter.

envelope

Opcode	Inputs	Outputs	Short Description
adsr	4 (i, i, i, i)	1 (k)	Control-rate ADSR envelope.
envlpxr	8 (k, i, i, i, i, i, i, i)	1 (k)	Control-rate envelope with optional release scaling controls.
expon	3 (i, i, i)	1 (k)	Control-rate exponential segment between two points.
expseg	9 (i, i, i, i, i, i, i, i, i)	1 (k)	Control-rate exponential breakpoint envelope generator.
expsega	9 (i, i, i, i, i, i, i, i, i)	1 (a)	Audio-rate exponential breakpoint envelope generator.
expsegr	5 (i, i, i, i, i)	1 (k)	Control-rate exponential envelope with release segment.
linenr	4 (k, i, i, i)	1 (k)	Control-rate envelope with note-release stage.
linseg	7 (i, i, i, i, i, i, i)	1 (k)	Control-rate linear breakpoint envelope generator.
linsegr	5 (i, i, i, i, i)	1 (k)	Control-rate linear breakpoint envelope with release segment.
madsr	6 (i, i, i, i, i, i)	1 (k)	MIDI release-sensitive ADSR envelope.
mxadsr	6 (i, i, i, i, i, i)	1 (k)	Extended MIDI release-sensitive ADSR envelope.
transegr	7 (i, i, i, i, i, i, i)	1 (k)	Control-rate transition envelope with release-triggered final segment.

filter

Opcode	Inputs	Outputs	Short Description
butterbp	4 (a, k, k, i)	1 (a)	Second-order Butterworth band-pass filter.
butterbr	4 (a, k, k, i)	1 (a)	Second-order Butterworth band-reject filter.
butterhp	3 (a, k, i)	1 (a)	Second-order Butterworth high-pass filter.
butterlp	3 (a, k, i)	1 (a)	Second-order Butterworth low-pass filter.
diode_ladder	6 (a, k, k, i, i, i)	1 (a)	Diode ladder low-pass filter model.
exciter	5 (a, k, k, k, k)	1 (a)	Harmonic exciter that adds controlled upper partials.
fofilter	5 (a, k, k, k, i)	1 (a)	Formant filter.
moogladder	3 (a, k, k)	1 (a)	Moog ladder low-pass filter.
moogladder2	3 (a, k, k)	1 (a)	Nonlinear Moog-style ladder filter with audio-rate modulation support.
moogvcf	5 (a, k, k, i, i)	1 (a)	Moog ladder voltage-controlled filter emulation.
resonx	6 (a, k, k, i, i, i)	1 (a)	Multi-layer resonant band-pass filter.
rezzy	5 (a, k, k, i, i)	1 (a)	Resonant low-pass or high-pass filter.
skf	5 (a, k, k, i, i)	1 (a)	Sallen-Key low-pass or high-pass filter.
statevar	5 (a, k, k, i, i)	4 (a, a, a, a)	State-variable filter with simultaneous HP/LP/BP/BR outputs.
tbvcf	6 (a, k, k, k, k, i)	1 (a)	TB-303 style voltage-controlled filter model.
vclpf	4 (a, k, k, i)	1 (a)	Virtual-analog low-pass filter.

fm

Opcode	Inputs	Outputs	Short Description
crossfmi	9 (k, k, k, k, k, i, i, i, i)	2 (a, a)	Interpolating crossed frequency-modulation oscillator pair.
crossfmpmi	9 (k, k, k, k, k, i, i, i, i)	2 (a, a)	Interpolating crossed frequency/phase-modulation oscillator pair.
crosspmi	9 (k, k, k, k, k, i, i, i, i)	2 (a, a)	Interpolating crossed phase-modulation oscillator pair.
fmb3	10 (k, k, k, k, k, i, i, i, i, i)	1 (a)	B3 organ FM model.
fmbell	12 (k, k, k, k, k, k, i, i, i, i, i, i)	1 (a)	Bell FM model.
fmmetal	11 (k, k, k, k, k, k, i, i, i, i, i)	1 (a)	Metallic FM model.
fmpercfl	11 (k, k, k, k, k, k, i, i, i, i, i)	1 (a)	Percussive flute FM model.
fmrhode	11 (k, k, k, k, k, k, i, i, i, i, i)	1 (a)	Rhodes electric piano FM model.
fmvoice	11 (k, k, k, k, k, k, i, i, i, i, i)	1 (a)	Voice-like FM model.
fmwurlie	11 (k, k, k, k, k, k, i, i, i, i, i)	1 (a)	Wurlitzer electric piano FM model.

math

Opcode	Inputs	Outputs	Short Description
a_mul	2 (a, a)	1 (a)	Multiply two audio signals.
k_mul	2 (k, k)	1 (k)	Multiply two control signals.
tanh	1 (a)	1 (a)	Hyperbolic tangent function for audio waveshaping and limiting.

midi

Opcode	Inputs	Outputs	Short Description
ampmidi	2 (i, i)	1 (i)	Generate init-rate amplitude from MIDI note velocity.
ampmidicurve	3 (k, k, k)	1 (k)	Map MIDI velocity to gain using dynamic range and curve exponent.
ampmidid	2 (k, i)	1 (k)	Map MIDI velocity to amplitude using a decibel range.
cpsmidi	-	1 (i)	Read active MIDI note pitch as cycles-per-second.
midi_note	1 (i)	2 (k, k)	Extract MIDI note frequency and velocity amplitude.
midictrl	3 (i, i, i)	1 (k)	Read a MIDI controller value with optional scaling.
notnum	-	1 (i)	Read the MIDI note number for the current note event.

mixer

Opcode	Inputs	Outputs	Short Description
mix2	2 (a, a)	1 (a)	Mix two audio signals.
ntrpol	3 (a, a, k)	1 (a)	Linear crossfade between two audio signals.
pan2	3 (a, k, i)	2 (a, a)	Stereo panner.

modulation

Opcode	Inputs	Outputs	Short Description
jitter	3 (k, k, k)	1 (k)	Control-rate segmented random line generator.
lfo	3 (k, k, i)	1 (k)	Low-frequency oscillator for control-rate modulation.
portk	3 (k, k, i)	1 (k)	Control-rate portamento/slew limiter with optional init value.
random	2 (k, k)	1 (k)	Control-rate random value generator between a minimum and maximum.
randomh	5 (k, k, k, i, i)	1 (k)	Control-rate random sample-and-hold generator with update frequency control.
randomi	5 (k, k, k, i, i)	1 (k)	Control-rate interpolated random generator with update frequency control.
release	-	1 (k)	Return 1 during note release phase and 0 otherwise.
samphold	4 (k, k, i, i)	1 (k)	Control-rate sample-and-hold processor.
vibr	4 (k, k, i, i)	1 (k)	Simple vibrato control oscillator with table lookup.
vibrato	9 (k, k, k, k, k, k, k, k, i)	1 (k)	Randomized vibrato generator.

noise

Opcode	Inputs	Outputs	Short Description
noise	4 (k, k, i, i)	1 (a)	Variable-color random audio noise.
pinker	-	1 (a)	Pink noise generator.
pinkish	2 (a, i)	1 (a)	Pinkening filter for shaping white noise toward a pink spectrum.

oscillator

Opcode	Inputs	Outputs	Short Description
fof	15 (k, k, k, k, k, k, k, k, i, i, i, i, i, i, i)	1 (a)	Formant/granular source using sinusoid bursts.
fof2	15 (k, k, k, k, k, k, k, k, i, i, i, i, k, k, i)	1 (a)	FOF source with per-grain phase indexing and glissando.
foscili	7 (k, k, k, k, k, i, i)	1 (a)	Audio-rate FM oscillator with harmonic ratios.
gbuzz	7 (k, k, k, k, k, i, i)	1 (a)	Generalized buzz oscillator with controllable harmonics.
grain	10 (k, k, k, k, k, k, i, i, i, i)	1 (a)	Classic granular synthesis oscillator with table-based grains.
grain2	9 (k, k, k, i, k, i, i, i, i)	1 (a)	Easy-to-use granular synthesis texture generator.
grain3	11 (k, k, k, k, k, k, i, k, i, i, i)	1 (a)	Granular oscillator with independent pitch and frequency modulation.
granule	22 (k, i, i, i, i, i, i, i, i, i, k, i, k, i, i, i, i, i, i, i, i, i)	1 (a)	Multi-voice granular processor with independent gap and grain-size controls.
moog	9 (k, k, k, k, k, k, i, i, i)	1 (a)	Mini-Moog style synthesizer model source.
oscil3	4 (k, k, i, i)	1 (a)	Cubic-interpolating oscillator with low distortion.
oscili	3 (k, k, i)	1 (a)	Classic interpolating oscillator.
poscil3	4 (k, k, i, i)	1 (a)	High-precision cubic interpolating oscillator.
syncphasor	3 (k, a, i)	2 (a, a)	Audio-rate normalized phase generator with sync trigger I/O.
vco	5 (k, k, i, k, i)	1 (a)	Band-limited voltage-controlled oscillator.
vco2	6 (k, k, i, k, k, i)	1 (a)	Improved anti-aliased analog-style oscillator.
voice	8 (k, k, k, k, k, k, i, i)	1 (a)	Formant-based vocal source synthesizer.
vosim	8 (k, k, k, k, k, k, i, i)	1 (a)	Vocal formant synthesis using overlapping sinusoidal bursts.

output

Opcode	Inputs	Outputs	Short Description
outs	2 (a, a)	-	Stereo output sink.

physical_modeling

Opcode	Inputs	Outputs	Short Description
dripwater	8 (k, i, i, i, i, i, i, i)	1 (a)	Stochastic dripping-water physical model source.
marimba	11 (k, k, i, i, i, k, k, i, i, i, i)	1 (a)	Physical model of a marimba bar and resonator.
pluck	6 (k, k, i, i, i, i)	1 (a)	Karplus-Strong plucked-string model.
sekere	5 (i, i, i, i, i)	1 (a)	PhISEM sekere shaker model.
sleighbells	8 (k, i, i, i, i, i, i, i)	1 (a)	PhISEM sleigh bells model.
wgbow	8 (k, k, k, k, k, k, i, i)	1 (a)	Waveguide bowed-string model.
wgbowedbar	9 (k, k, k, k, k, i, i, i, i)	1 (a)	Waveguide bowed-bar model.
wgclar	10 (k, k, k, i, i, k, k, k, i, i)	1 (a)	Waveguide clarinet model.
wgflute	10 (k, k, k, i, i, k, k, k, i, i)	1 (a)	Waveguide flute model.
wgpluck2	5 (i, k, i, k, k)	1 (a)	Waveguide plucked string model with pick and reflection controls.
wguide2	4 (a, k, k, k)	1 (a)	Two-point waveguide resonator.

reverb

Opcode	Inputs	Outputs	Short Description
freeverb	6 (a, a, k, k, i, i)	2 (a, a)	Stereo Freeverb processor with room size and high-frequency damping controls.
platerev	9 (i, i, k, i, i, i, i, a, a)	2 (a, a)	Physical-model plate reverb with stereo output taps.
reverb2	4 (a, k, k, i)	1 (a)	Schroeder reverb processor.
reverbsc	7 (a, a, k, k, i, i, i)	2 (a, a)	8-delay-line stereo FDN reverb based on Sean Costello's design.

routing

Opcode	Inputs	Outputs	Short Description
inleta	1 (S)	1 (a)	Receive an audio-rate signal from a named instrument inlet port.
inletk	1 (S)	1 (k)	Receive a control-rate signal from a named instrument inlet port.
outleta	2 (S, a)	-	Send an audio-rate signal to a named instrument outlet port.
outletk	2 (S, k)	-	Send a control-rate signal to a named instrument outlet port.

soundfont

Opcode	Inputs	Outputs	Short Description
sfinstr3	8 (i, i, k, k, i, i, i, i)	2 (a, a)	Play a SoundFont instrument as stereo audio with cubic interpolation.
sfload	-	1 (i)	Load a SoundFont2 file and return a file handle.
sfplay3	8 (i, i, k, k, i, i, i, i)	2 (a, a)	Play a SoundFont preset as stereo audio with cubic interpolation.

spectral

Opcode	Inputs	Outputs	Short Description
pvsanal	7 (a, i, i, i, i, i, i)	1 (f)	Phase-vocoder analysis from an audio input to an fsig stream.
pvshift	6 (f, k, k, k, k, k)	1 (f)	Shift fsig partial frequencies by a fixed amount in Hz.
pvsmooth	3 (f, k, k)	1 (f)	Smooth fsig amplitude and frequency trajectories with lowpass filtering.
pvsmorph	4 (f, f, k, k)	1 (f)	Morph between two fsig streams by amplitude and frequency interpolation.
pvsosc	8 (k, k, k, i, i, i, i, i)	1 (f)	Generate oscillator spectra directly as an fsig stream.
pvsvoc	5 (f, f, k, k, k)	1 (f)	Cross-synthesize fsig amplitudes and excitation frequencies.
pvswarp	7 (f, k, k, k, k, k, k)	1 (f)	Warp and shift the spectral envelope of an fsig stream.
pvsynth	2 (f, i)	1 (a)	Resynthesize audio from an fsig stream via overlap-add.

tables

Opcode	Inputs	Outputs	Short Description
GEN	-	1 (i)	Routine-aware function table generator (meta-opcode) that renders ftgen/ftgenonce from a specialized editor.
ftgen	12 (i, i, i, i, i, i, i, i, i, i, i, i)	1 (i)	Create a function table at init time using a GEN routine.
ftgenonce	12 (i, i, i, i, i, i, i, i, i, i, i, i)	1 (i)	Generate a function table once and reuse it across instances.
vco2init	6 (i, i, i, i, i, i)	1 (i)	Precompute band-limited wavetable sets for vco2 oscillators.

utility

Opcode	Inputs	Outputs	Short Description
downsamp	2 (a, i)	1 (k)	Convert an audio signal to control-rate by downsampling.
k_to_a	1 (k)	1 (a)	Interpolate control signal to audio-rate.
maxalloc	1 (i)	-	Limit simultaneous instrument allocations for the compiled instrument.
upsamp	1 (k)	1 (a)	Convert a control signal to audio-rate by repeating k-values.
xtratim	1 (i)	-	Extend current note duration by an additional init-time amount.

Graph Editor

The graph editor is the central visual patching surface for instrument design.

Core Graph Editing Features

- Typed input/output ports (audio, control, init, string, function-table)
- Node placement and repositioning (drag nodes on the canvas)
- Connection creation between compatible ports
- Connection and node selection highlighting
- Deletion of selected nodes/connections via explicit delete action
- Zoom controls and fit-to-graph navigation
- Node category color coding
- Inline constant value editing for `const_a`, `const_i`, `const_k`, `const_s`

Canvas Navigation

The graph editor supports pan/zoom navigation and includes a zoom HUD in the lower-right corner:

- Zoom percentage display
- `−` zoom out
- `+` zoom in
- `Fit` to fit the current graph into view

If the graph is empty, `Fit` resets toward a centered default view.

Node Appearance and Visual Cues

Category Coloring

Nodes are color-coded by category (for example generator, filter, MIDI, constants, etc.) to help visually scan signal flow.

Optional Inputs

Optional inputs are visually distinct from required inputs.

Formula Highlight On Sockets

If an input socket has a saved Input Formula (combine formula), the socket is highlighted so you can immediately see custom combine logic exists.

Adding Nodes

You can add nodes in two ways:

- Click from the opcode catalog
- Drag an opcode from the catalog and drop it into the graph canvas

Drag-and-drop placement is preferable for larger patches because it preserves your layout intent.

Connections

Creating Connections

- Connect outputs to compatible inputs.

- The backend compiler remains the source of truth for final type/compile validation.

Selecting Connections

- Connections can be selected directly (not just nodes).
- Selected connections receive a visual highlight.
- Ctrl-based accumulation allows multi-select of connections and nodes.

Connection Deletion Is Intentional

Orchestron prevents accidental edge removal during socket interaction.

- Connections are not removed by casual socket clicks/drags.
- To remove a connection, select it explicitly and use the delete action.

Selection Behavior

- Click nodes to select.
- Ctrl-click accumulates node selections.
- Click connections to select them.
- Ctrl-click accumulates/deselects connection selections.
- Background click clears selection (when Ctrl is not held).

Delete Selected Elements

The graph editor overlay includes a trash/delete button (top-right of the graph canvas area).

- Disabled when nothing is selected
- Enabled when at least one node or connection is selected

Deletion flow:

- If one item is selected, deletion can happen directly.
- If multiple items are selected, Orchestron opens a confirmation dialog listing all affected items (nodes and connections) before deletion.

Node-Specific Controls

Constant Nodes

The constant opcodes (`const_a` , `const_i` , `const_k` , `const_s`) provide inline editable values directly on the node. `const_s` accepts lowercase letters, digits, and underscores, must begin with a lowercase letter, and is limited to 50 characters.

Documentation Button (?)

Nodes with documentation expose a `?` button to open the opcode documentation modal.

GEN Button (for `GEN` node)

The `GEN` meta-opcode node shows a `GEN` button that opens the specialized GEN table editor.

Layout Persistence

- Node positions are stored in the patch graph.
- The graph structure (nodes + connections) is part of the patch data.
- The visual layout and related UI metadata (for example input formulas, GEN node configuration) are stored in `graph.ui_layout` .

Compile Implications

- Compilation order is derived from graph dependencies.
- Invalid wiring, missing required inputs, or invalid formulas surface compile diagnostics.
- Graph compile status is reflected in the instrument page header (`compiled` , `pending changes` , `errors`).

Integrated Help Focus

The graph editor `?` help now concentrates on graph-specific behavior: persisted layout, deliberate deletion, constant-node inline editing, and the Input Formula Assistant workflow for combining multiple signals on one input.

Stereo blocks and audio interfaces

Stereo Input and Stereo Output are collapsible views of ordinary paired inleta/outleta nodes. Expand them to edit the underlying opcodes and connections. Exact port names and input formulas survive collapsing, expansion, saving and export. Direct Audio Output (outs) stays distinct.

The interface panel describes musical role, stable group IDs, display names, exact member ports, purpose and designated main input/output. Stereo blocks and their mappings share the same creation and deletion operations. Advanced mono/custom groups remain metadata views; changing a stereo group to another layout exposes the ordinary nodes without deleting them.

Create, rename and remove a stereo interface

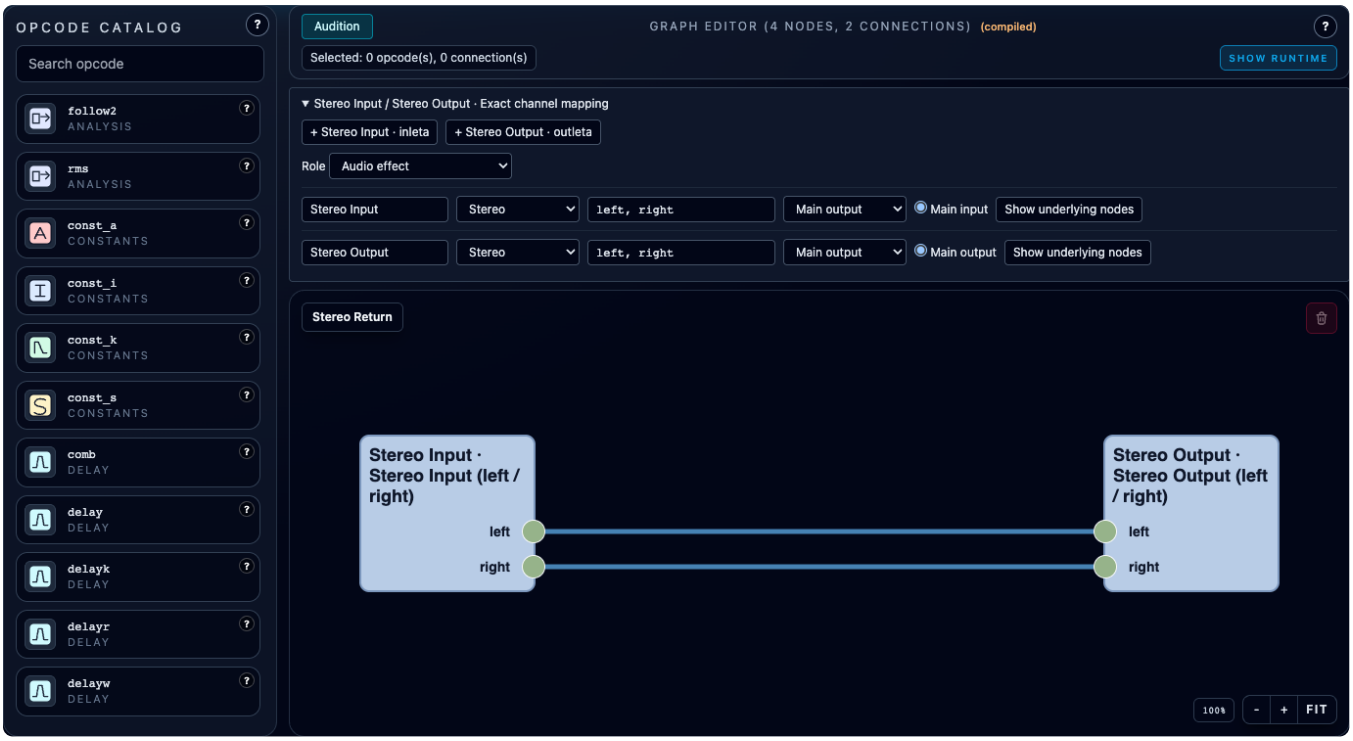
1. Click **Stereo Input** or **Stereo Output** in the opcode catalog, drag either onto the canvas, or expand **Stereo Input / Stereo Output · Exact channel mapping** and use its matching add button. Each action creates two channels and their mapping. Connect the new audio ports as needed.
2. Set Role and review the group's display name, purpose and designated Main input/Main output. New pairs use `left` / `right` or an available `busN.left` / `busN.right` pair. Channel names are unique within each direction.
3. Edit **Left** and **Right**, then choose **Apply** to review and confirm the rename. **Cancel** discards the draft. Renaming changes the mapping and underlying opcode names together. A string constant supplying an old name remains available to its other connections.
4. **Show underlying nodes** exposes the pair; **Collapse** returns to its grouped view. Delete a collapsed block or choose **Remove** on its mapping to remove both channels, attached connections and affected input formulas. The confirmation lists all affected elements. Deleting only one expanded channel removes the mapping and leaves the other as an ordinary opcode. Deleting a connection alone retains the mapping.
5. Deleting the designated main group clears its selection without promoting another group. Select a remaining group explicitly when needed.
6. Compile and save before assigning the patch in Perform. Check Audio Mixer and Routing after any external port changes.

Upgrade or repair an existing instrument

- **Group existing channels:** choose Stereo Input or Stereo Output, select the existing Left/Right nodes and review the change. This preserves the channels' exact names, IDs, positions, audio wiring and formulas. Channel names become part of the stereo block's configuration: values from connected `const_s` nodes are stored directly on the underlying channels, and their naming connections are removed. Dedicated naming constants disappear from the canvas. Constants still used by other nodes or formulas remain available to those consumers; editing or deleting them no longer changes the grouped names. Edit the block's **Left**/**Right** fields to rename its channels. Channels already used by another mapping, duplicate names and unresolved expressions must be repaired first.
- **Convert Direct Audio Output:** select one `outs` node and review its replacement with a named stereo outlet pair. Each side retains its incoming wires, literal value and input formula. Missing inputs remain unconnected. Other direct outputs are left intact. This changes the destination model: connect the named ports in the performance mixer to hear them.
- Ungrouped old patches are not converted automatically. Previously saved valid stereo groups receive the same

channel-name cleanup when opened, including groups that are currently expanded. Stale mappings show a reason and **Repair** / **Remove mapping**. Repair can bind a chosen existing pair or explicitly **Create missing channels**. Removing an unresolved mapping removes only metadata, preserving existing nodes.

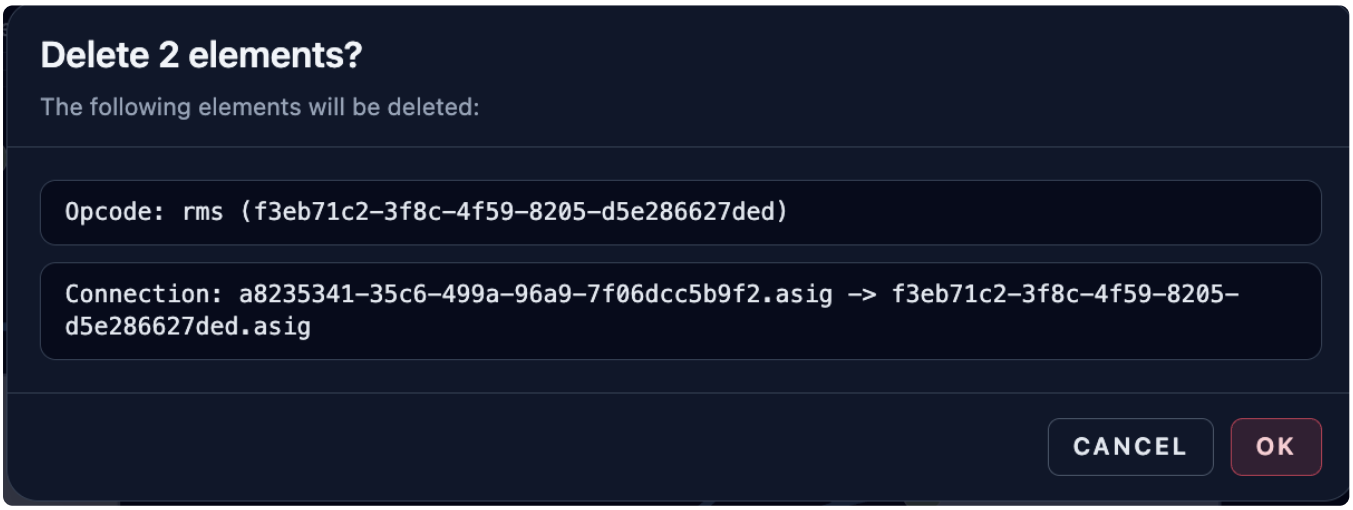
- Conversion and renaming preserve role and activation. Previews list affected current-performance routes for explicit repair in the mixer after saving; other saved performances remain unchanged.



A continuous pass-through effect with explicit left/right groups and collapsed Stereo Input and Stereo Output blocks.

Screenshots

Graph editor detail showing selection state, delete button, and zoom controls.



Delete-selection confirmation dialog listing selected graph items.

Input Formula Assistant

The Input Formula Assistant lets you define exactly how incoming signals are combined on a single target input.

When To Use It

Most commonly, you use it when **multiple outputs are connected to the same input**.

Examples:

- Mix two modulation sources before feeding a filter cutoff input
- Scale one source more than another (`in1 * 0.8 + in2 * 0.2`)
- Apply utility transforms (`abs(in1)`, `dbamp(in1)`) before an input

It is also useful for advanced cases with only one or even zero connected inputs:

- One input: reshape or scale the signal (`ceil(in1)`, `in1 * 0.5`)
- Zero inputs: provide a constant expression (`0.25`, `ampdb(6)`) as a formula-driven value

How To Open It

- Double-click an **input socket** in the graph editor.
- The assistant opens for that exact target node + target input port.

What The Assistant Shows

Connected Inputs (Token Mapping)

The left panel lists inbound connections to the target input and assigns tokens such as:

- `in1`
- `in2`
- `in3`

Each token shows source information (node/opcode/port) so you can see what signal you are referencing.

Operator and Function Buttons

The assistant provides insertion buttons for:

- Operators: `+`, `-`, `*`, `/`
- Grouping: `(`, `)`
- Unary functions: `abs()`, `ceil()`, `floor()`, `ampdb()`, `dbamp()`
- Built-in literals: `sr` (configured audio sample rate)
- Custom numeric literals via the number entry field

Formula Text Area

You can type directly into the formula field.

Token Selection / Delete Helper

The assistant tokenizes the current expression and displays clickable tokens.

- Click a token to select that exact range in the text area.
- Use `Delete Selection` to remove the selected token/range quickly.

Validation Rules

The formula must be syntactically valid before it can be saved.

Validation includes:

- Non-empty expression
- Valid numeric literals
- Supported characters/operators only
- Balanced parentheses
- Valid token ordering
- Known identifiers only (`inN` tokens, supported built-in literals like `sr` , or supported unary function names)

If validation fails:

- The formula text area border changes to an error style
- A list of validation errors is shown
- `Save Formula` is disabled

Save / Clear / Cancel

- `Save Formula` : stores the expression and token bindings for that target input
- `Clear Formula` : removes the saved formula for the target input
- `Cancel / Close` : closes the dialog without changing the saved formula

What Happens During Compile

- If multiple connections target one input and **no** formula is defined, Orchestron falls back to summing the inbound signals.
- If a formula exists, the compiler resolves the token bindings and compiles the expression.
- Formula validation also happens backend-side, so invalid formulas fail compile with diagnostics.

Formula Examples

Weighted Mix

```
in1 * 0.7 + in2 * 0.3
```

Gain Conversion

```
dbamp(in1)
```

Rectified Modulation

```
abs(in1)
```

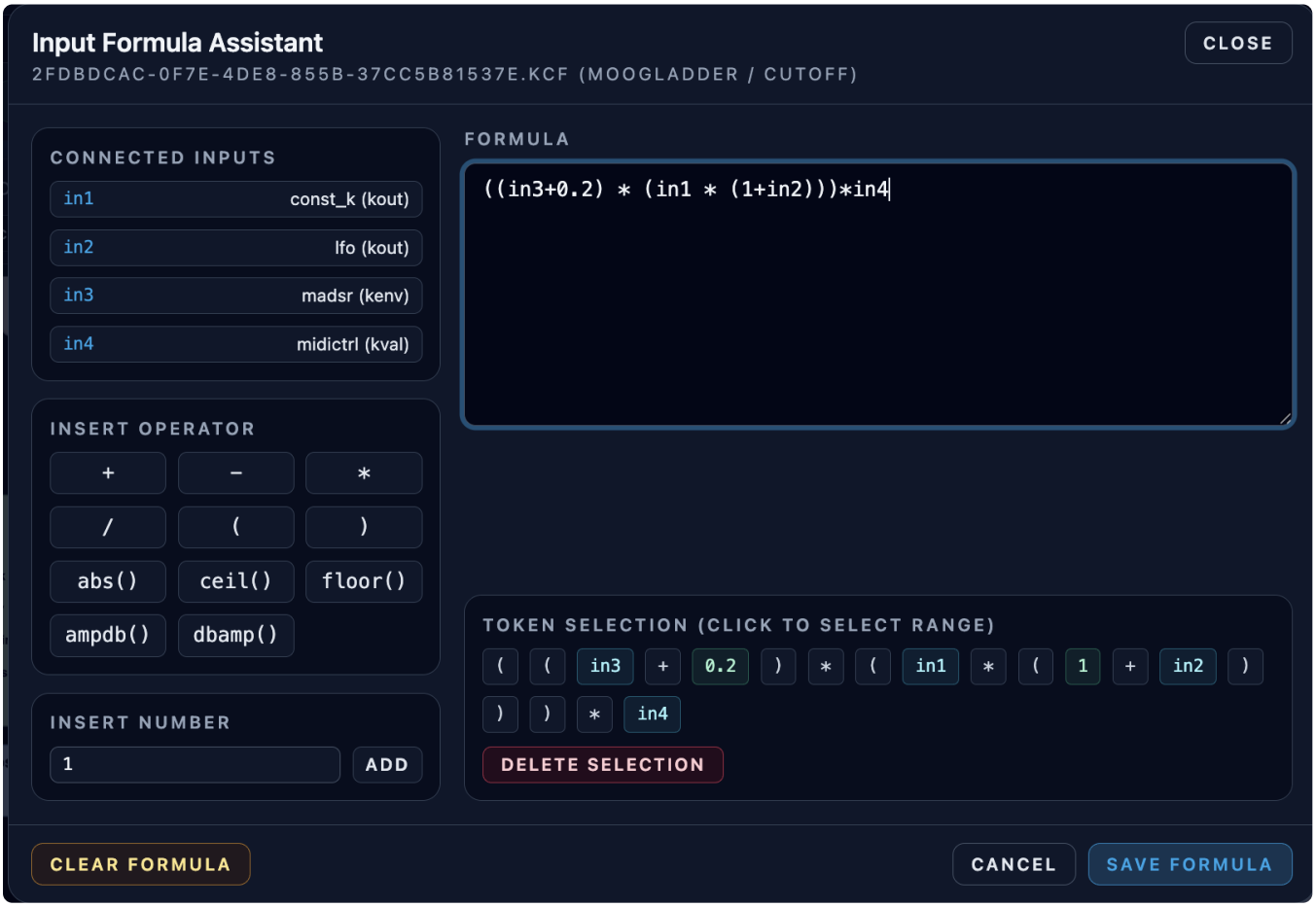
Constant Override

```
0.5
```

Sample-Rate-Aware Expression

```
sr * 0.5
```

Screenshots



Input Formula Assistant with connected input tokens and editable formula.

GEN Table Editor (GEN Meta-Opcode)

The `GEN` meta-opcode is a specialized table generator node that opens a dedicated editor for building `ftgen` / `ftgenonce` lines visually.

Why The GEN Meta-Opcode Exists

Csound function-table generation is powerful but argument-heavy. The GEN editor provides a structured UI for common GEN routines and a preview of the generated `ftgen` / `ftgenonce` line.

Opening The GEN Editor

- Add the `GEN` opcode to the graph.
- Click the `GEN` button on the node card.

Editor Overview

The editor has two main areas:

- Left side: generation mode, routine selection, parameters
- Right side: preview (`Effective GEN` , flattened args, rendered line) and notes

Table Generation Mode (Top Section)

Opcode Mode

Choose which Csound table opcode style to generate:

- `ftgen`
- `ftgenonce`

Routine Selection

You can choose from structured editors for common routines and a named routine option:

- `GEN01` - Audio File
- `GEN02` - Value List
- `GEN07` - Segments
- `GEN08` - Cubic Spline Segments
- `GEN10` - Harmonic Sine Partials
- `GEN11` - Harmonic Cosine Partials
- `GEN17` - Step Table From x/y Pairs
- `GEN20` - Window / Distribution Function
- `GENpadsynth` (named routine)
- Custom routine number / raw arguments

Core Table Fields

- `Routine Number`
- `Table Number`
- `Table Size`
- `Start Time` (disabled for `ftgenonce` UI mode)
- `Normalize table` (positive GEN number vs negative GEN number behavior)

The backend validates GEN table settings before storage and again before Csound orchestra emission. `Table Size` can be

`0` only for `GEN01`; other routines require a positive size. The maximum table size is `4194304` samples, and raw or structured GEN argument lists are capped at 512 entries. Raw argument text is limited to 8192 characters, with each token limited to 512 characters.

Supported Routine Editors (Structured UI)

Routine	Purpose	Main Parameters In Editor
`GEN10`	Harmonic sine partial amplitudes	Harmonic amplitude list
`GEN11`	Harmonic cosine partials	harmonic count, lowest harmonic, multiplier
`GEN02`	Literal value list	Numeric value list
`GEN07`	Segments	Start value + repeated length/value rows
`GEN08`	Cubic spline segments	Start value + repeated length/value rows
`GEN17`	Step table from x/y pairs	x/y pairs (editable list)
`GEN20`	Window / distribution	window type, max, optional parameter (for selected windows)
`GEN01`	Load audio file into table	uploaded asset, skip time, format, channel

`GENpadsynth` (Named Routine)

Orchestron supports a named GEN routine for padsynth (`GENpadsynth`).

- Select the `GENpadsynth` routine option.
- Enter padsynth parameters and partial amplitude/frequency pairs in the raw arguments editor.
- Argument order follows the Csound `GENpadsynth` documentation.

Custom / Raw Routine Arguments

For routines without a dedicated structured editor:

- Use the raw arguments editor.
- Separate tokens by commas or new lines.
- Quote string literals manually (for example `"file.wav"`).
- Prefix an argument with `expr:` to force raw Csound expression rendering without quotes.
- Keep raw argument lists within the backend validation limits: 512 tokens total, 512 characters per token, and 8192 characters for the raw text field.

Examples:

```
1, 0.5, expr:1024*2, "file.wav"
```

GEN01 Audio File Workflow (Important)

The `GEN01` editor loads sound files through uploaded backend assets:

Uploaded Asset

- Click `Upload Audio File`
- Select an audio file (UI accepts common formats such as WAV/AIFF/FLAC/MP3/OGG and `audio/*`)
- Orchestron stores the uploaded asset on the backend and references it in the GEN node config

Benefits:

- Instrument/performance exports can include the audio file automatically
- Imports can restore the asset automatically when using ZIP bundles
- Runtime compilation resolves the asset inside the configured backend asset directory
- Raw absolute paths and parent-traversal paths are rejected before Csound starts

Additional GEN01 Parameters

- `Skip Time`
- `Format`
- `Channel`

Upload Limits and Notes

- Default backend upload limit for GEN audio assets is 64 MiB per file (`VISUALCSOUND_GEN_AUDIO_ASSET_MAX_BYTES`)
- Persistent generated-asset storage is also capped by total bytes (`VISUALCSOUND_GEN_AUDIO_ASSETS_MAX_TOTAL_BYTES`, 1 GiB by default) and asset count (`VISUALCSOUND_GEN_AUDIO_ASSETS_MAX_COUNT`, 1024 by default)
- Garbage collection removes unreferenced generated assets older than `VISUALCSOUND_GEN_AUDIO_ASSET_GC_MIN_AGE_SECONDS` (24 hours by default)
- The backend streams GEN audio uploads and rejects requests once they exceed the configured cap
- ZIP bundle imports also enforce request, member-count, JSON, and total-uncompressed-size caps before importing bundled assets
- Deployments should set a matching maximum request size at the ASGI server, reverse proxy, or load balancer
- The preview/note panel explains that uploaded assets are required for GEN01 sample loading

Preview Panel

The preview section shows:

- `Effective GEN` (including normalization sign / named routine behavior)
- Flattened argument list
- Rendered `ftgen` / `ftgenonce` line preview

Use this to verify the exact Csound line before compile.

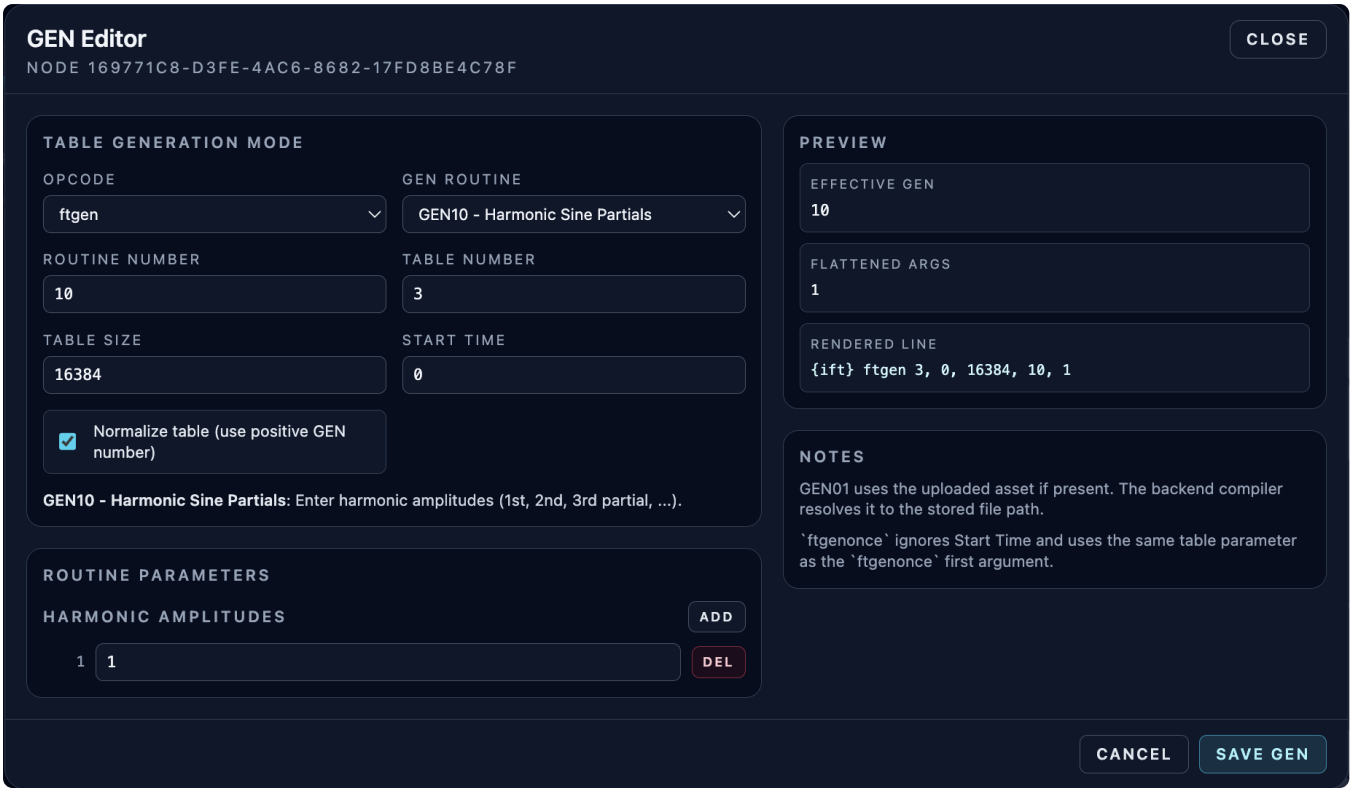
Editor Notes Worth Knowing

- `ftgenonce` behavior differs from `ftgen` and the editor shows notes about the differences.
- `GEN01` asset-backed generation is resolved by the backend compiler to contained stored asset paths.
- `GEN01` asset dependencies are automatically bundled in ZIP exports when referenced.

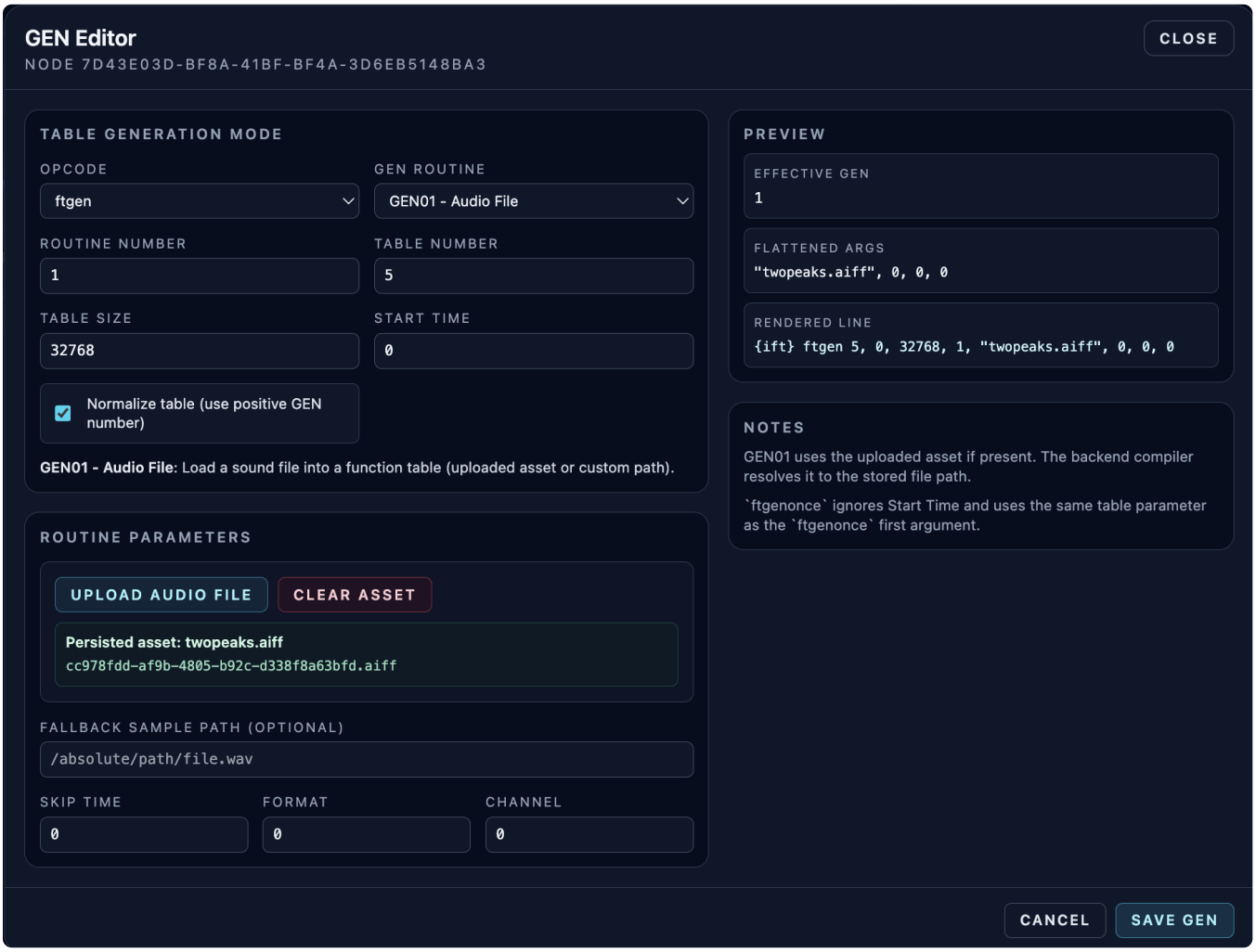
Save Behavior

- `Save GEN` writes the GEN node configuration into patch UI metadata (`graph.ui_layout`).
- Closing without saving leaves the previous GEN node settings unchanged.

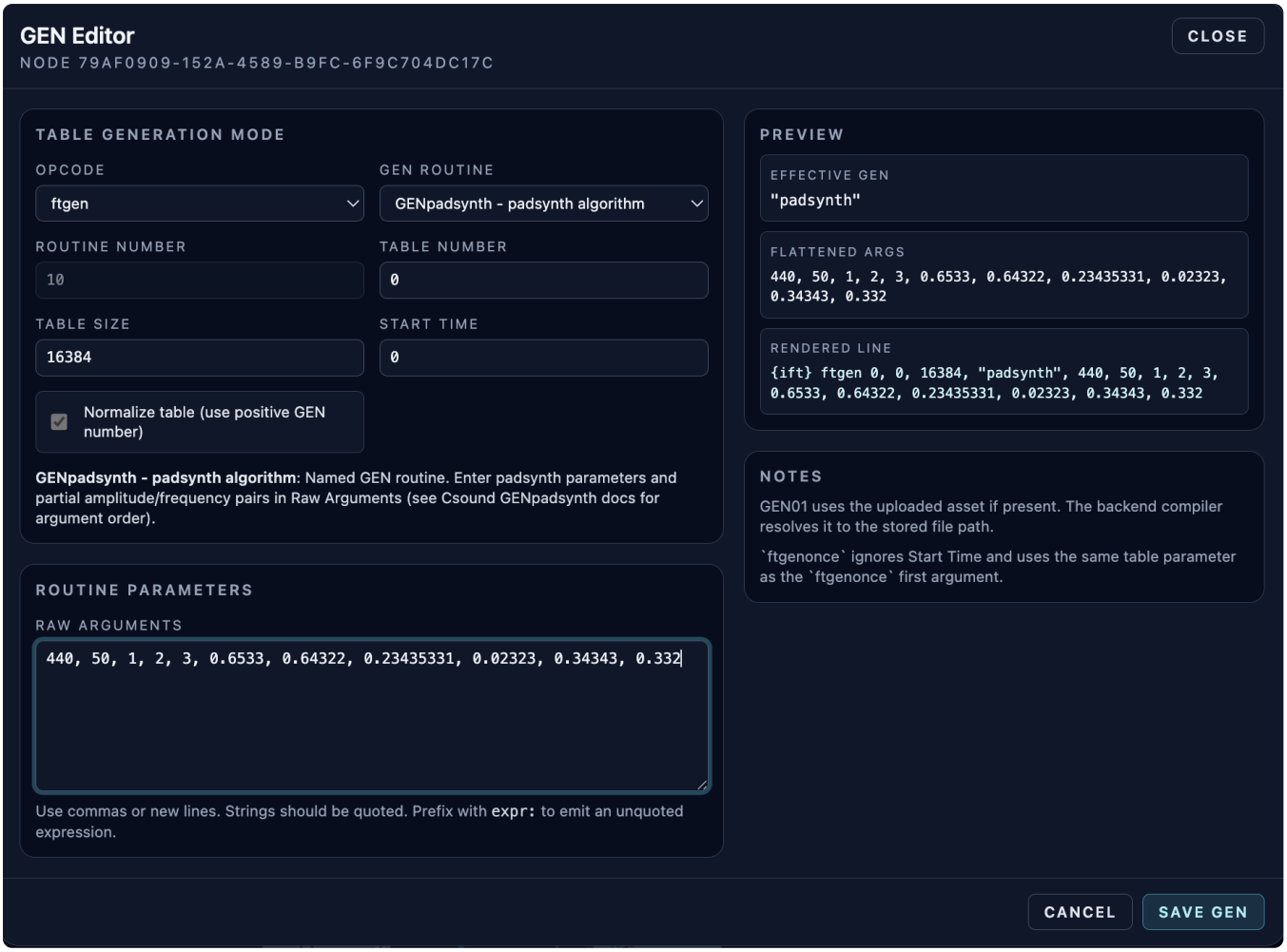
Screenshots



GEN editor in structured routine mode with live preview panel.



GEN01 table setup using an uploaded backend audio asset.



GENpadsynth named routine using raw arguments input.

Performance

This chapter covers the `Perform` page (labeled `Perform / Performance` depending language), where you build a playable multi-instrument setup and perform it live.

What You Can Do Here

- Assemble an instrument rack from saved patches and assign MIDI channels
- Mix audio with dB faders, pan/balance, mute/solo, meters, inserts and pre/post sends
- Route instruments and returns through Master or explicit direct paths
- Start/stop the instrument engine session
- Create multiple melodic sequencers, drummer sequencers, and controller sequencers
- Add backend-run arpeggiators that turn held notes into routed arpeggiated instrument output
- Use pattern pads with queued switching and pad-loop sequences
- Arrange multiple track timelines in the multitrack arranger with shared transport and loop-range playback
- Perform live with piano rolls and manual MIDI controller knobs
- Save/load/clone/delete performances
- Import/export performance bundles (with optional patch definitions)
- Export offline Csound render ZIPs either with `.csd + .mid` playback or with inline Csound score playback

Chapter Contents

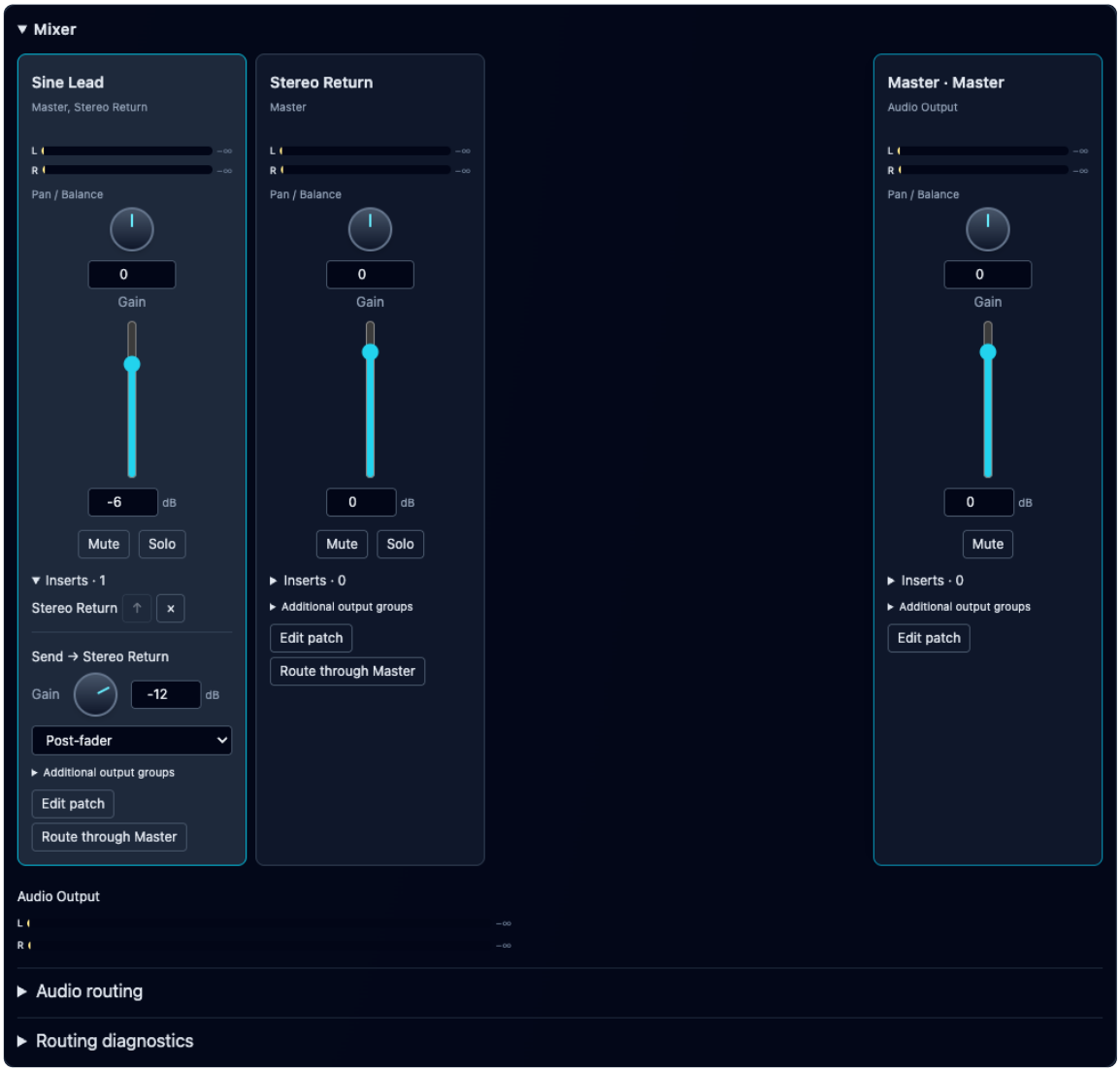
Important Concept: Patch vs Performance

- **Patch (Instrument Design page):** one instrument graph
- **Performance (Perform page):** a full live setup that references one or more patches and stores sequencer/piano-roll/controller state plus explicit audio routes and mixer settings

Practical Workflow

1. Save at least one patch in Instrument Design.
2. Add instruments to the performance rack and assign channels.
3. Open Mixer, check the destination of each strip, and connect effects/returns. Guided instruments get a neutral Master automatically; existing direct routes stay direct.
4. Use Routing diagnostics → Check routing, then Start Instruments. Adjust existing mixer controls during playback.
5. Build melodic sequencers / drummer sequencers / controller sequencers / piano rolls.
6. Save the performance to retain the rack, routes and current mix.
7. Export the performance bundle for backup/sharing, or use `Export CSD (MIDI) / Export CSD (SCORE)` when you need an offline Csound render package.

Screenshots



Perform mixer with a post-fader send, dedicated insert and pinned Master. See the following chapters for rack and sequencer controls.

Instrument Rack and Engine Transport

The Instrument Rack is the top section of the Perform page and controls the live session, instrument assignments, and performance metadata.

Performance Metadata and Library Actions

The rack includes fields and actions for the current performance:

- `Performance Name`
- `Description`
- `Load Performance` dropdown
- `Save Performance`
- `Clone`
- `Delete`
- `Export`
- `Export CSD (MIDI)`
- `Export CSD (SCORE)`
- `Import`

These actions operate on the performance configuration (instrument rack + audio routing/mixer + sequencers + controllers + piano rolls), not on individual patch definitions.

- `Export` writes an Orchestron `.orch.json` / `.orch.zip` performance bundle for backup, sharing, and re-import.
- `Export CSD (MIDI)` writes an offline-render ZIP with a compiled `.csd`, the arranger performance as `.mid`, bundled uploaded sample/SF assets, and a `README.txt` with the render command. `Export CSD (SCORE)` embeds notes and controller sweeps directly in the Csound score, omits the `.mid`, rewrites supported MIDI opcodes for score playback, and writes a matching no-`-F` render command. Both modes seed enabled manual MIDI Controller lane values at time 0 on each assigned instrument channel and use 32-bit float WAV output (`-f`) to preserve headroom. GEN01 and `sfload` sample files must be uploaded/imported assets; raw local `samplePath` values are rejected before compilation.

Instrument Assignments (Rack Slots)

Each rack entry selects a saved patch and a MIDI channel (1–16), or displays Continuous activation. Up to 64 user patch instances are supported, including processors. Generated mixer stages do not occupy rack slots or MIDI channels. Use distinct MIDI channels for note-triggered instances.

The separate audio mixer below the rack provides audio faders, pan/balance knobs, mute/solo, pre/post sends, inserts and meters. The old numeric Level field has been removed. Mixer gain affects held notes and external MIDI audio without changing note velocity.

While instruments run, adding/removing assignments, changing patch/channel assignments, and changing routing topology are locked. Existing mixer controls stay live. Use **Stop to edit routing** before changing connections or insert order.

Add Instrument

- Use `Add Instrument` to create another rack slot.
- This enables multi-instrument performances driven by different MIDI channels.
- Continuous effects run as explicit rack instances. Adding an insert creates a dedicated instance automatically; simply saving an effect in the library does not start it.
- In the `orchestron-performance-creator` CLI, use `edit instruments list` to discover stable rack binding IDs and audio ports. Build arbitrary chains with `edit routes add/remove/clear/list`, or run `edit add-standard-effects` to add the standard reverb, compressor, and speaker-output send/dry matrix.
- The button is unavailable while the engine is running; stop instruments before changing rack assignments.

The CLI validates its staged rack through the same backend route resolver used by session creation and compilation. Run

`edit validate`, then `edit create-runtime --start` to create a CLI-owned engine session. Because instrument assignments and Csound audio connections are fixed at compile time, use `edit rebuild-runtime` after changing the rack or its routes; `edit push-runtime` updates mixer, sequencer and arpeggiator settings when the compiled rack still matches.

Rack Transport (Instrument Engine Control)

The rack transport controls the instrument runtime session itself.

Buttons:

- `Start Instruments`
- `Stop Instruments`

`Start Instruments / Stop Instruments`

These start/stop the underlying instrument engine session.

Global arrangement transport now lives in the multitrack arranger section. There, cassette-style `Rewind`, `Stop`, `Play`, and `Fast forward` buttons drive sequencers that have `Pad Looper` enabled and move the shared playhead in `1-beat` transport blocks. Arranger `Play` stops sequencers whose `Pad Looper` is off, while arranger `Stop` stops only the pad-loop-driven sequencers; manually started non-pad-loop sequencers can keep running. The arranger `Stop` button does not stop the instrument engine; `Stop Instruments` in the rack does that. Double-clicking arranger `Stop` resets the playhead to the selected loop start or to step `0` when no manually running sequencer keeps transport active.

Session State Badge

The rack shows current session state (localized label), for example:

- running
- stopped / idle

This is the state of the instrument engine session, not just the sequencer transport.

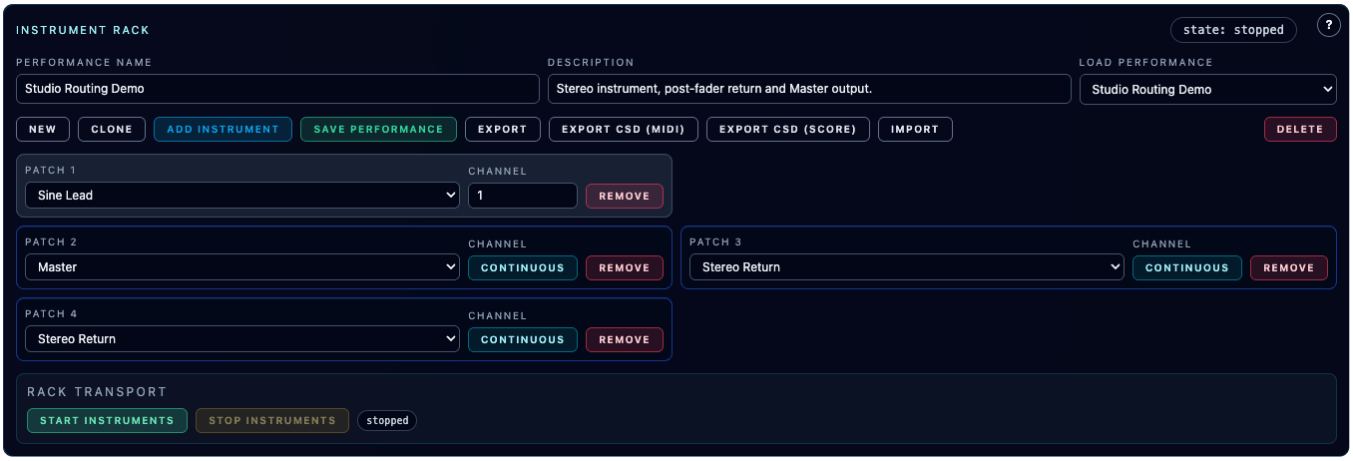
Error Banner

If engine start/stop or transport actions fail, the Perform page shows an error banner below the rack transport section.

Tips

- Save the performance after significant changes (rack assignments, sequencers, piano rolls, controller mappings).
- Use distinct MIDI channels for note-triggered rack instances.
- Use `sname` labels on source `outleta` nodes to name the available matrix channels. The label can be stored directly on the node or supplied by a direct `const_s` connection.

Screenshots



Instrument rack detail with performance metadata, assignments, and transport controls.

Melodic Sequencers and Step Editing

Melodic sequencers are step-based pattern sequencers used for note playback.

This page covers the **melodic sequencer** editor. Drum-machine style programming is documented in Drummer Sequencers.

Global Sequencer Clock

The sequencer section contains a global clock with:

- Running/stopped state badge
- `BPM` field (range `30..300`)

The backend runs the step timing (native runtime clock), which reduces browser timing jitter during playback.

Adding / Removing Melodic Sequencers

- `Add Melodic Sequencer` creates a new melodic sequencer card
- `Add Drummer Sequencer` creates a drummer sequencer card (documented separately in Drummer Sequencers)
- Each melodic sequencer has its own `Remove` button

Per-Sequencer Controls

Each melodic sequencer card provides:

- sequencer state badge (running/stopped or queued start/stop state)
- `Start / Stop` (sequencer enable state; can be used manually while the multitrack arranger is stopped)
- `Remove`
- `Clear Steps`
- `MIDI Channel` (`1..16`)
- `Scale` (root + scale type)
- `Mode` (Ionian, Dorian, Phrygian, Lydian, Mixolydian, Aeolian, Locrian)
- `Meter` (`2..7` over `4` or `8`)
- `Grid` (`2`, `4`, or `8`, steps per beat)
- `Beat Ratio` (`1:1`, `2:1`, `3:2`, `4:3`, `3:4`, `5:4`, `4:5`, `7:4`)
- `Beats` (`1..8`, with the current meter numerator exposed directly)

The step editor width is derived from the sequencer's own timing and length:

- steps per beat = the sequencer's selected `Grid`
- pad steps = `beats * steps per beat`
- default timing (`4/4`, grid `4`) gives `16` steps for a `4`-beat pad
- odd meters can still use matching one-bar pad lengths such as `3` beats in `3/4` or `5` beats in `5/4`

`Beat Ratio` changes how quickly the sequencer advances relative to the shared transport beat without changing its stored pad length, meter, or grid:

- `1:1` keeps normal speed
- ratios above `1:1` make the melodic sequencer run faster
- ratios below `1:1` make the melodic sequencer run slower

This is separate from `Meter` and `Grid`, so you can combine polymeter and true polyrhythm on the same performance page.

Queued Start/Stop State Labels

When changes are queued to take effect at the next cycle boundary, the sequencer state label can show queued states such as:

- starting at step 1
- stopping at step 1

Scale, Scale Type, and Mode

Orchestron separates:

- Scale root (for example `C`, `Db`, `F#`)
- Scale type (`major`, `neutral`, `minor`)
- Mode (7 diatonic modes)

This supports guided note entry and better readability in the sequencer step editor and piano roll highlighting.

Step Editor (Per-Step Note Programming)

Each step cell supports:

- Rest or note selection
- Chord selection, including the standard `5` power chord (root + perfect fifth)
- Hold toggle (`HOLD`)
- Octave field (separate from pitch class)
- In-scale/out-of-scale status display
- Playhead highlighting during playback

Step Note Entry Workflow

Each step has a pitch-class dropdown grouped into:

- `Rest`
- In-scale notes (for the current sequencer scale/mode)
- Out-of-scale notes

The octave is edited separately via the `OCT` field.

This split note/octave design makes pattern entry faster than selecting full MIDI note numbers.

Theory Aids In Steps

- In-scale notes are highlighted
- In-scale notes show degree labels (`1..7`)
- Out-of-scale notes remain selectable (for chromatic writing)
- Enharmonic note labels are available where relevant (for example sharps/flats)
- Chord choices are grouped as none, diatonic, or chromatic for the selected step root and mode. A `5` power chord is diatonic only when both the root and perfect fifth are in the current mode.

Hold Toggle

Each step includes a `HOLD` toggle.

Use `HOLD` to sustain behavior across steps according to the sequencer runtime logic and the note/gate pattern.

Step Cell Visual States

Step cells visually indicate:

- Active playhead step (when the melodic sequencer + transport are running)
- In-scale programmed note
- Out-of-scale programmed note

- Rest / hold combinations

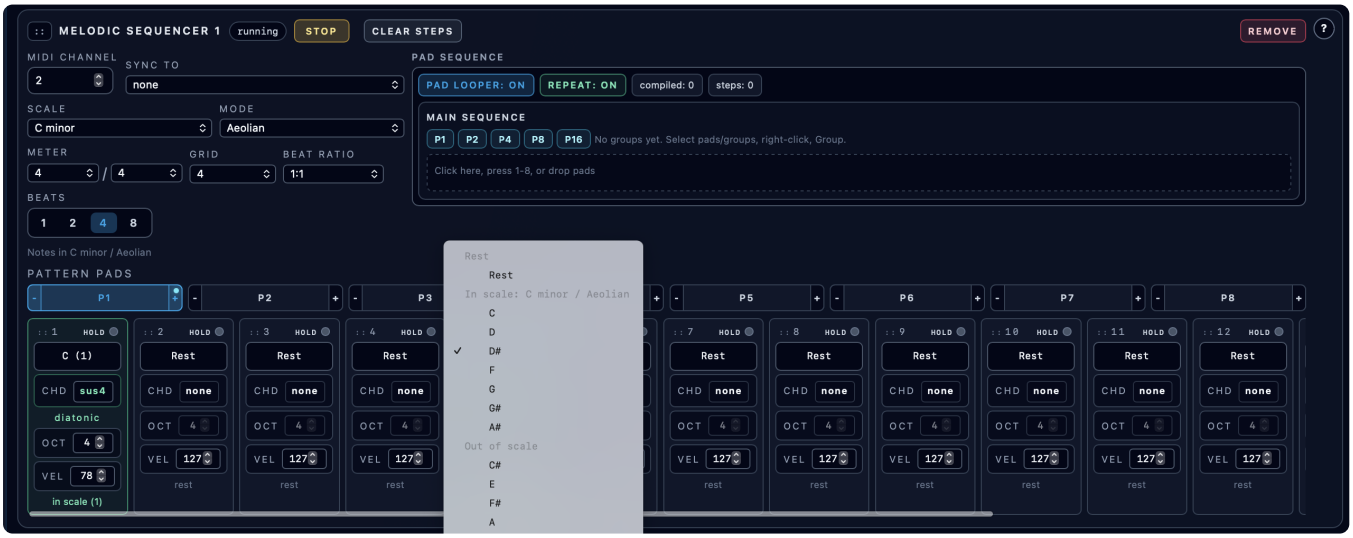
Clear Steps

`Clear Steps` resets the current melodic sequencer step contents (for the active pad pattern context) so you can quickly reprogram it.

Related Features

- Pattern pads (P1..P8), queued pad switching, pad copying, transposition, and pad-loop sequence controls are documented in Pattern Pads, Queued Switching, and Pad Looper.

Screenshots



Melodic sequencer detail showing step editing, note selection, octave, and hold controls.

Drummer Sequencers

Drummer sequencers are drum-machine style step sequencers for fixed MIDI drum keys.

They are optimized for per-step drum hit programming instead of melodic note/chord entry.

What Makes Them Different

Compared to melodic sequencers, drummer sequencers:

- use MIDI key numbers (`0..127`) per row (drum instrument selection)
- do not use scale / mode / chord controls
- do not use pad transpose edge buttons
- let you program multiple drum rows per step
- store velocity per active drum hit (per row + step)

Adding / Removing Drummer Sequencers

- Use the `Add Drummer Sequencer` button in the sequencer section header to add a drummer sequencer card
- Each drummer sequencer card has its own `Remove` button

Per-Drummer-Sequencer Controls

Each drummer sequencer card provides:

- sequencer state badge (running/stopped or queued start/stop state)
- `Start / Stop` (sequencer enable state; can be used manually while the multitrack arranger is stopped)
- `Remove`
- `Clear Steps`
- `+ Key` (add another drum row)
- `MIDI Channel (1..16)`
- `Meter (2..7 over 4 or 8)`
- `Grid (2, 4, or 8, steps per beat)`
- `Beat Ratio (1:1, 2:1, 3:2, 4:3, 3:4, 5:4, 4:5, 7:4)`
- `Beats (1..8, with the current meter numerator exposed directly)`
- Pad Looper controls (same concept as melodic sequencers)

Drum Rows (`Keys`)

The left side of the drummer grid contains vertically stacked drum rows.

Each row has:

- row index label
- MIDI key input (`0..127`)
- row remove button (`x`)

Auditioning Drum Keys While Editing

When the instrument engine session is running, changing a row key sends a short one-shot MIDI note preview on the drummer sequencer's MIDI channel.

This helps identify which drum sound is mapped to a given MIDI key number.

Drum Step Grid (LED Matrix)

The grid is row-based:

- rows = selected drum keys
- columns = `beats * steps per beat`

This keeps the `Keys` column horizontally aligned with the LED rows.

LED States

- inactive step: dark LED with visible border
- active step: red LED
- current playing column: highlighted across every drum row
- active step in the current playing column: flashing green LED

Velocity (Per Hit)

Velocity is shown by LED color saturation (stronger color = higher velocity).

The LED border color stays visible even at low velocities, so low-velocity active hits remain distinguishable from inactive steps.

While dragging vertically on a hit, the UI also shows a live numeric `velocity: xx` readout next to the edited step.

Editing Workflow

- Click an inactive LED to activate a hit
- Click an active LED (without dragging) to deactivate it
- Click and drag vertically on an LED to change velocity (`0..127`)
- While dragging, watch the live `velocity` readout for the exact value
- Keyboard:
- `Enter` / `Space` toggles the hit
- `Arrow Up` / `Arrow Down` adjusts velocity

Pattern Pads and Pad Looper

Drummer sequencers support the same `P1..P8` pattern-pad workflow as melodic sequencers.

If a drummer sequencer is running, pad changes are queued to the next loop boundary. If it is stopped, pad changes apply immediately so you can edit another pad while the shared transport keeps running elsewhere.

Drummer pad length is stored in beats, while meter, grid, and beat ratio are configured per drummer sequencer.

This keeps drummer pads beat-based while still allowing one-bar patterns in odd meters such as `3/4`, `5/4`, or `7/8` by matching the pad beat count to the meter, while `Beat Ratio` changes how fast the row pattern cycles against the shared transport without changing the stored beat length.

They also support pad-loop sequences (`Pad Looper`, `Repeat`, pad sequence list).

Differences vs melodic sequencer pads:

- no transpose edge buttons (`-` / `+`)
- pad content is drum-row hit data + per-hit velocities (not note/chord/theory state)

Screenshots

Drummer sequencer showing multiple drum rows, per-step hits, and velocity editing on the LED grid.

Typical Uses

- Kick / snare / hat step programming
- Percussion layers with multiple rows
- Switching between groove variations using pattern pads
- Automating drum pattern changes with pad looper while performing on piano roll/controllers

Pattern Pads, Queued Switching, and Pad Looper

Pattern pads are the per-track pattern banks (`P1..P8`). They support live switching, copying, and pad-loop sequencing for melodic, drummer, and controller sequencers (with transposition available on melodic sequencer pads only).

Pattern Pads Overview

Each melodic sequencer, drummer sequencer, and controller sequencer contains 8 pattern pads:

- `P1` to `P8`
- one active pad
- optional queued pad during live playback

Pad length is stored per pad in beats:

- melodic sequencer pads: `1..8` beats
- drummer sequencer pads: `1..8` beats
- controller sequencer pads: `1..8` beats, plus `16` beats

Meter and grid belong to the owning sequencer instance, not to individual pads.

The pad-length controls keep the common musical shortcuts (`1` , `2` , `4` , `8`) visible and also expose the owning sequencer's current meter numerator directly, so a `3/4` , `5/4` , or `7/8` sequencer can reach a one-bar pad length without leaving the beat-based model.

Each pad stores the pattern content for that track:

- melodic sequencers: step notes/holds and related pad theory state used by pad operations
- drummer sequencers: per-row drum hits and per-hit velocities
- controller sequencers: controller curve/keypoint patterns

Pad States

Pad buttons use visual states to indicate:

- Active pad
- Queued pad (will switch on loop boundary)
- Idle pad

Live Pad Switching

Pad press behavior depends on transport state:

- When sequencer transport is stopped: pad selection changes immediately
- When sequencer transport is running and that sequencer is also running: the pad switch is queued and applied on the next loop boundary, restarting the new pad at step `0`
- When sequencer transport is running but that sequencer is stopped: pad selection still changes immediately for editing

This avoids mid-pattern timing glitches and keeps pattern changes musical.

Pad Copy (Drag-and-Drop)

You can copy one pad onto another by dragging and dropping pad buttons.

What gets copied:

- step note/hold data

- pad scale/mode settings used by pad-aware behavior (as documented in the integrated help)

This is the fastest way to create variations.

Pad Edge Transpose Buttons (– / +) (Melodic Sequencer Pads)

Melodic sequencer pad buttons include small edge buttons for transposition.

Drummer sequencer pads do **not** include transpose edge buttons.

Short Press (Quick Click)

- Transposes stored notes within the current scale by one degree up/down
- Keeps the same scale root and mode

Use this for quick harmonic variants of the same pattern.

Long Press (Hold, about 350 ms)

- Performs a diatonic key-step transpose (moves pad tonic/root to adjacent scale degree)
- Keeps the mode
- Updates the pad scale root accordingly

Use this to move the pattern to a new key center while preserving mode character.

Pad Looper (Pad Sequence)

Each track includes a **Pad Looper** section that can play a sequence of pattern tokens automatically.

Controls:

- `Pad Looper: On/Off`
- `Repeat: On/Off`
- Pad Sequence entry area

Building The Pad Sequence

You can add pad steps to the pad-loop sequence by:

- Clicking the sequence area and pressing keyboard `1..8`
- Dragging pad buttons into the sequence area

Pause Tokens (Silence Patterns)

The pad looper supports explicit silence tokens:

- `P1`
- `P2`
- `P4`
- `P8`
- `P16`

Each token inserts a pause segment for the given number of beats.

How to use them:

- Click a pause token button to append it to the current sequence editor
- Drag a pause token into root, group, or super-group sequences

Pause tokens are always available in the editor token list, even when not used anywhere in the pattern.

Sequence Display

The pad sequence area shows:

- ordered sequence tokens (pads, pauses, groups, super-groups)
- remove (`x`) button per sequence item
- current pad-loop position highlight while running

Nested Pad Sequences (Groups and Super-Groups)

The pad looper now supports **nested pattern sequences** on all sequencer types.

You can build reusable structures in two hierarchy levels:

- **Groups** labeled with capital letters (`A` , `B` , `C` , ...)
- **Super-groups** labeled with roman numerals (`I` , `II` , `III` , ...)

This lets you create sequences such as:

- root sequence: `A B B A`
- where `A` and `B` are reusable grouped pad patterns

Creating Groups / Super-Groups

- In the pad sequence editor, select multiple sequence items (Ctrl/Cmd-click)
- Right-click to open the context menu
- Choose:
- `Group` to create a lettered group
- `Super-group` to create a roman-numeral super-group (from selected groups)

Editing Groups (Reference-Based)

- Click a group (`A`) or super-group (`II`) to open/edit it
- The selected group expands into its own editable sequence area
- Changes are stored by reference, so **all occurrences** of that group/super-group in the root sequence use the updated content

You can edit grouped content by:

- reordering items (drag-and-drop)
- adding pads (`1..8` or drag pad buttons)
- adding pause tokens (`P1` , `P2` , `P4` , `P8` , `P16`)
- removing items
- ungrouping selected items back inline

Recursion / Hierarchy Rules

To prevent recursive pattern definitions, hierarchy rules are enforced:

- groups can contain pads and pause tokens
- super-groups can contain pads, pause tokens, and groups
- root sequence can contain pads, pause tokens, groups, and super-groups
- same-level or higher-level nesting is blocked

Playback Highlighting (Nested)

While the sequencer is running, the pad looper highlights:

- the currently playing item in the **root sequence**
- the active group/super-group reference when the runtime position is inside it
- the currently playing item **inside an opened group/super-group editor**

Visual Color Coding

Nested pad-sequence tokens are color-coded by hierarchy:

- pads: green shades
- pauses (`P1` , `P2` , `P4` , `P8` , `P16`): cyan shades
- groups (`A` , `B` , `C` , ...): orange shades
- super-groups (`I` , `II` , `III` , ...): violet shades

Repeat Toggle

- `Repeat: On` loops the pad sequence continuously
- `Repeat: Off` runs through the programmed pad sequence once and then stops that sequencer
- `Pad Looper: Off` leaves the sequencer on its current pad instead of stepping through the pad sequence

When To Use Pad Looper vs Manual Pad Presses

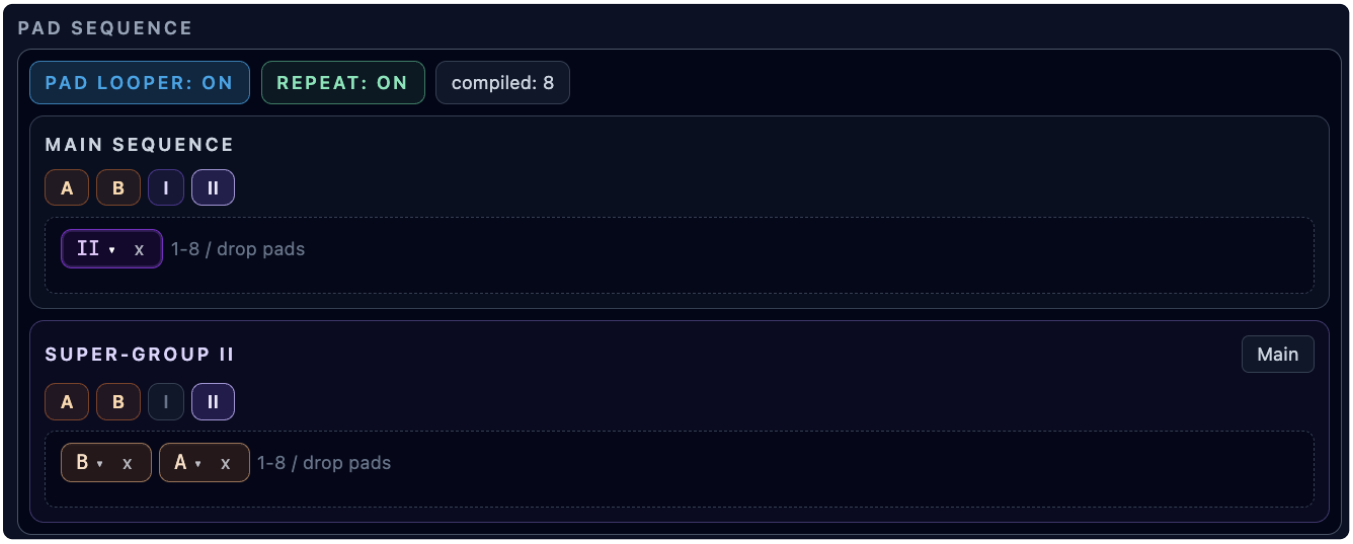
Use manual pad presses for performance improvisation.

Use pad looper when you want:

- repeatable song-form-like pattern changes
- scripted variation cycles
- hands-free movement through pad banks while you play piano rolls/controllers

Screenshots

Pattern pads with transpose edge buttons and pad-loop sequence controls.



Nested pad-loop editor with reusable groups (A, B, C, ...) and super-groups (I, II, III, ...).

Multitrack Arranger

The **Multitrack Arranger** is a shared timeline view for pad-loop arrangements across all sequencer types. It appears on the Perform page when at least one melodic sequencer, drummer sequencer, or controller sequencer exists.

What It Arranges

The arranger shows one row per track:

- melodic sequencers
- drummer sequencers
- controller sequencers

Each row includes:

- track title (`Melodic Sequencer N`, `Drummer Sequencer N`, `Controller Sequencer N`)
- track subtitle (MIDI channel + assigned patch name, or `CC N` for controller sequencers)
- a root pattern timeline aligned to the shared transport beat grid

Root Timeline Tokens

The root timeline displays arranged pattern tokens:

- pad tokens (`1..8`)
- group tokens (`A`, `B`, `C`, ...)
- super-group tokens (`I`, `II`, `III`, ...)

Pause tokens are part of the underlying data model, but they are hidden in the root timeline to keep the overview compact.

The arranger always shows the current absolute playhead position, even while stopped.

Arranger Transport

The arranger header provides cassette-style transport controls with icon buttons:

- `Rewind`: move the playhead `1` beat backward
- `Stop`: stop sequencers whose `Pad Looper` is on while preserving the current playhead position; manually started sequencers whose `Pad Looper` is off keep running
- `Play`: start the instrument engine if needed, start sequencers whose `Pad Looper` is on, and stop sequencers whose `Pad Looper` is off so arranger playback stays synchronized
- `Fast forward`: move the playhead `1` beat forward
- `?`: open the integrated multilingual help modal for a concise arranger workflow summary

The transport controls the pad-loop arrangement for melodic sequencers, drummer sequencers, and controller sequencers. Individual sequencer `Start / Stop` buttons remain available for composing and auditioning patterns outside arranger playback. Arpeggiators, piano rolls, and manual MIDI controller lanes remain individually controlled.

Double-click `Stop` to reset the playhead to the selected loop start. If no loop range is selected, the playhead resets to step `0`.

Root Timeline Editing

You can edit arrangement structure directly on each row:

- click to select a token
- use `Ctrl/Cmd/Shift + click` for additive multi-selection
- right-click to open the context menu

Context menu actions:

- `Add pad` submenu (insert a pad token at the clicked pause gap, or append at the end)
- `Add group` submenu (insert an existing lettered group token)
- `Add super-group` submenu (insert an existing roman-numeral super-group token)
- `Copy / Paste` (duplicate selected pads/groups/super-groups; root pastes use a large-enough pause gap or the sequence end)
- `Group` (creates a lettered group)
- `Super-group` (creates a roman-numeral super-group)
- `Ungroup` (expands grouped content inline)
- `Remove` (deletes selected tokens)

Drag Reorder in Root

Use the `::` handle on a token to drag it on the root timeline.

- drag moves are quantized to the shared transport beat grid
- dragging a selected contiguous block moves that whole block
- dropping beyond current content can extend the timeline
- when needed, pause spans are re-materialized automatically to preserve timing structure
- quick swap is supported for single-token moves onto an adjacent non-pause token with matching length

Group/Super-group Editor

Click a group or super-group token to open its nested editor.

In the opened editor, you can:

- return to root with `Main`
- append pads using `1..8`
- append pauses using `P1`, `P2`, `P4`, `P8`, `P16`
- drag-and-drop tokens to reorder within the same container
- remove individual tokens with `x`
- right-click for `Add pad`, `Add group`, `Add super-group`, `Copy`, `Paste`, `Group`, `Super-group`, `Ungroup`, `Remove`

The editor also shows `Total steps` for the opened container, resolved from beat lengths on the shared transport grid.

Zoom and Timeline Navigation

Arranger timeline controls:

- cassette-style transport buttons
- `Zoom -` / `Zoom +`
- live zoom percent readout
- horizontal scroll via mouse wheel (and `Shift + wheel` support)
- bottom scrollbar for long timelines

Loop Range Selection

Above the horizontal scrollbar, the arranger shows a shared loop-range ruler quantized to beat blocks.

- drag on the ruler to define a playback range, including a single beat
- click the highlighted range to clear the selection
- the selected span is highlighted across all arranger rows
- when a range is selected, playback loops inside that range
- when no range is selected, playback continues until stopped; continuously looping tracks keep cycling according to

their own repeat settings

Keyboard Shortcuts

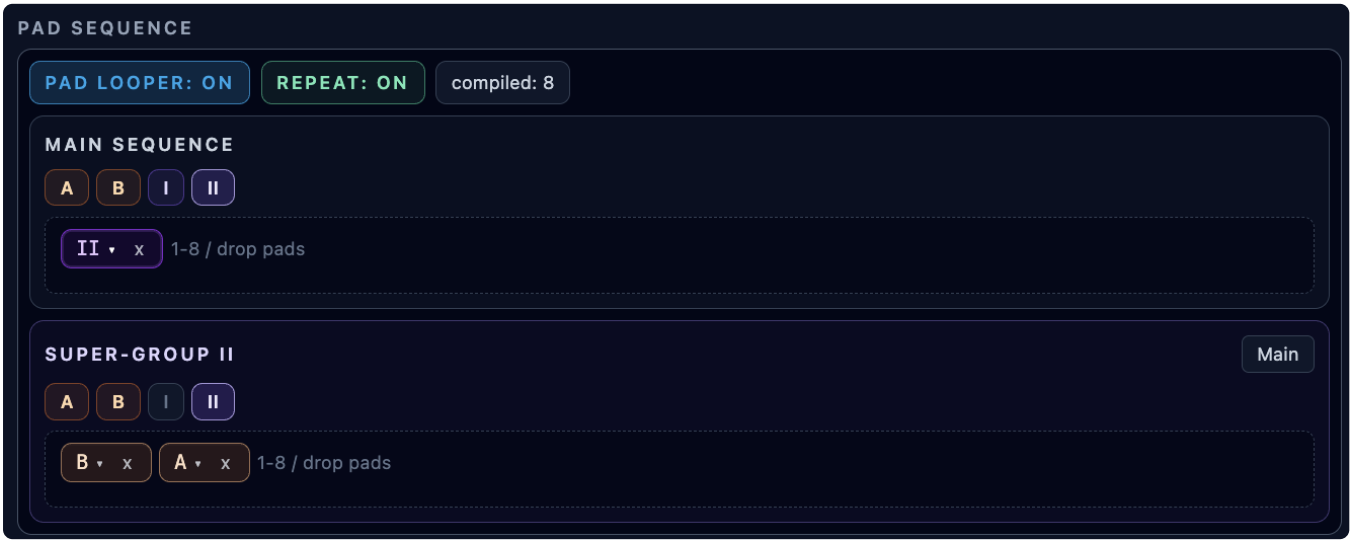
When a timeline container is focused:

- `1..8` : append corresponding pad token
- `Delete / Backspace` : remove current token selection

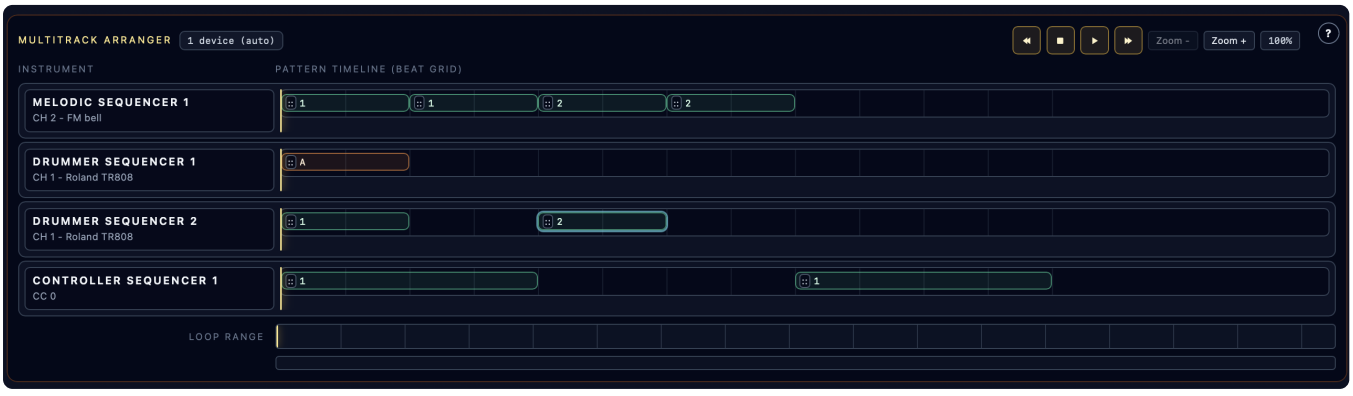
Notes

- Arranger edits write directly into each track's pad-loop pattern state.
- Playback range selection is stored with the performance and restored when the performance is loaded again.
- The section currently shows `1 device (auto)` as the device summary.
- If no sequencer-type tracks exist, the multitrack arranger is hidden.

Screenshot



Nested sequence editing inside the pattern pad sequencer (groups and super-groups).



Multitrack arranger (track view, all instruments).

Controller Sequencers

Controller Sequencers automate MIDI CC values using editable curves.

What A Controller Sequencer Does

A controller sequencer sends its programmed CC curve to the backend sequencer, which samples that curve over a repeating length and emits timed MIDI Control Change messages during playback.

Typical uses:

- filter sweeps
- morph controls
- modulation depth animation
- macro movement synchronized to transport

Adding / Removing Controller Sequencers

- `Add Controller Sequencer` creates a new controller sequencer card
- Each controller sequencer has a `Remove` button

Per-Controller-Sequencer Controls

Each controller sequencer provides:

- Running/stopped state badge
- `Start / Stop` enable toggle; can be used manually while the multitrack arranger is stopped
- `Controller # (0..127)`
- `Meter (2..7 over 4 or 8)`
- `Grid (2, 4, or 8, steps per beat)`
- `Beat Ratio (1:1, 2:1, 3:2, 4:3, 3:4, 5:4, 4:5, 7:4)`
- Curve length in beats (`1..8`, plus `16`)
- CC label preview (`CC N`)
- Curve editor

Curve Length (`1..8`, plus `16`, beats)

This defines the repeat length for the curve sampling relative to that controller sequencer's own timing. Keypoints stay normalized across the full pad duration, so the same curve shape stretches automatically when you choose a longer beat length.

`Beat Ratio` changes how quickly the controller sequencer moves through that curve relative to the shared transport. Faster ratios create repeating automation polyrhythms without changing the stored keypoint positions.

Curve Editor Interactions

The curve editor is an interactive graph view (spline-based curve display and sampling).

Supported interactions:

- Click background to add a point (interior points only)
- Drag a point to change position/value
- Double-click an interior point to remove it
- Endpoints are boundary anchors (positions remain at start/end of the curve)

Visual Playback Indicators (When Running)

When the main transport is running and the controller sequencer is enabled, the editor shows:

- a vertical playback position line
- a marker showing the currently sampled CC value on the curve

This makes it easy to understand exactly what value is being sent at each transport moment.

Live Use Notes

- Controller sequencers run alongside melodic sequencers on the same backend transport clock.
- While transport is running, pad presses are queued and applied on the next controller-pad boundary.
- You can combine automated controller sequencers with manual MIDI controller knob lanes on the same performance page.
- If you automate the same CC number from multiple sources, the last-sent value wins at the MIDI receiver side (plan mappings accordingly).

Screenshots

Controller sequencer with curve editor, CC configuration, and playback indicator.

Arpeggiators

Arpeggiators are backend-run virtual instruments. They do not make sound by themselves. Instead, each arpeggiator listens on its own input MIDI channel, consumes notes sent to that channel, and generates arpeggiated notes on a selected target instrument channel.

This lets any note source in a performance drive an arpeggiator:

- melodic sequencers
- piano rolls
- drummer sequencers or external MIDI note sources
- any external controller routed through the selected session MIDI input

Routing

Each arpeggiator has two important channels:

- `Input Channel` : the virtual instrument channel that other devices play into
- `Target Channel` : the existing instrument channel that receives the generated arpeggiated notes

Input channels must be unique across arpeggiators. The target channel cannot be another arpeggiator input channel, so arpeggiator chaining is intentionally disabled for now.

When an arpeggiator is stopped, notes on its input channel are still consumed but no arpeggiated notes are emitted. This keeps the virtual-channel routing predictable.

Panel Header and Help

Each arpeggiator card uses a static device label such as `Arpeggiator 1` in the same header style as the other perform devices. The status badge plus `Start` / `Stop` and `Remove` controls sit in the card header, and the `?` button opens the arpeggiator-specific integrated help page in the current GUI language.

Running Behavior

Arpeggiators run on the backend while the instrument engine session is running. They are not tied to the multitrack arranger transport.

If an arpeggiator is enabled and notes are held on its input channel, it continues stepping according to the configured rate, gate, swing, and tempo. This means a piano roll or external keyboard can hold a chord and hear the arpeggiator run even when the arranger transport is stopped.

Presets and Settings

The arpeggiator panel includes built-in presets and lets you save user presets into the performance. Presets include the same kind of settings found on commercial arpeggiators:

- rate, gate, swing, octave range, and repeats
- patterns such as up, down, up/down, down/up, as-played, random, chord pulse, inside-out, and outside-in
- latch, restart behavior, probability, transpose, and scale quantize
- input, fixed, accent, or random velocity handling
- fixed velocity, accent cycle, velocity humanize, and timing humanize
- scale root, scale type, and mode for quantizing sources that do not provide their own scale

Built-in presets are always available. User-saved presets are stored inside the performance snapshot and move with exported/imported performances.

Scale and Mode

When a melodic sequencer or piano roll drives an arpeggiator, the backend receives that source's current scale/mode context and uses it for scale quantization.

When the source does not provide scale information, such as a drummer sequencer or external MIDI controller, the arpeggiator uses its own selected scale root, scale type, and mode.

Visualization

Each arpeggiator card shows:

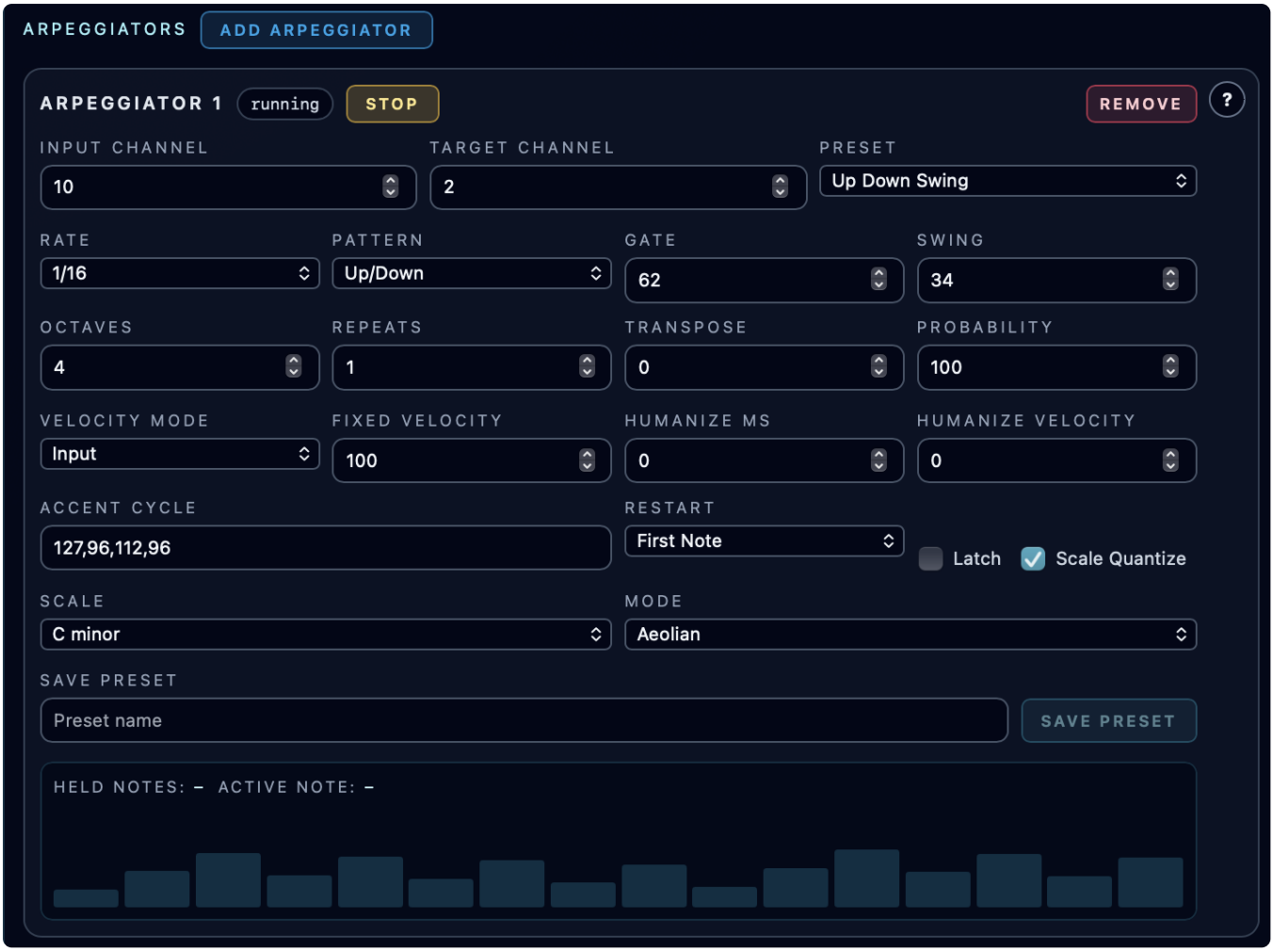
- held input notes
- the active generated note
- a 16-step activity display that highlights the current arpeggiator step while notes are held

The visualization follows backend runtime status, so it reflects the actual arpeggiator running in the audio session rather than a frontend-only preview.

Persistence

Saving a performance stores arpeggiator routing, enabled state, settings, and user presets. Transient runtime state such as currently held notes and active step is not persisted.

Screenshots



Arpeggiator panel with routing, preset controls, timing settings, and live activity display.

Piano Rolls

Piano Rolls provide manual performance input with scale-aware keyboard highlighting.

Purpose

Piano Rolls are for live/manual playing on an on-screen keyboard while the instrument engine is running.

They coexist with melodic sequencers and controller sequencers, which means you can jam manually on top of sequenced playback.

Starting a piano roll is independent from the multitrack arranger transport. If the instrument engine is stopped, `Start` brings up the engine so the keyboard can play without starting arrangement playback.

Add / Remove / Enable

Each piano roll supports:

- `Add Piano Roll`
- `Remove`
- `Start / Stop` (enable/disable that piano roll)

A piano roll only sends notes when:

- that piano roll is enabled
- instruments are running

Piano Roll Controls

Each piano roll has:

- `MIDI Channel (1..16)`
- `Scale`
- `Mode`

Scale/Mode Following Behavior During Live Playback

When the instrument engine is running and a piano roll is enabled, the piano roll can follow the currently running melodic sequencers for harmonic guidance.

Shared Scale/Mode Case

If running melodic sequencers agree on the same scale+mode:

- the piano roll shows that shared scale/mode
- keys are highlighted according to that theory

Mixed Scale/Mode Case

If running melodic sequencers disagree:

- piano roll controls show `mixed`
- highlighting is computed from the intersection of the active theories
- only notes common to the active scales/modes are highlighted

This is especially helpful when multiple tracks create potentially conflicting harmonic contexts.

Keyboard Range and Layout

The piano keyboard spans:

- `C1 .. B7` (84 notes)

UI behavior:

- Full-width horizontal keyboard section
- Scroll arrows appear when the keyboard overflows the viewport
- Smooth horizontal scrolling controls

Key Highlighting and Degree Labels

Highlighted keys show:

- in-scale status
- scale degree labels (`1..7`)
- rainbow degree coloring for quick visual recognition

In mixed-theory situations, a key can show combined degree labels (for example `2/5`) when it belongs to multiple active track theories.

Playing Notes (Pointer Interaction)

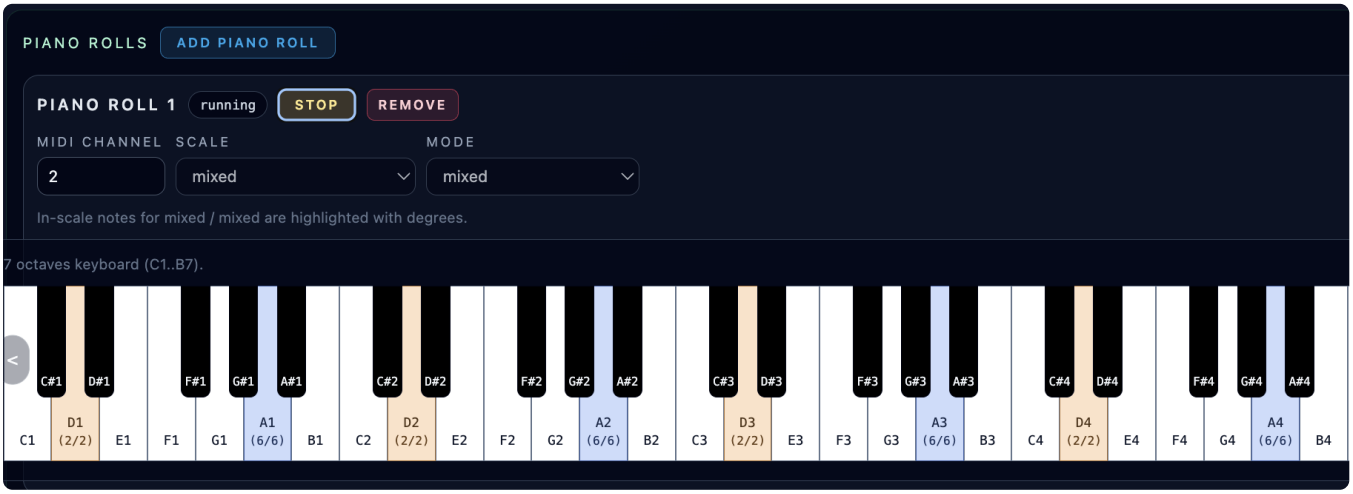
- Press/hold on a key sends `note_on`
- Release sends `note_off`
- Works for both white and black keys

The UI tracks held notes and clears them if the piano roll becomes inactive or the session stops.

Practical Use Cases

- Live melody or bass improvisation on top of sequencer patterns
- Testing an instrument patch in musical context without external MIDI hardware
- Performing harmonically guided lines while melodic sequencers define the key center

Screenshots



Piano roll with scale-aware key highlighting and mixed-theory display behavior.

MIDI Controllers

The MIDI Controllers panel provides manual CC control lanes with interactive knobs.

Overview

This panel is for manual MIDI Control Change performance (hands-on CC control), separate from controller sequencer automation.

Capacity

- Up to **6** controller lanes can be added

The `Add Controller` button is disabled after 6 lanes are present.

Per-Controller Lane Controls

Each controller lane includes:

- Controller name label (`Controller N` if unnamed)
- Running/stopped state badge
- `Controller #` field (`0..127`)
- `Start / Stop` enable toggle
- `Remove` button
- Knob control (value `0..127`)
- Numeric value display

Knob Interaction

The knob uses pointer drag interaction:

- Click and drag vertically to change the value
- Up increases the CC value
- Down decreases the CC value

The UI also shows a numeric value readout next to the knob for exact control.

Live Sending Behavior

When a controller lane is enabled and the instrument session is running:

- changing the knob sends MIDI `control_change` messages immediately

This is ideal for expressive live performance adjustments.

Manual Controllers vs Controller Sequencers

Use manual controller lanes when you want:

- direct, improvised control
- tactile-feeling UI knobs
- on-the-fly sweeps and tweaks

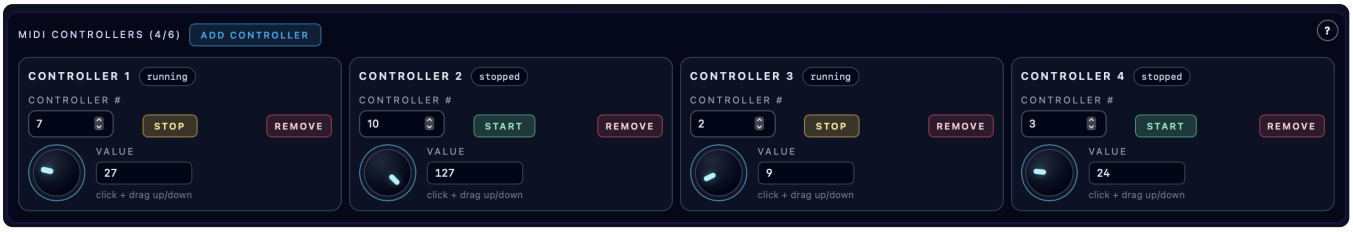
Use controller sequencers when you want:

- repeatable automation

- transport-synced movement
- curve-based modulation patterns

Both can exist in the same performance.

Screenshots



Manual MIDI controller lanes with knobs, CC numbers, and values.

Configuration

The Configuration chapter covers both UI-visible settings and practical runtime/deployment behaviors that affect how Orchestron is used.

What Is Covered

- GUI language switching and integrated help/docs behavior
- Config page engine settings (`sr` , control rate, buffers, derived `ksmps`) plus browser-clock latency controls when relevant
- MIDI setup and runtime input binding workflow
- Unified browser-clock playback and latency tuning from the Config page
- Internal loopback versus external host-bridge MIDI binding
- App-state persistence and default values

Chapter Contents

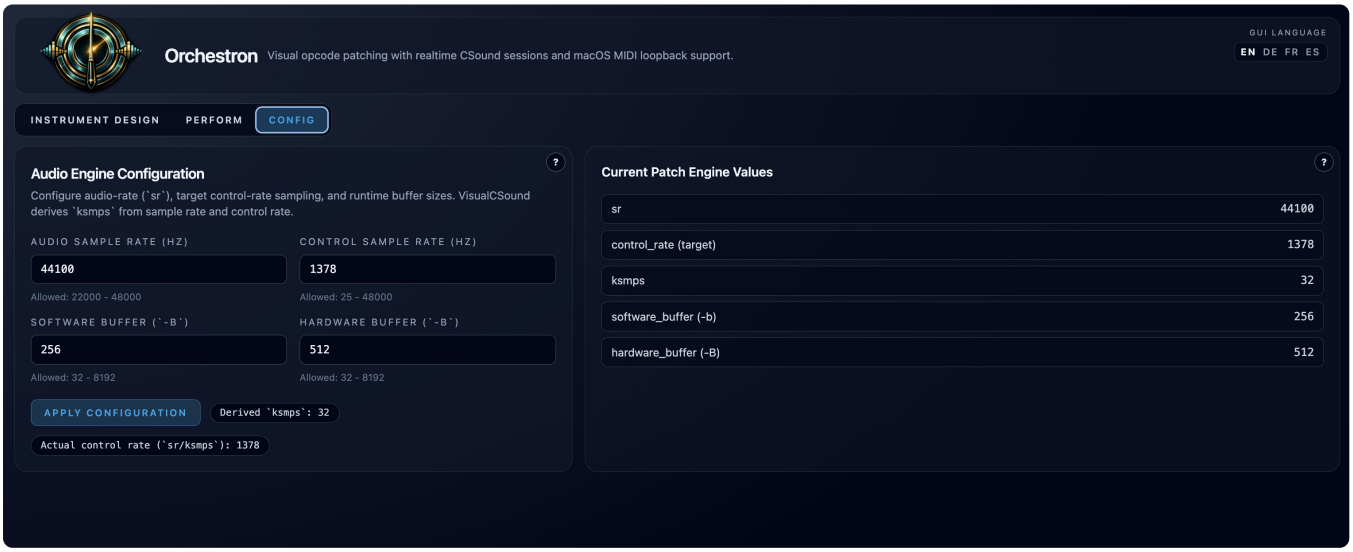
Configuration Scope Clarification

Orchestron has three kinds of configuration:

- **Patch-level configuration** (stored with a patch, editable in the Config page)
- **Runtime/startup configuration** (browser-clock startup options and optional host MIDI bridge token)
- **Workspace runtime tuning** (browser-clock latency settings stored in app state)

Both are important for successful use.

Screenshots



Configuration page overview of engine settings. The dedicated Browser-Clock Latency chapter shows the browser-clock-specific tuning section in detail.

Audio Engine Settings (Config Page)

The Config page controls both patch-level Csound engine settings and browser-clock latency settings for the current workspace.

Where These Settings Are Stored

Config page changes are stored in the current patch's `graph.engine_config`.

That means:

- different patches can use different engine settings
- switching patches can change the visible Config page values

Browser-clock latency settings are different:

- they apply to the active browser-clock session runtime
- they are stored in app state, not in the patch
- applying them updates the active browser-clock controller without changing patch data

Editable Settings (Config Page)

Patch Engine Section

Audio Sample Rate (`sr`)

- Field label: `Audio Sample Rate (Hz)`
- Allowed range: **22000 .. 48000**
- Integer only

Control Sample Rate (`control_rate` target)

- Field label: `Control Sample Rate (Hz)`
- Allowed range: **25 .. 48000**
- Integer only

This is the target control-rate value used to derive `ksmps`.

Software Buffer (`-b`)

- Field label: `Software Buffer (-b)`
- Allowed range: **32 .. 8192**
- Integer only

Hardware Buffer (`-B`)

- Field label: `Hardware Buffer (-B)`
- Allowed range: **32 .. 8192**
- Integer only

Derived Values Shown In The UI

The Config page previews derived values before applying:

- Derived `ksmps`
- Actual control rate (`sr / ksmps`)

This helps you see the real runtime effect of your target control-rate choice.

Current Patch Engine Values (Read-Only Panel)

The right-side panel shows the normalized engine values stored in the patch:

- `sr`
- `control_rate`
- `ksmps`
- `software_buffer`
- `hardware_buffer`

Use this panel to verify what will be used at compile/start time.

Browser-Clock Latency Section

The Config page shows an additional section for browser-owned audio latency tuning whenever a browser-clock session is active.

See Browser-Clock Latency for the unified browser-clock workflow explanation and the full UI screenshot of this section.

Editable fields:

- steady queue low water / high water (`ms`)
- startup queue low water / high water (`ms`)
- underrun recovery boost / maximum underrun boost (`ms`)
- maximum blocks per render request
- steady / startup / recovery parallel request limits
- immediate note render blocks
- immediate note render cooldown (`ms`)

These values control the browser PCM queue depth and render request behavior, which are the dominant latency/stability knobs on the unified browser-clock path.

The patch engine section still matters because `sr` , `ksmps` , and the configured buffers are used by the backend render worker when it compiles and starts the headless Csound session.

Field-By-Field Meaning

Steady Low Water / Steady High Water

- These define the normal browser PCM queue target after startup is complete.
- Lower values reduce live-play latency.
- Higher values make clicks and underruns less likely.

Startup Low Water / Startup High Water

- These define the larger queue target used while the stream is still priming after connect or after an underrun recovery phase.
- They are usually kept above the steady values so the stream can stabilize before dropping into lower-latency playback.

Underrun Recovery Boost / Max Underrun Boost

- When the browser detects underruns, VisualCSound temporarily adds extra queue headroom.
- `Underrun Recovery Boost` controls how much extra buffer is added per underrun event.
- `Max Underrun Boost` limits how far that temporary recovery buffer can grow.

Max Blocks Per Request

- Caps the maximum backend render chunk size requested in one browser-clock render request.
- Smaller values reduce head-of-line delay for live notes.
- Larger values reduce request overhead but can make live interaction feel less immediate.

Steady / Startup / Recovery Parallel Requests

- These limits control how many render requests may remain in flight at once.
- More parallelism can keep the queue full more aggressively.
- Too much parallelism can also create larger bursts and less predictable latency.

Immediate Note Render Blocks / Immediate Note Render Cooldown

- A live `note_on` sends a small urgent render request in addition to the normal refill logic.
- `Immediate Note Render Blocks` sets the size of that urgent render burst.
- `Immediate Note Render Cooldown` throttles how often these urgent requests can be sent.
- These controls help manual piano playing feel more responsive, but values that are too aggressive can reintroduce clicks.

Practical Adjustment Order

1. Start with `steady low/high water`.
2. If clicks remain, raise the steady or startup watermarks slightly.
3. If latency still feels too high, reduce `max blocks per request` carefully.
4. Only then tune immediate note render and parallel request limits.

Validation Behavior

The page validates:

- integer format
- allowed range
- high-water fields must be greater than their matching low-water fields

`Apply Configuration` is disabled until all visible values are valid.

Apply Configuration

`Apply Configuration` writes the new engine settings into the current patch state.

Practical effect:

- future compile/start operations for the patch use the updated engine config
- the patch must still be saved if you want the change persisted in the patch library

`Apply Browser-Clock Settings` writes the browser-clock latency settings into app state and refreshes the active browser-clock controller if one is connected.

Other Engine Fields Present But Not Edited In UI

The patch engine config also contains values such as:

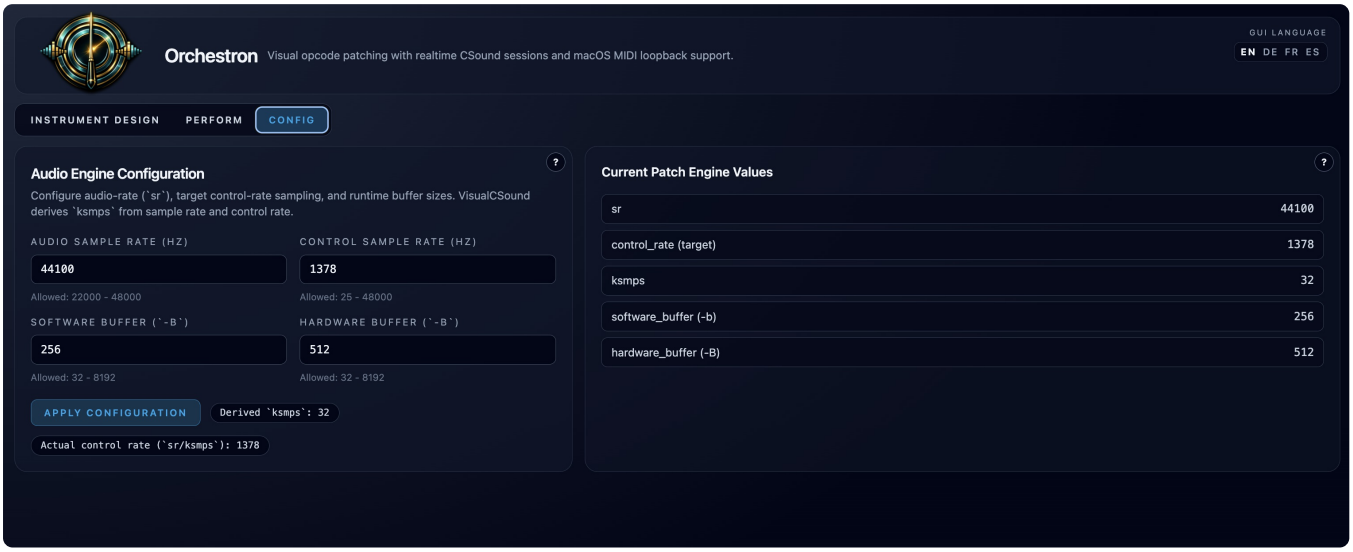
- `nchnls` (default `2`)
- `0dbfs` (default `1`)

These are part of the patch model, but the current Config page focuses on the user-facing timing and buffer controls listed above.

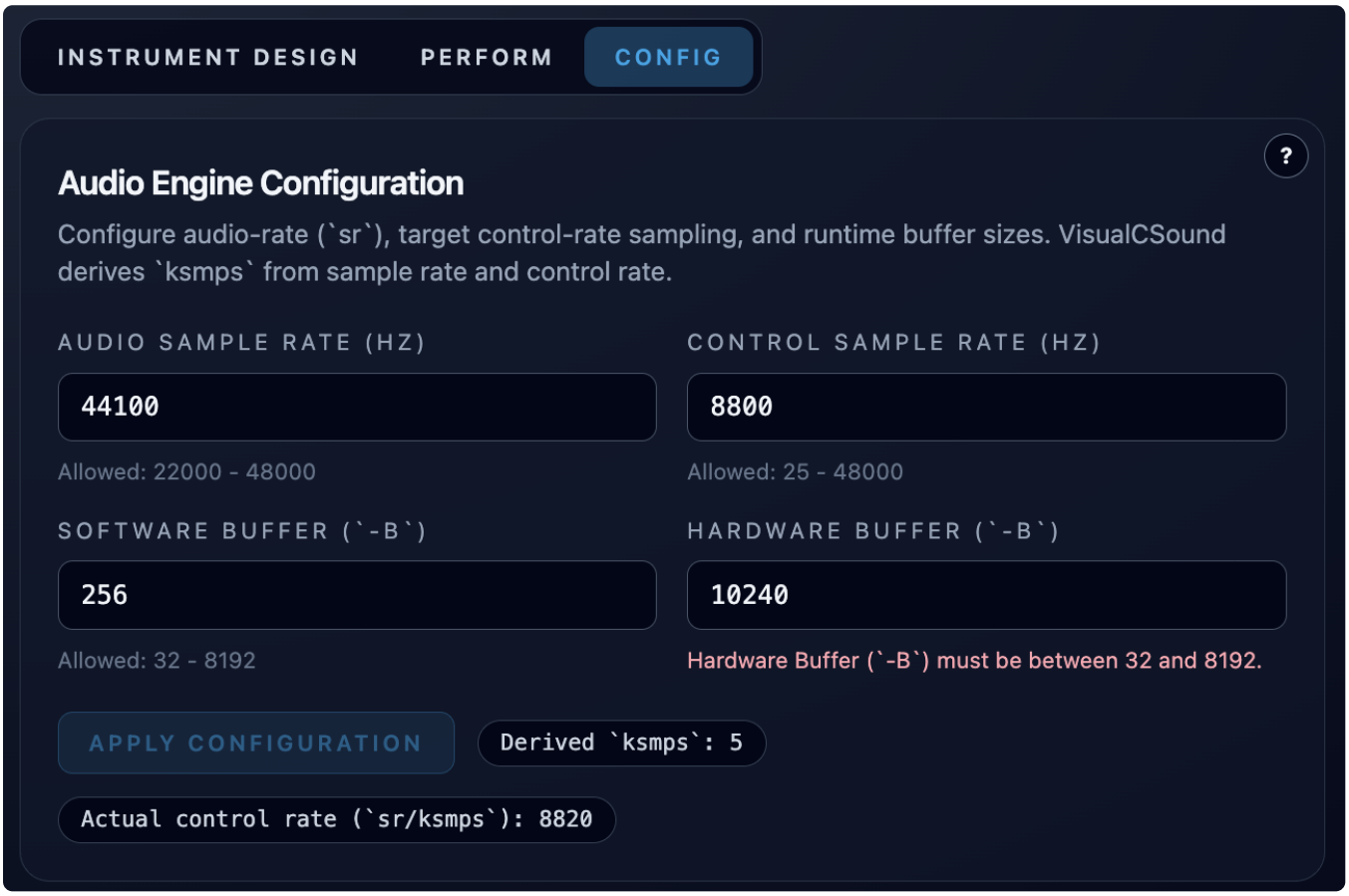
Tuning Tips

- Lower buffers reduce latency but increase glitch risk.
- Higher buffers improve stability but increase latency.
- Use the Runtime panel and live testing to find a stable setting for your machine and patch complexity.
- On the browser-clock path, start by adjusting browser-clock low/high water, render blocks, and parallel request limits before changing patch `-b/-B`.
- For live MIDI sessions, prefer `ksmps <= 32`. The runtime warns when a live session starts with a larger `ksmps` because MIDI execution is still quantized to the k-period.

Screenshots



Config page audio engine settings in a valid state.



Config page validation messages with invalid values entered.

The dedicated Browser-Clock Latency chapter shows the browser-clock tuning section in detail.

Persistence and Defaults

Orchestron persists both library data (patches/performances) and a separate app-state snapshot (workspace state).

Persistent JSON documents are bounded before storage so direct API clients cannot grow request memory or the SQLite database without limit. Defaults are 8 MiB for app state (`VISUALCSOUND_APP_STATE_MAX_BYTES`), 4 MiB for saved patch graphs (`VISUALCSOUND_PATCH_GRAPH_MAX_BYTES`), 1 MiB for patch UI layout metadata (`VISUALCSOUND_PATCH_UI_LAYOUT_MAX_BYTES`), 8 MiB for performance configs (`VISUALCSOUND_PERFORMANCE_CONFIG_MAX_BYTES`), and 64 KiB for any single nested JSON string (`VISUALCSOUND_PERSISTED_JSON_STRING_MAX_BYTES`).

Two Persistence Layers (Important)

1. Library Data (Explicit Save)

These are stored when you click save actions:

- Patches (Instrument Design `Save`)
- Performances (Perform page `Save Performance`)

2. App-State Snapshot (Automatic)

Orchestron also persists a workspace/app-state snapshot in the backend to restore your working context after reload.

Persisted app state includes (current implementation):

- active page (`Instrument Design / Perform / Config`)
- GUI language
- instrument tabs (including editable patch snapshots)
- active instrument tab
- sequencer state (tracks, controller sequencers, arpeggiators, piano rolls, MIDI controllers)
- performance workspace metadata (`currentPerformanceId` , name, description)
- sequencer instrument bindings (stable rack instance IDs)
- explicit audio routes, Master selection, insert ownership and mixer strip/send values (app-state version 2)
- active MIDI input selection reference
- browser-clock latency settings for the current workspace/runtime path

New sessions default that MIDI input reference to `internal:loopback` unless a different external input is explicitly bound.

What This Means In Practice

- You can reload the app and continue from a similar workspace state.
- Unsaved edits may reappear because they exist in the app-state snapshot.
- Unsaved edits are **not** the same as saved library data.

Always use explicit save actions when you want stable, shareable library entries.

Default New Patch Values

New opens the built-in/template chooser. The selected starter supplies its name, graph, Activation and audio interface. Choose Empty patch for an empty graph (`nodes = []` , `connections = []`). All starters use these default engine settings:

- `sr = 48000`
- `control_rate = 1500`
- `ksmps = 32`
- `nchnls = 2`
- `software_buffer = 128`

- `hardware_buffer = 512`
- `0dbfs = 1`

Saved mixes and older performances

Save Performance, Load Performance, Clone and native performance bundles retain version 11 audio routing and mixer settings, including Master, dedicated inserts, sends and pre/post selection. App-state restore also retains the current mix. These are snapshots; recording mixer automation is not included.

Opening versions 1–10 converts the old Level values to audio gain in dB and resolves legacy connections into explicit routes. Missing Level means 0 dB. The migration notice explains that Level no longer scales MIDI velocity, so velocity-sensitive instruments may change timbre. Check the mix and save the migrated performance. See Mixer persistence and compatibility.

Compile Status vs Persistence

The compile status badge (`compiled` , `pending changes` , `errors`) indicates compile state for the current patch snapshot. It does **not** indicate whether the patch has been saved to the patch library.

Runtime Defaults

- runtime audio mode is fixed to `browser_clock`
- the default session MIDI binding is `internal:loopback`
- external host MIDI is disabled unless `VISUALCSOUND_HOST_MIDI_TOKEN` is configured and a helper connects
- runtime session creation is bounded by `VISUALCSOUND_SESSION_MAX_ACTIVE` , `VISUALCSOUND_SESSION_MAX_ACTIVE_PER_CLIENT` , `VISUALCSOUND_SESSION_CREATE_RATE_PER_MINUTE` , and `VISUALCSOUND_SESSION_CREATE_RATE_BURST` ; stopped idle sessions are deleted after `VISUALCSOUND_SESSION_IDLE_TIMEOUT_SECONDS`
- session event WebSocket observers are bounded by `VISUALCSOUND_SESSION_EVENT_WS_MAX_SUBSCRIPTIONS_TOTAL` , `VISUALCSOUND_SESSION_EVENT_WS_MAX_SUBSCRIPTIONS_PER_SESSION` , `VISUALCSOUND_SESSION_EVENT_WS_CONNECT_RATE_PER_MINUTE` , and `VISUALCSOUND_SESSION_EVENT_WS_CONNECT_RATE_BURST`

Recommended Habits

1. Use app-state restore for convenience.
2. Use `Save` / `Save Performance` for intentional versions.
3. Use `Export` bundles for backups or sharing across machines.